# JAVA

Docente:
Bernardo Alonso Mira García



Documentacion Spring data

https://docs.spring.io/spring-data/jpa/docs/current/reference/html/

https://www.aluracursos.com/blog/que-es-maven

https://www.aluracursos.com/blog/rest-concepto-y-fundamentos

Encriptador de texto

https://youtu.be/oQgt8BF2wAg

https://youtu.be/DVjp-TJihAM

https://www.openapis.org/

Hotel alura

https://youtu.be/1luEBNCweG4

https://youtu.be/8y5fqKVfDWU

Conversor de monedas

https://youtu.be/uPlaC3VL3HA



Estructura del Curso

1. Android Studio
2. Fundamentos de Java
3. Git & Git hub
4. Agil para desarrollo Móvil



# Desarrollo Móvil en Cifras

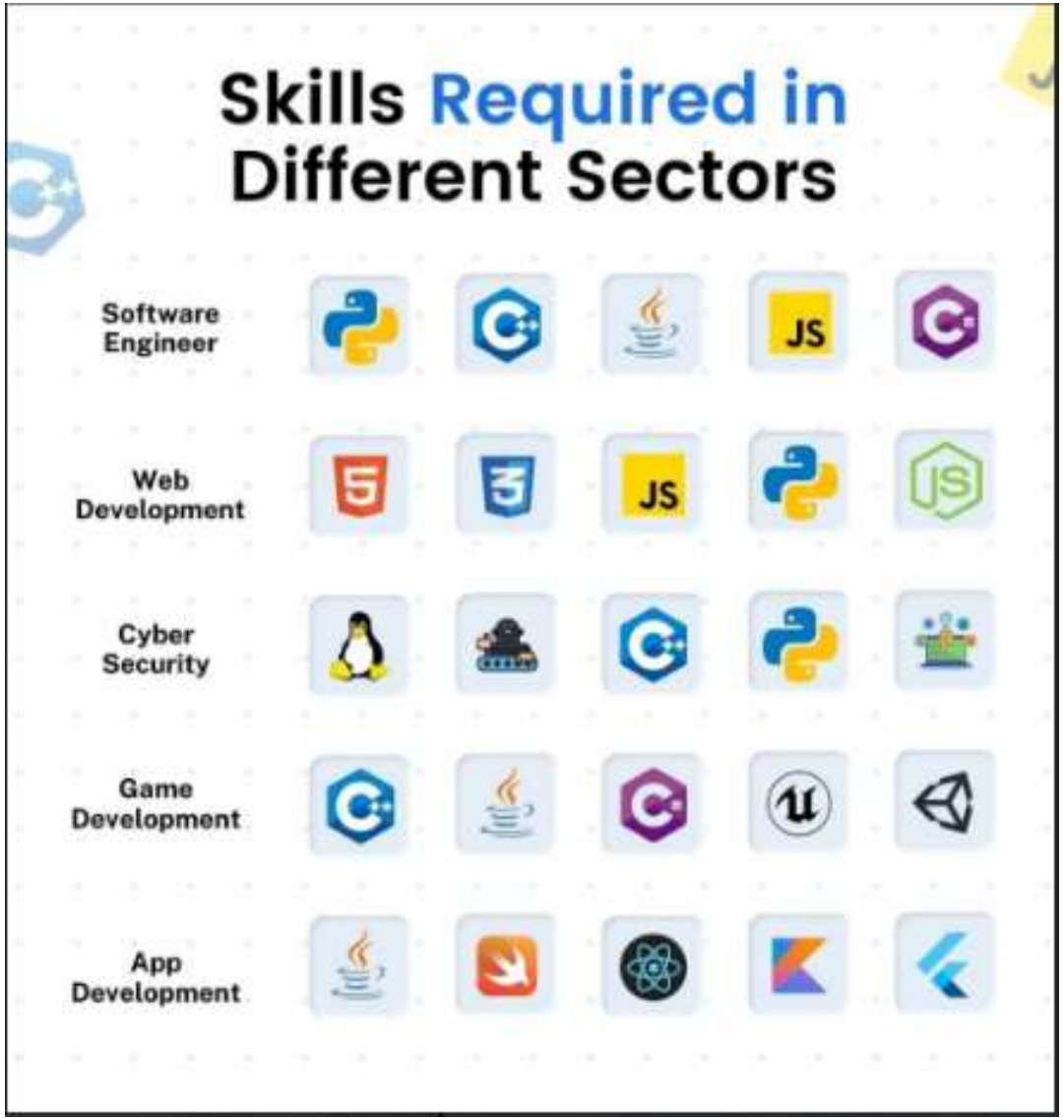
- Para el año 2021 se calculaban mas de 6000 millones de dispositivos en el mundo.
- Se estiman mas de 8.9 millones de aplicaciones desarrollada al año 2020 .
- 230 mil millones de descargas de aplicaciones móviles al 2022.
- Existen, al menos, 70 tiendas de Apps.
- Este mercado contribuye con más de 4,6 Billones de dólares anuales al PIB mundial.
- Un usuario en promedio revisa su teléfono celular al menos 150 veces al día.
- El 89% de las veces, un usuario recomienda una marca que conoció a través de un sitio que experimento a través de su móvil.
- 8 de cada 10 publicistas encuentra a su consumidor a través de los datos demográficos que provee la ubicación móvil.
- Las personas pueden pasar hasta 2/3 de su tiempo en aplicaciones móviles, esto es mas común en personas de 18 a 24 años.
- El grupo de las mujeres de 18 a 24 años , son los usuarios que mas prefieren el acceso a internet a través del móvil.

https://emma.io/blog/25-estadisticas-de-mobile-app-marketing-2021/#:~:text=El%20n%C3%BAmero%20total%20de%20aplicaciones,8%2C9%20millones%20en%202020





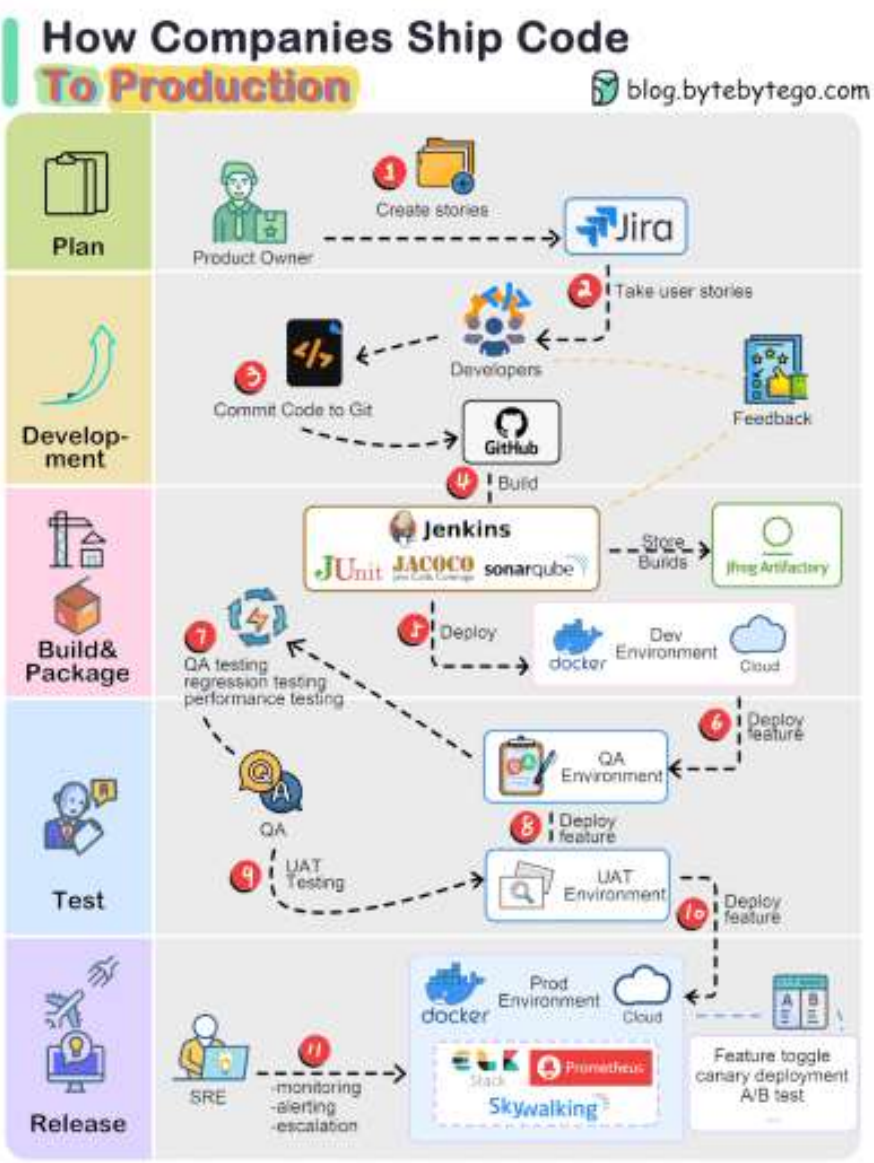

Lo mínimo que debes conocer como desarrollador


Skills Required in Different Sectors
Software Engineer
Web Development
Cyber Security
Game Development
App Development
JS
JS


TALK
<CODE>
TO ME

# Como se despliega código a producción

La siguiente gráfica muestra como las compañías despliegan el código a producción siguiendo un proceso lógico de planeación , desarrollo , construcción de paquetes , testing y lanzamiento.



# SOLID

نور عمرد

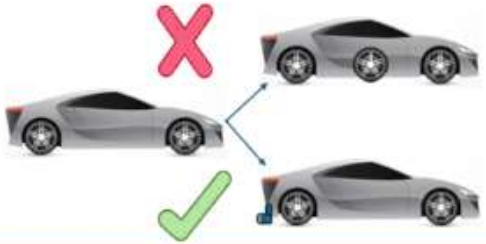
## Single Responsibility




## Open-Closed

Open for extension

Closed for modification



## Liskov Substitution



Can't replace its parent in eating meat.

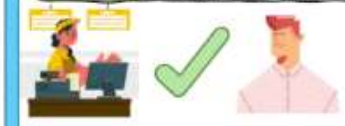


## Interface Segregation

Why are you forcing me to take gum?



Sorry, you will not be forced to take anything you don't need.



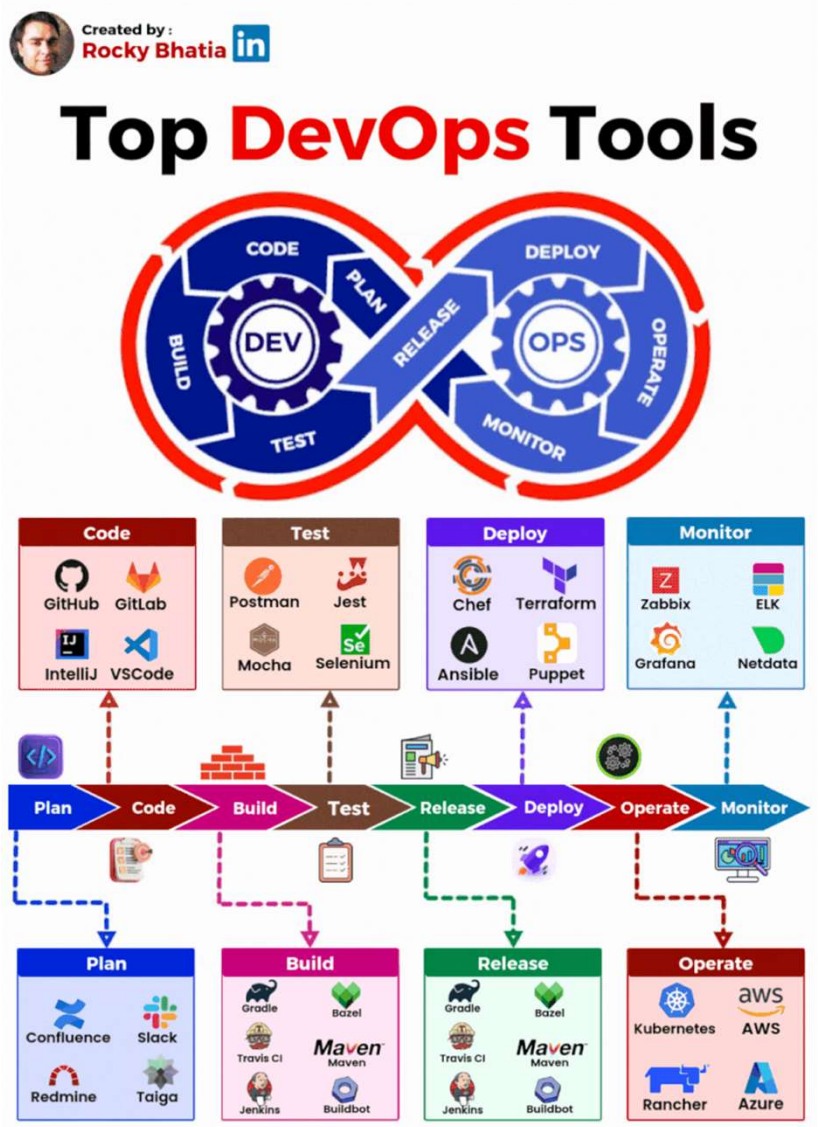
## Dependency Inversion

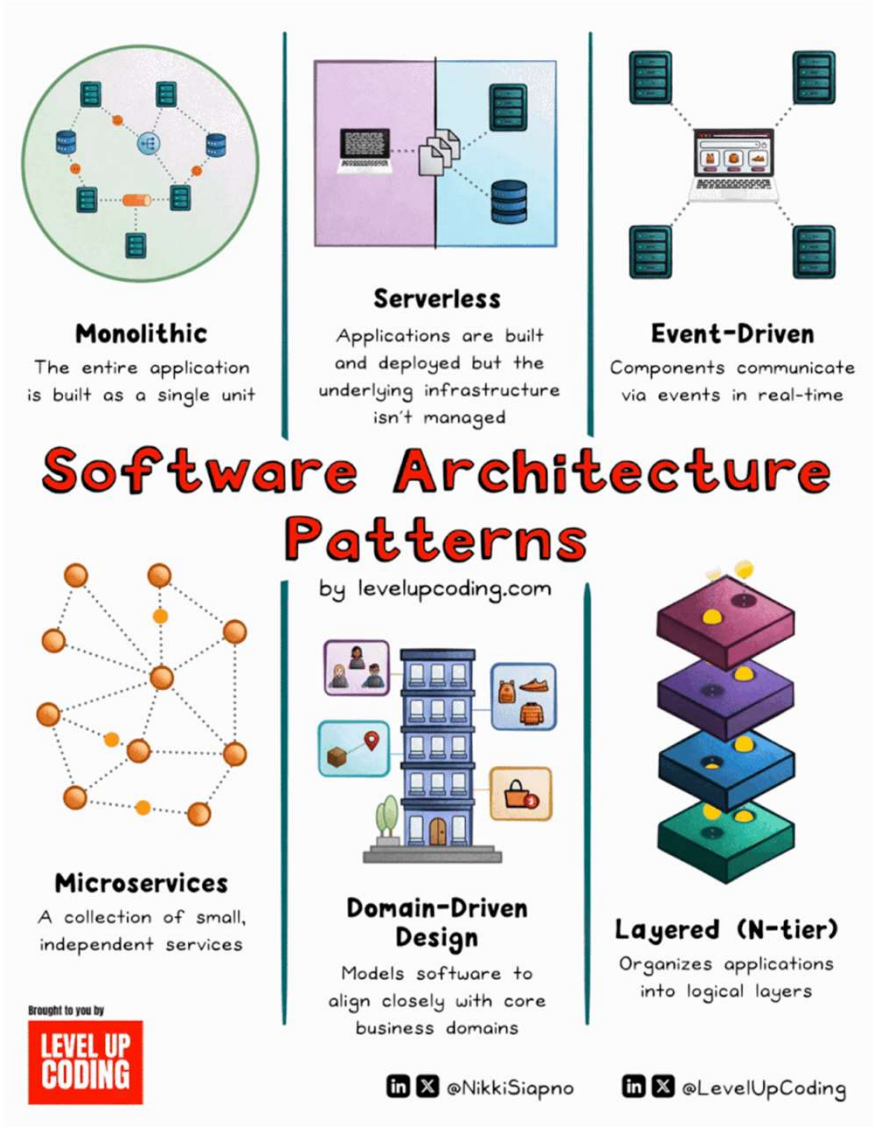
I can only code with JAVA

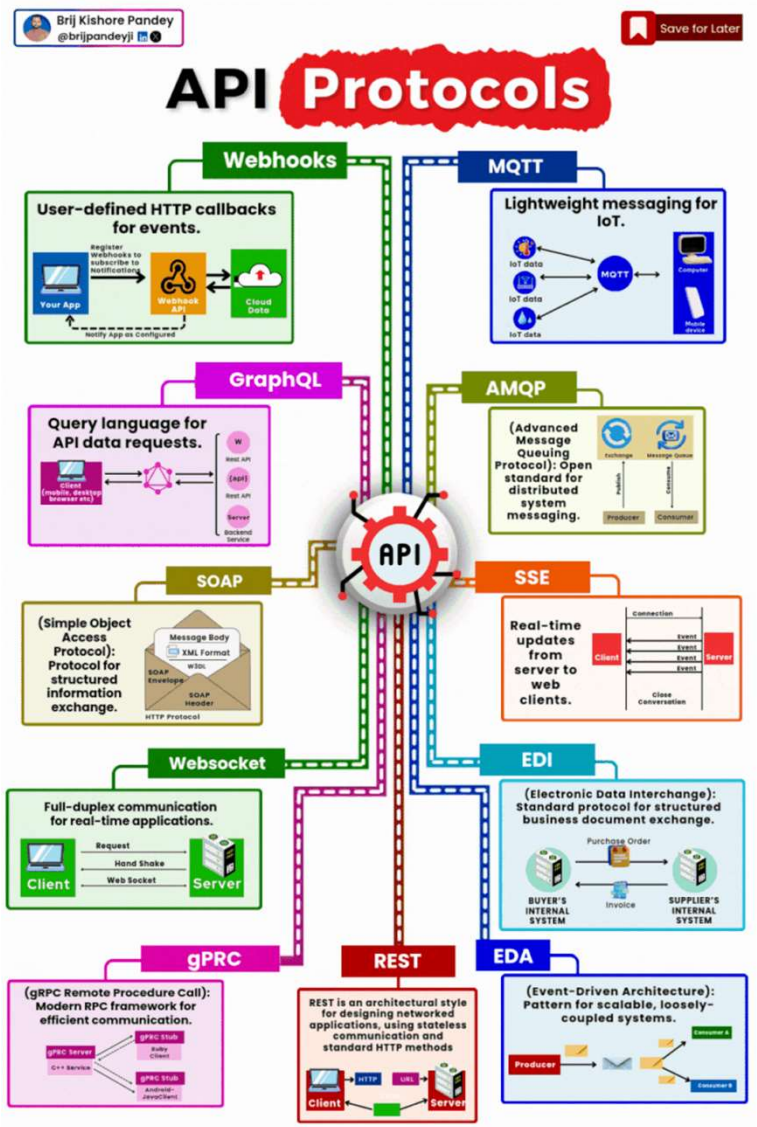


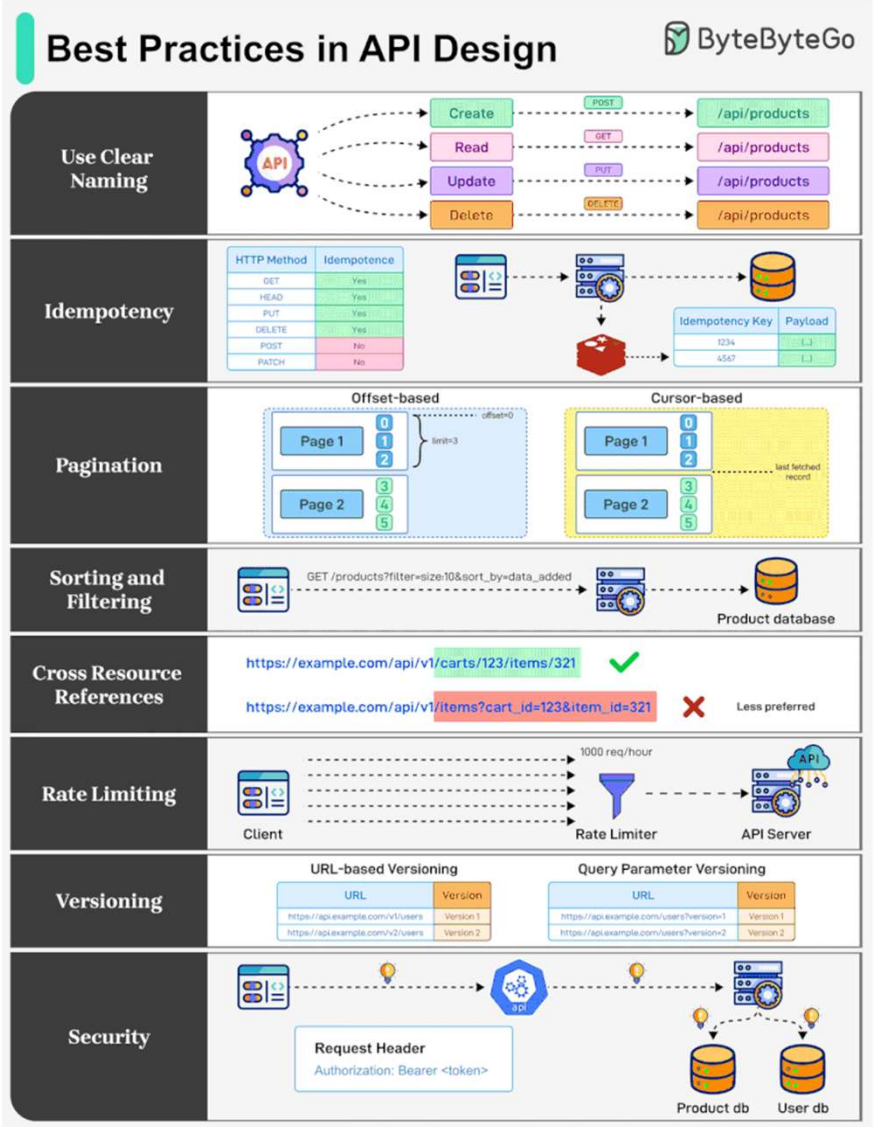
I can code with any programming language

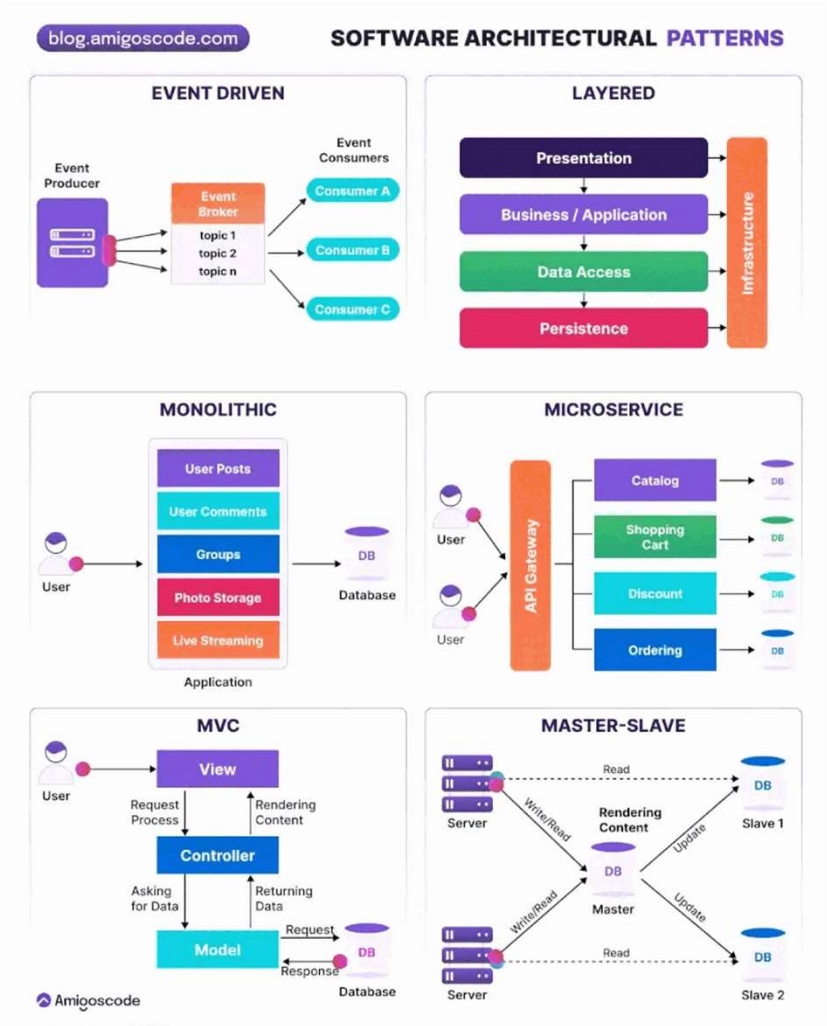


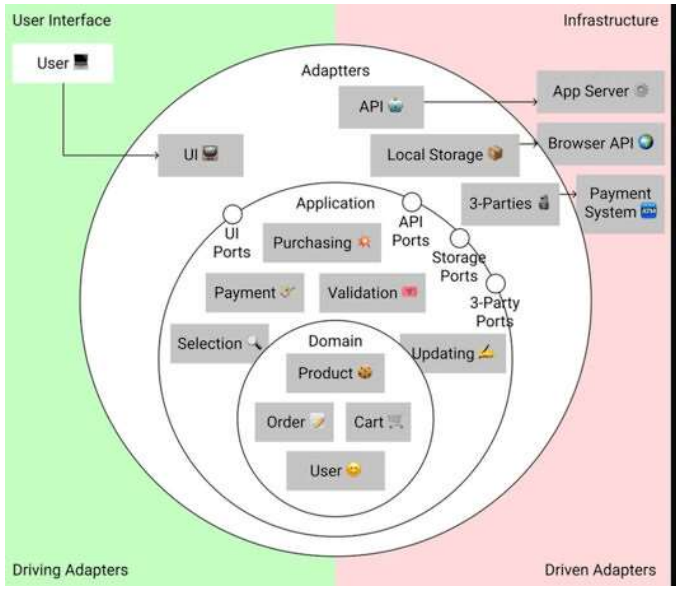

Created by :
Rocky Bhatia
Top DevOps Tools
CODE
PLAN
BUILD
TEST
RELEASE
DEPLOY
OPERATE
MONITOR
DEV
OPS
Code
GitHub
GitLab
IntelliJ
VSCode
Test
Postman
Jest
Mocha
Selenium
Deploy
Chef
Terraform
Ansible
Puppet
Monitor
Zabbix
ELK
Grafana
Netdata
Plan
Code
Build
Test
Release
Deploy
Operate
Monitor
Plan
Confluence
Slack
Redmine
Taiga
Build
Gradle
Bazel
Travis CI
Maven
Jenkins
Buildbot
Release
Gradle
Bazel
Travis CI
Maven
Jenkins
Buildbot
Operate
Kubernetes
AWS
Rancher
Azure


Monolithic
The entire application is built as a single unit
Serverless
Applications are built and deployed but the underlying infrastructure isn't managed
Event-Driven
Components communicate via events in real-time
Software Architecture Patterns
by levelupcoding.com
Microservices
A collection of small, independent services
Domain-Driven Design
Models software to align closely with core business domains
Layered (N-tier)
Organizes applications into logical layers
Brought to you by
LEVEL UP CODING
@NikkiSiapno
@LevelUpCoding


Brij Kishore Pandey
@brijpandeyji
Save for Later
API Protocols
Webhooks
User-defined HTTP callbacks for events.
Register Webhooks to subscribe to notifications
Your App
Webhook API
Cloud Data
Notify App as Configured
MQTT
Lightweight messaging for IoT.
IoT data
IoT data
IoT data
MQTT
Computer
Mobile device
GraphQL
Query language for API data requests.
Client (mobile, desktop, browser etc.)
Rest API
{API}
Rest API
Server
Backend Service
AMQP
(Advanced Message Queuing Protocol): Open standard for distributed system messaging.
Exchange
Message Queue
Publish
Consume
Producer
Consumer
API
SOAP
(Simple Object Access Protocol): Protocol for structured information exchange.
Message Body
XML Format
W3C
SOAP Envelope
SOAP Header
HTTP Protocol
SSE
Real-time updates from server to web clients.
Connection
Client
Server
Event
Event
Event
Event
Close Conversation
Websocket
Full-duplex communication for real-time applications.
Client
Request
Hand Shake
Web Socket
Server
EDI
(Electronic Data Interchange): Standard protocol for structured business document exchange.
Purchase Order
BUYER'S INTERNAL SYSTEM
Invoice
SUPPLIER'S INTERNAL SYSTEM
gRPC
(gRPC Remote Procedure Call): Modern RPC framework for efficient communication.
gRPC Server
C++ Service
gRPC Stub
Ruby Client
gRPC Stub
Android-JavaClient
REST
REST is an architectural style for designing networked applications, using stateless communication and standard HTTP methods
Client
HTTP
URL
Server
EDA
(Event-Driven Architecture): Pattern for scalable, loosely-coupled systems.
Producer
Consumer A
Consumer B


Best Practices in API Design
ByteByteGo
Use Clear Naming
API
Create
Read
Update
Delete
POST
GET
PUT
DELETE
/api/products
/api/products
/api/products
/api/products
Idempotency
HTTP Method
Idempotence
GET
HEAD
PUT
DELETE
POST
PATCH
Yes
Yes
Yes
Yes
No
No
Idempotency Key
Payload
1234
4567
Pagination
Offset-based
Cursor-based
Page 1
Page 2
0
1
2
3
4
5
offset=0
limit=3
last fetched record
Sorting and Filtering
GET /products?filter=size:10&sort_by=data_added
Product database
Cross Resource References
https://example.com/api/v1/carts/123/items/321
https://example.com/api/v1/items?cart_id=123&item_id=321
Less preferred
Rate Limiting
1000 req/hour
Client
Rate Limiter
API
API Server
Versioning
URL-based Versioning
Query Parameter Versioning
URL
Version
https://api.example.com/v1/users
Version 1
https://api.example.com/v2/users
Version 2
https://api.example.com/users?version=1
Version 1
https://api.example.com/users?version=2
Version 2
Security
API
Request Header
Authorization: Bearer <token>
Product db
User db




```
src/
|_domain/
  |_user.ts
  |_product.ts
  |_order.ts
  |_cart.ts
|_application/
  |_addToCart.ts
  |_authenticate.ts
  |_orderProducts.ts
  |_ports.ts
|_services/
  |_authAdapter.ts
  |_notificationAdapter.ts
  |_paymentAdapter.ts
  |_storageAdapter.ts
  |_api.ts
  |_store.tsx
|_lib/
|_ui/
```

# Design Patterns Developer Need To Know

## Junior

**Why learn patterns?**

These four patterns help you create, organize, and extend objects efficiently.
Master these, and you'll write cleaner, more flexible code from day one.

**Builder**
- Build complex objects in clear, small steps.
- Keep object creation separate from the object itself.
- Make objects valid before they are used.

**Factory Method**
- Hide object creation details from the user.
- Make code easy to extend without changing it.

**Singleton**
- Only one instance exists in the whole program.
- Share it across the system
- Create it only when it is first needed.

**Decorator**
- Add new features without touching old code.
- Combine many small changes in flexible ways.
- Avoid deep and complex class trees.

## Middle

**Why learn patterns?**

These patterns teach you how to swap behaviors, simplify complex systems, and make your code more modular.

**Strategy**
- Swap different algorithms without changing the caller.
- Remove big if/else or switch statements.
- Test each algorithm on its own.

**Adapter**
- Connect classes with different interfaces.

**Abstract Factory**
- Create a family of objects

**Template Method**
- Let subclasses change small parts of base class

**Facade**
- Hide details of a complex system under a simple API

**Bridge**
- Split abstraction from its implementation

**Command**
- Wrap an action in an object. Allow undo, redo

**Mediator**
- Reduce direct links between classes by using a man-in-the-middle

## Senior

**Why learn patterns?**

These patterns give you superpowers for working with complex structures, dynamic behaviors, and resource optimization.

**Prototype**
- Copy objects without rebuilding them.
- Keep the state when making a copy.
- Avoid complex new logic.

**Composite**
- Handle tree-like structures easily. Treat one item and a group the same way.

**Chain of Responsibility**
- Pass a request along a list of handlers.

**State**
- Put each state's logic in its own class.

**Flyweight**
- Share data to save memory.

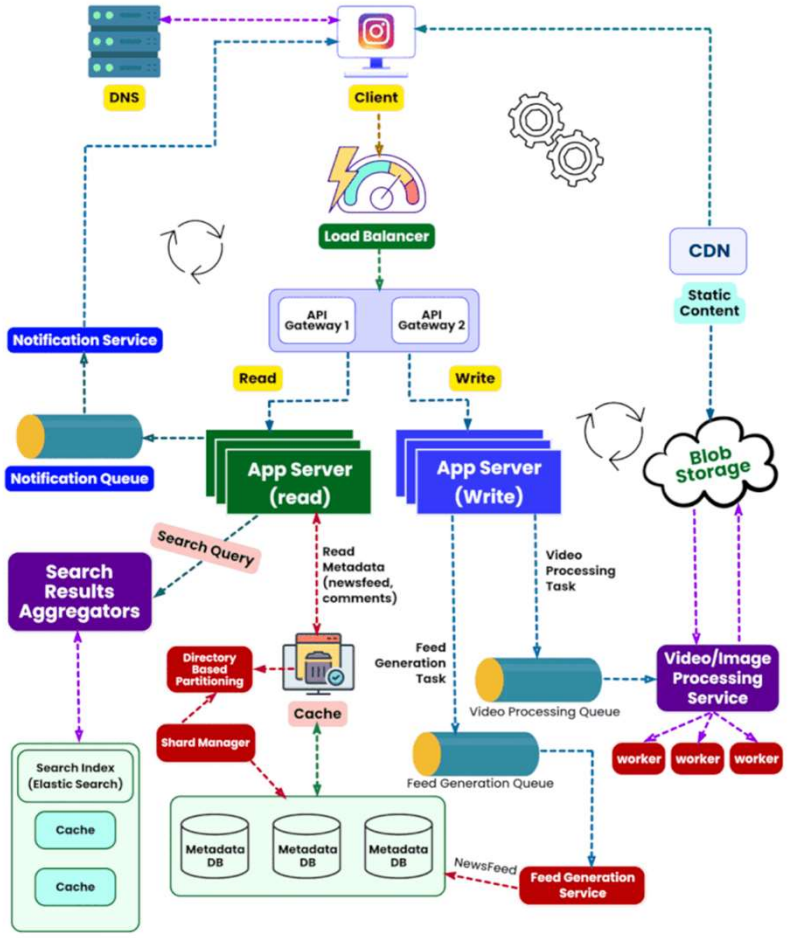
**Proxy**
- Control access to another object. Hide complex or remote operations

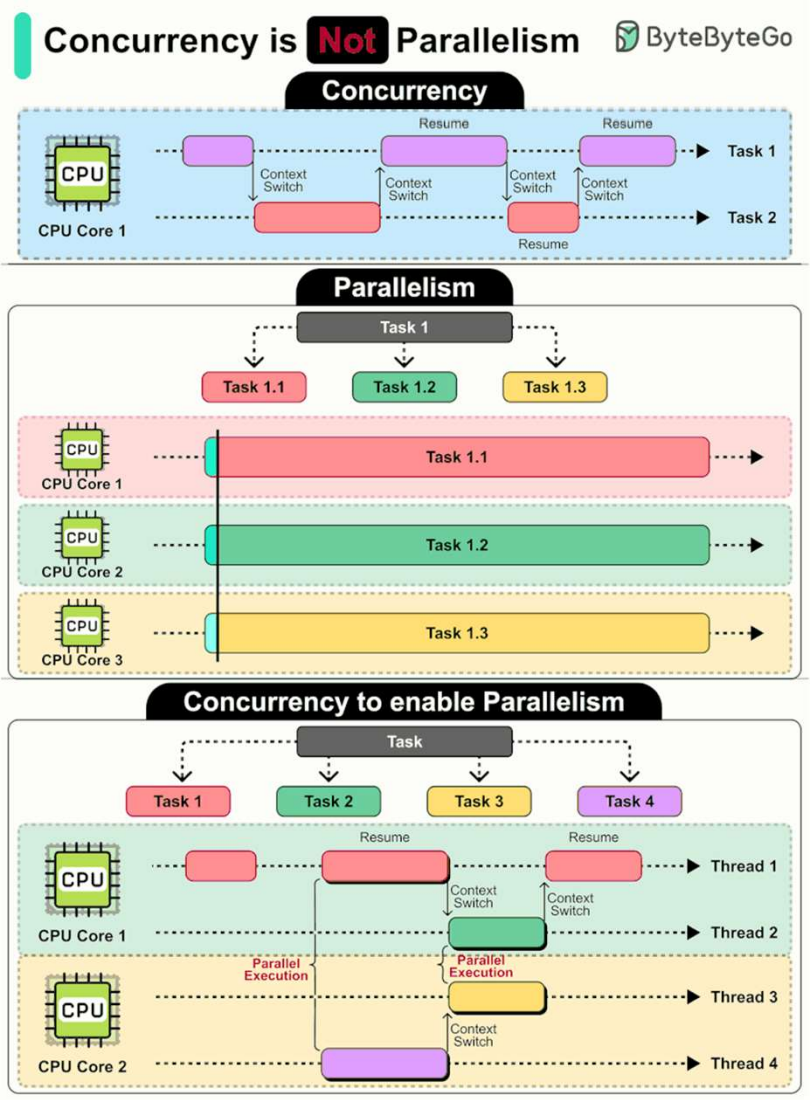
**Visitor**
- Add new actions without changing classes.

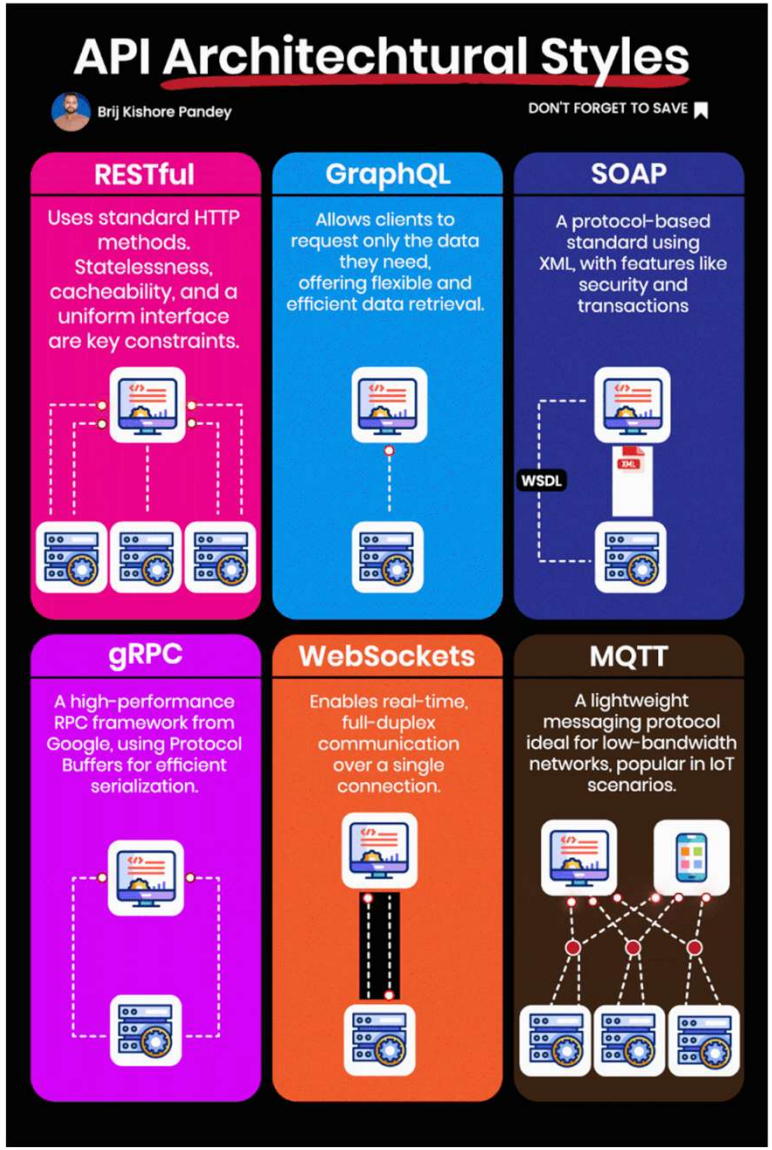
**Interpreter**
- Create and run a small language or rule set.

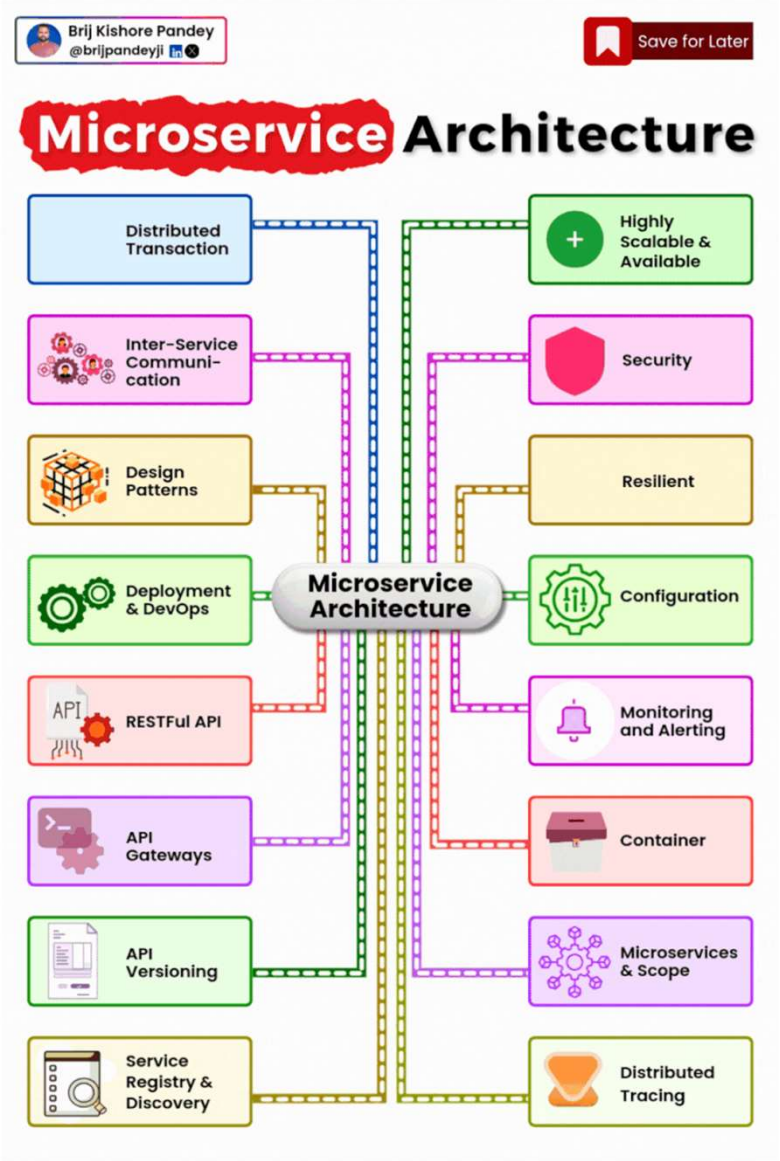
# Instagram System Design

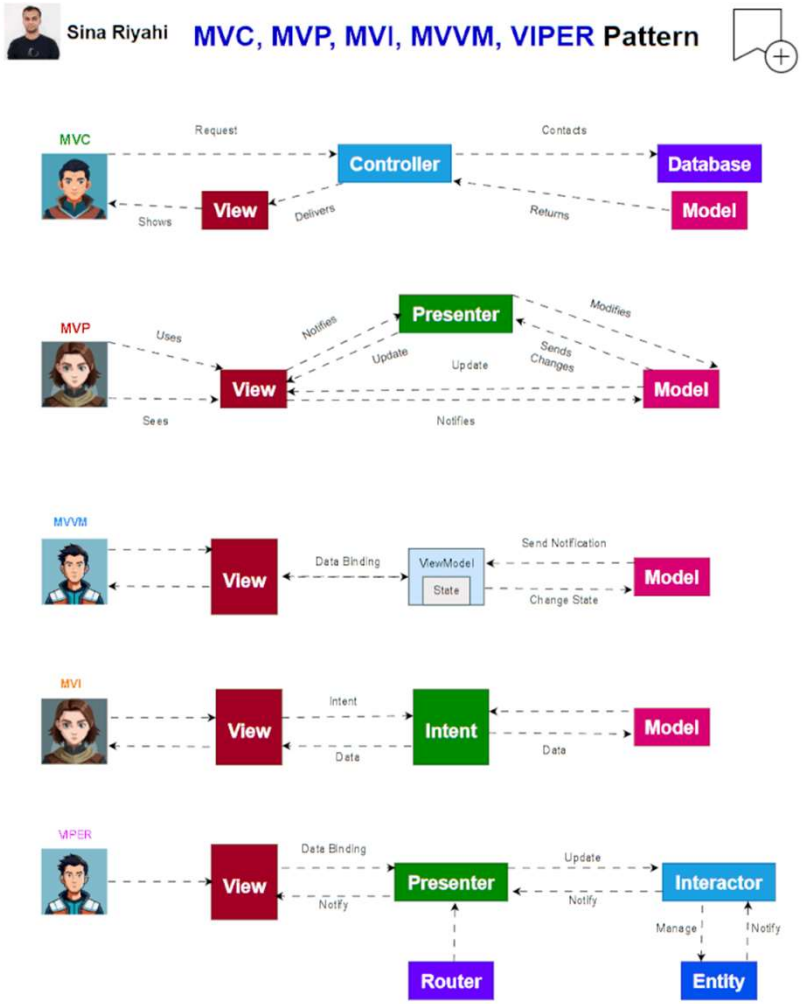
Created by : Rocky Bhatia

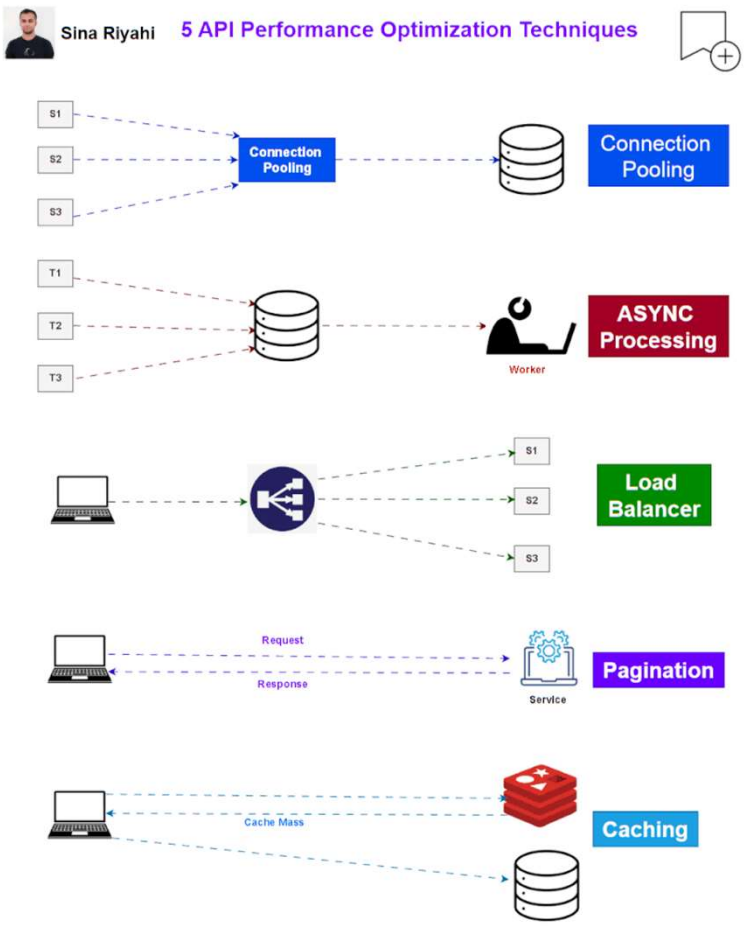



Concurrency is Not Parallelism
ByteByteGo
Concurrency
CPU
CPU Core 1
Resume
Resume
Context Switch
Context Switch
Context Switch
Context Switch
Task 1
Task 2
Resume
Parallelism
Task 1
Task 1.1
Task 1.2
Task 1.3
CPU Core 1
CPU Core 2
CPU Core 3
Concurrency to enable Parallelism
Task
Task 1
Task 2
Task 3
Task 4
Resume
Resume
Thread 1
Thread 2
Thread 3
Thread 4
Parallel Execution
Parallel Execution
Context Switch


API Architechtural Styles
Brij Kishore Pandey
DON'T FORGET TO SAVE
RESTful
Uses standard HTTP methods. Statelessness, cacheability, and a uniform interface are key constraints.
GraphQL
Allows clients to request only the data they need, offering flexible and efficient data retrieval.
SOAP
A protocol-based standard using XML, with features like security and transactions
WSDL
gRPC
A high-performance RPC framework from Google, using Protocol Buffers for efficient serialization.
WebSockets
Enables real-time, full-duplex communication over a single connection.
MQTT
A lightweight messaging protocol ideal for low-bandwidth networks, popular in IoT scenarios.


Brij Kishore Pandey
@brijpandeyji
Save for Later
Microservice Architecture
Distributed Transaction
Inter-Service Communi-cation
Design Patterns
Deployment & DevOps
RESTFul API
API Gateways
API Versioning
Service Registry & Discovery
Highly Scalable & Available
Security
Resilient
Configuration
Monitoring and Alerting
Container
Microservices & Scope
Distributed Tracing
Microservice Architecture


Sina Riyahi
MVC, MVP, MVI, MVVM, VIPER Pattern
MVC
Request
Contacts
Controller
Database
View
Model
Shows
Delivers
Returns
MVP
Presenter
Uses
Notifies
Modifies
Update
Update
Sends Changes
View
Model
Sees
Notifies
MVVM
Send Notification
Data Binding
ViewModel
State
View
Model
Change State
MVI
Intent
View
Intent
Model
Data
Data
VIPER
Data Binding
Update
View
Presenter
Interactor
Notify
Notify
Manage
Notify
Router
Entity




# QA: Pruebas de Caja Negra, Blanca y Gris

## CAJA NEGRA

– El tester no conoce el código interno.
– Se centra en entradas y salidas, validando que el sistema cumpla los requisitos funcionales.
– Ejemplo: rellenar un formulario y verificar el resultalo

## CAJA BLANCA

– El tester conoce el código, arquitectura y lógica interna.
– Busca asegurar que todas las rutas condiciones y bucles se ejecuten correctamente.
– Ejemplo: pruebas unitarias revisando flujos internos

## CAJA GRIS

– Enfoque intermedio: se conoce parte del funcionamiento interno.
– Combina la visión funcional con la revisión de aspectos técnicos clave.
– Ejemplo: probar una API sabiendo cómo esta estructurada la base de datos.

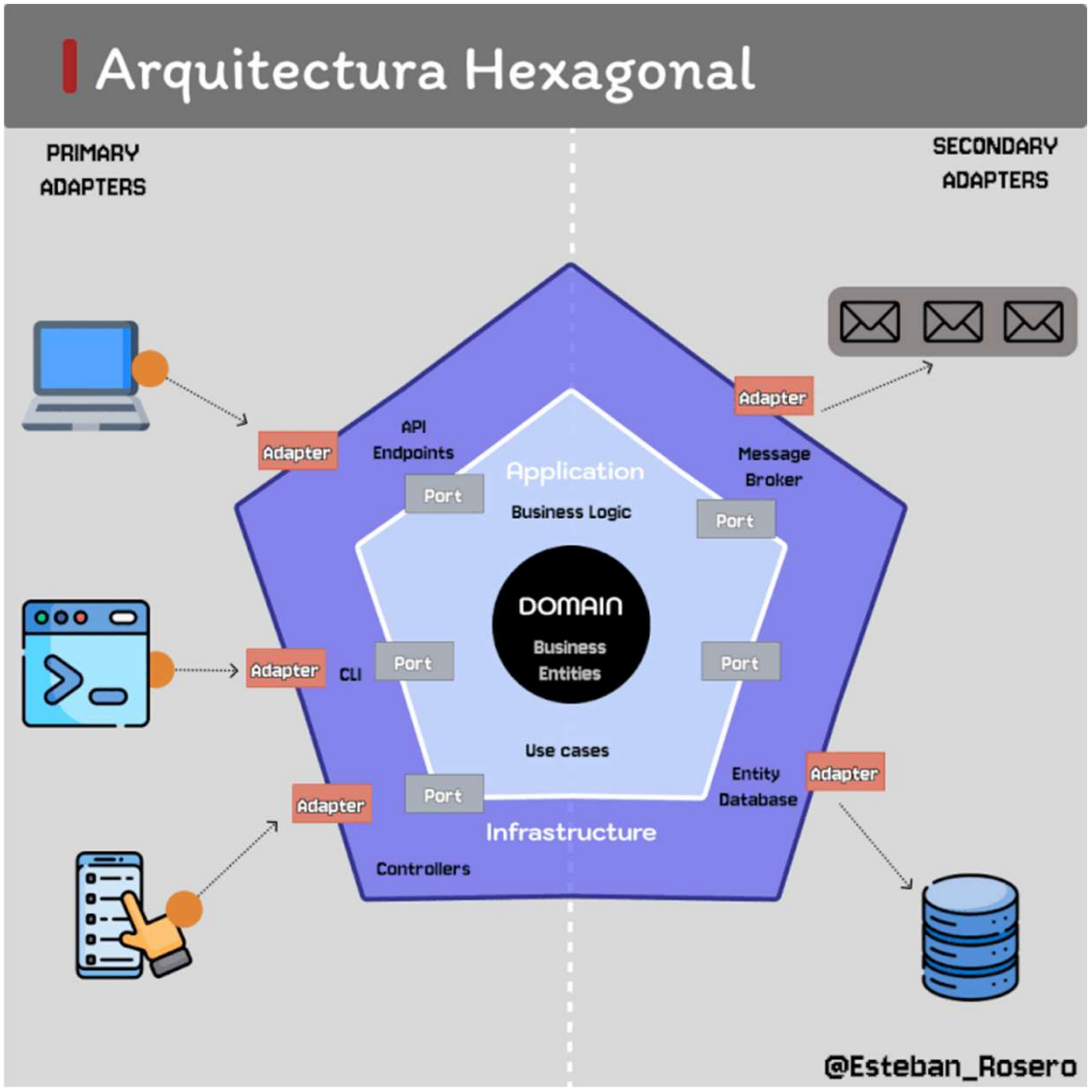



```java
@Slf4j
@RestController
@RequiredArgsConstructor
public class ContactController {

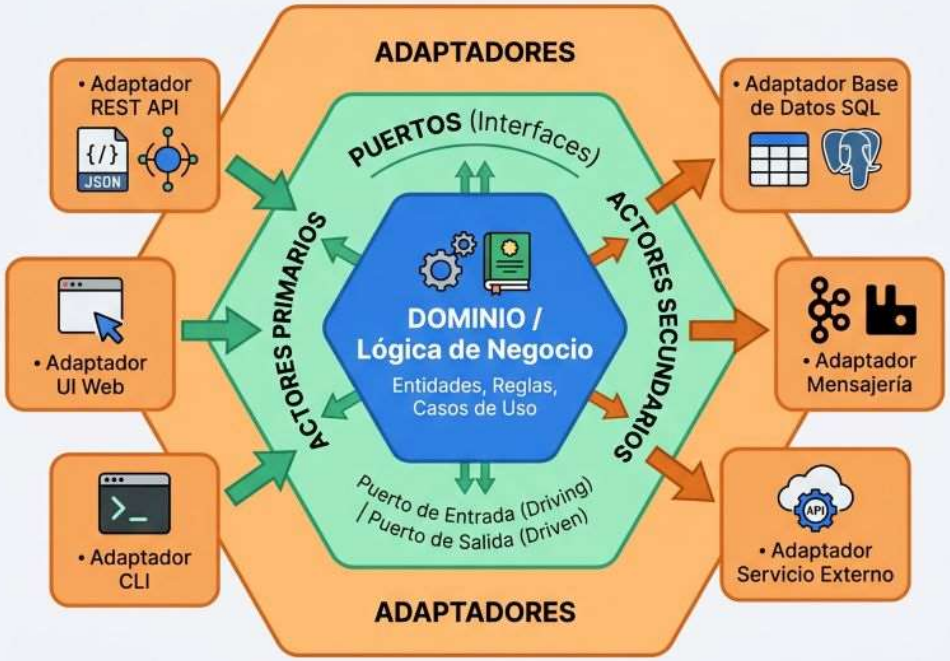
    @PostMapping("/contacts")
    public ContactDto createContactV1() {
        ContactDto result = new ContactDto();
        return result;
    }

    @PostMapping("/contacts")
    public ResponseEntity<ContactDto> createContactV2() {
        ContactDto result = new ContactDto();
        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(result);
    }

}
```




# ARQUITECTURA HEXAGONAL (PUERTOS Y ADAPTADORES)




## Ejemplo Práctico: E-commerce




Usuario hace pedido → Adaptador REST → Puerto OrderService → Caso de Uso "CrearPedido"

"CrearPedido" → Puerto RepositoryPort → Adaptador PostgreSQL → Guarda en BD

"CrearPedido" → Puerto NotificationPort → Adaptador Email → Notifica usuario

**INTERCAMBIABLE**
Cambias BD sin tocar el dominio

**TESTEABLE**
Testeas el núcleo con mocks

**DESACOPLADO**
Lógica aislada de infraestructura

# Master These 20 System Design Concepts Before Your FAANG Interview

DesignGurus.io

## Scalability



Designing systems that grow smoothly with user demand.

## Load Balancing



Distributes traffic across servers to prevent overload.

## Caching



Stores frequently accessed data in memory for faster responses.

## Data Modeling



Designing how data is structured before choosing DB types.

## Database Sharding



Splits large databases to improve performance and scale.

## Partitioning



Divides data across servers to boost throughput and efficiency.

## Replication



Duplicates data for high availability and fault tolerance.

## CAP Theorem



Understanding trade-offs between Consistency, Availability & Partition Tolerance.

## Eventual Consistency



Data in distributed systems becomes consistent over time.

## Message Queues



Enables async communication between services without blocking.

## API Gateway



Central point to manage and route external/internal API requests.

## WebSockets



Maintains a two-way connection for real-time communication (e.g., chat, live feeds).

## Microservices



Splits apps into loosely coupled services for flexibility and scale.

## Service Discovery



Enables microservices to find each other dynamically.

## Rate Limiting



Protects your system by controlling request rates.

## Fault Tolerance



Keeps your system running even when parts fail.

## Monitoring



Tracks performance and health with logs and metrics.

## Content Delivery Networks




Speeds up load times via geographically distributed servers.

## Authentication & Authorization



Verifies users and controls their access.

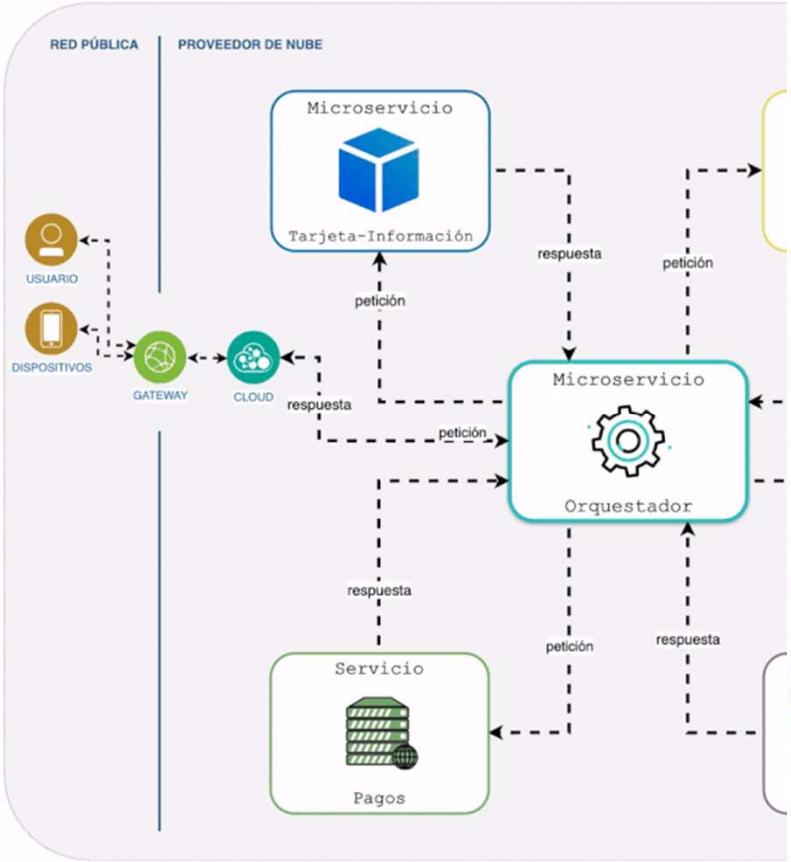
## Consistent Hashing



Distributes load evenly, especially when servers join or leave.

# ORQUESTACIÓN





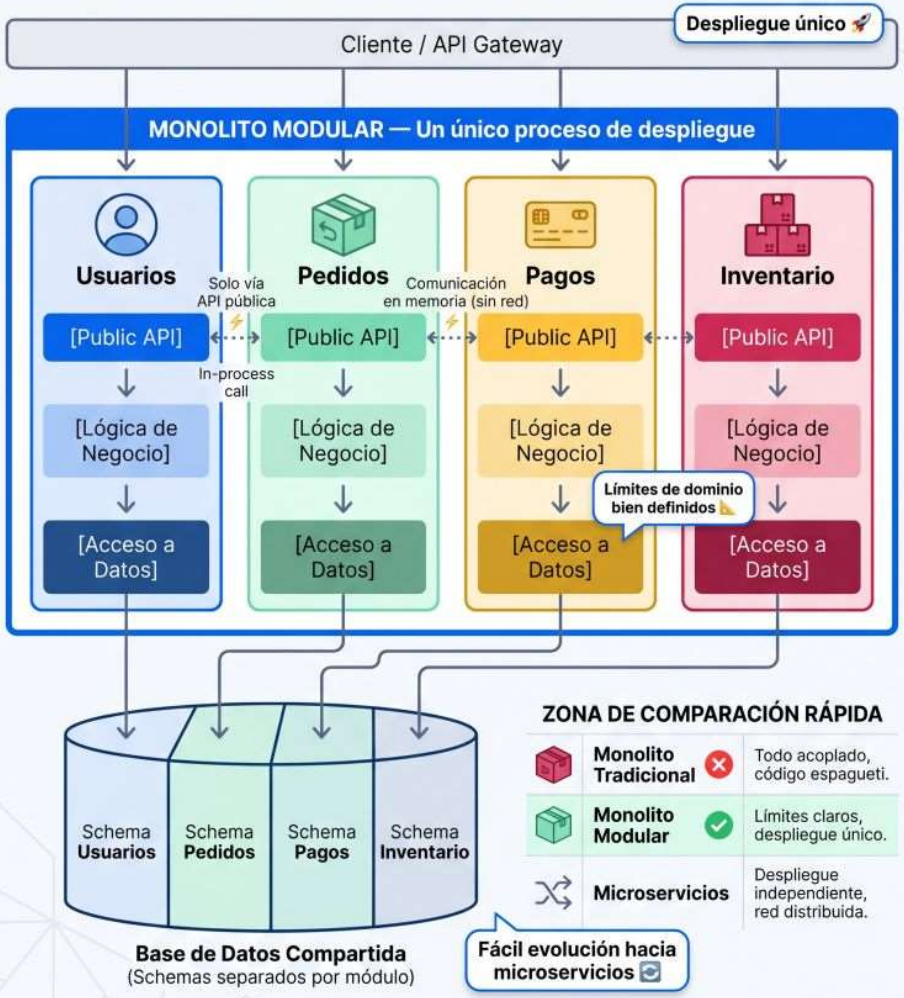

Microservicios




**Alexis Ahumada**
Senior App Developer

# MONOLITO MODULAR

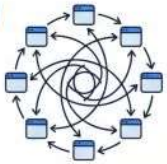
## Un único proceso de despliegue




# 9 Señales de que tu Arquitectura está Sobre-Ingenierizada

¿Estás construyendo un cohete para ir a la tienda?

1



**Microservicios para todo**

Incluso para un Hola Mundo

2



**Nadie entiende el sistema**

Ni el equipo que lo construyó

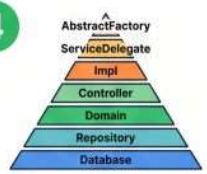
3



**Deploys que tardan horas**

Un hotfix tarda más que el build inicial

4




**3 capas de abstracción para un CRUD**

AbstractFactoryServiceDelegateImpl

5



**Kubernetes para 10 usuarios**

Infraestructura de NASA, tráfico de blog personal

6



**Más reuniones que código**

Arquitectura discutida, nada implementado

7



**Onboarding de semanas**

El README tiene 400 páginas

8



**Cada cambio rompe 5 cosas**

El acoplamiento oculto se cobra facturas

9

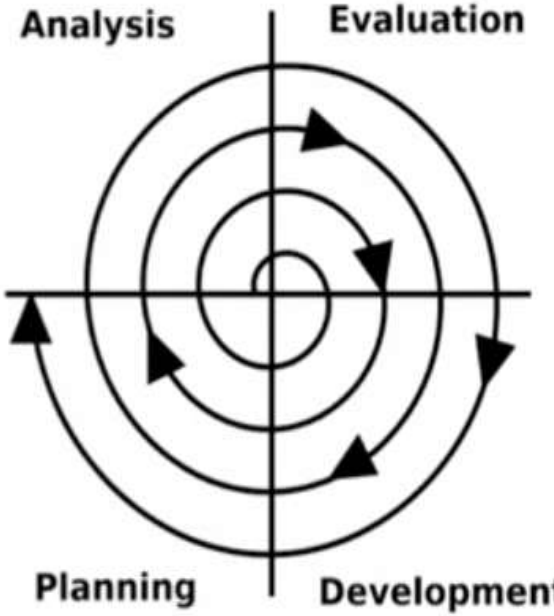


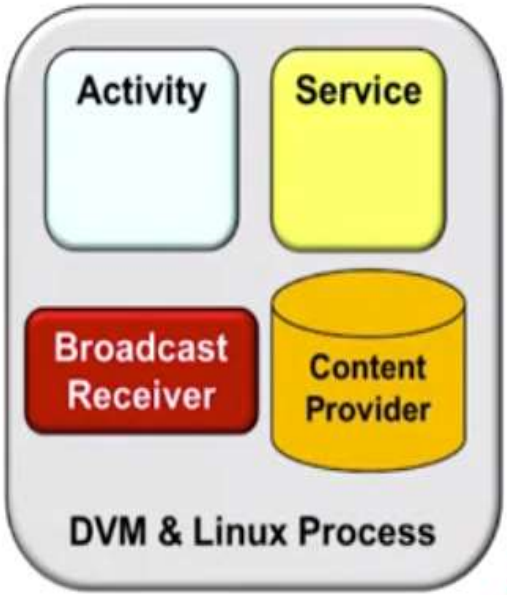
**La solución es más compleja que el problema**

Máquina enorme para una tarea trivial

Simplicidad es la máxima sofisticación — aplica en arquitectura de software también.

# Stack para Desarrollo Móvil en Android Studio





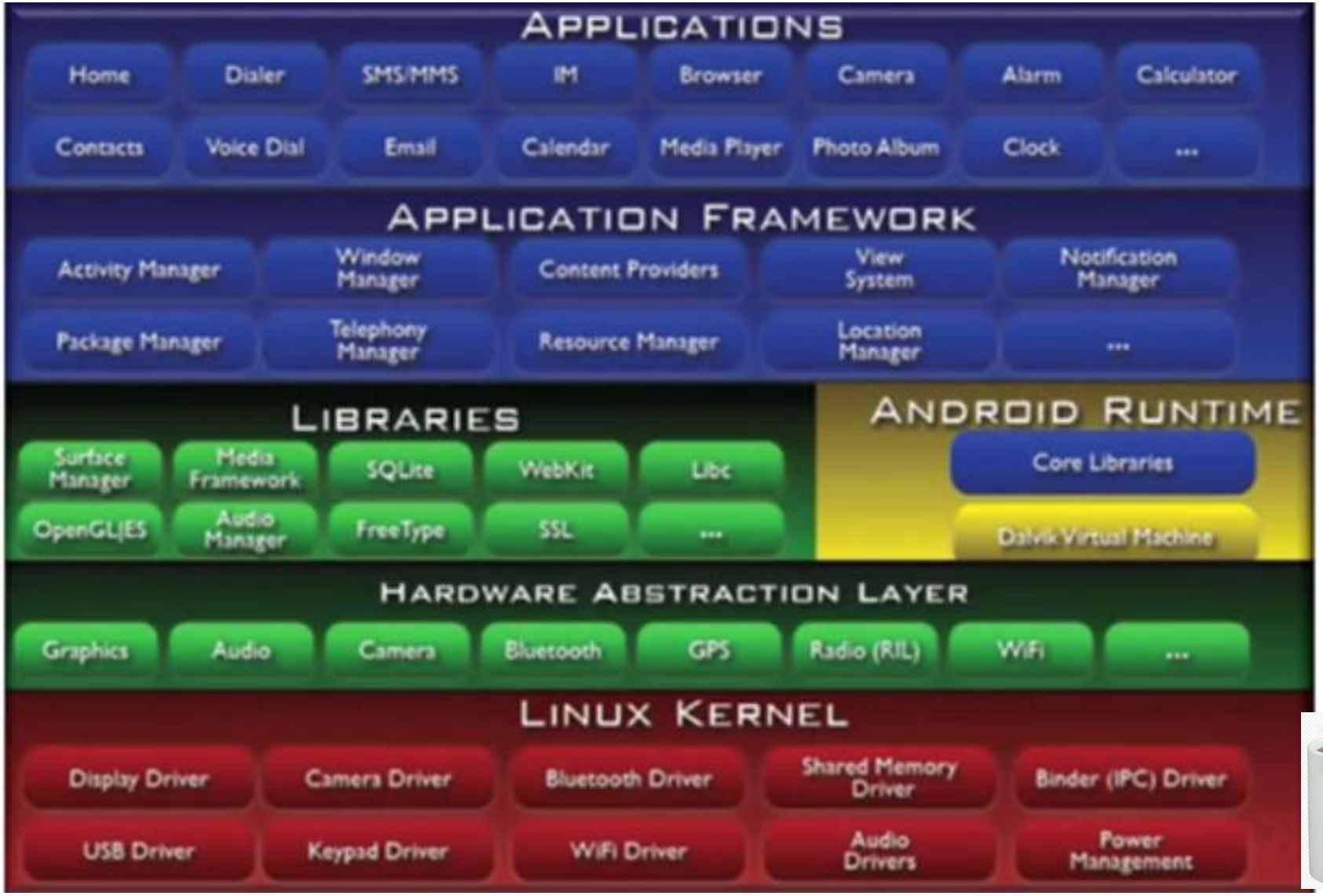






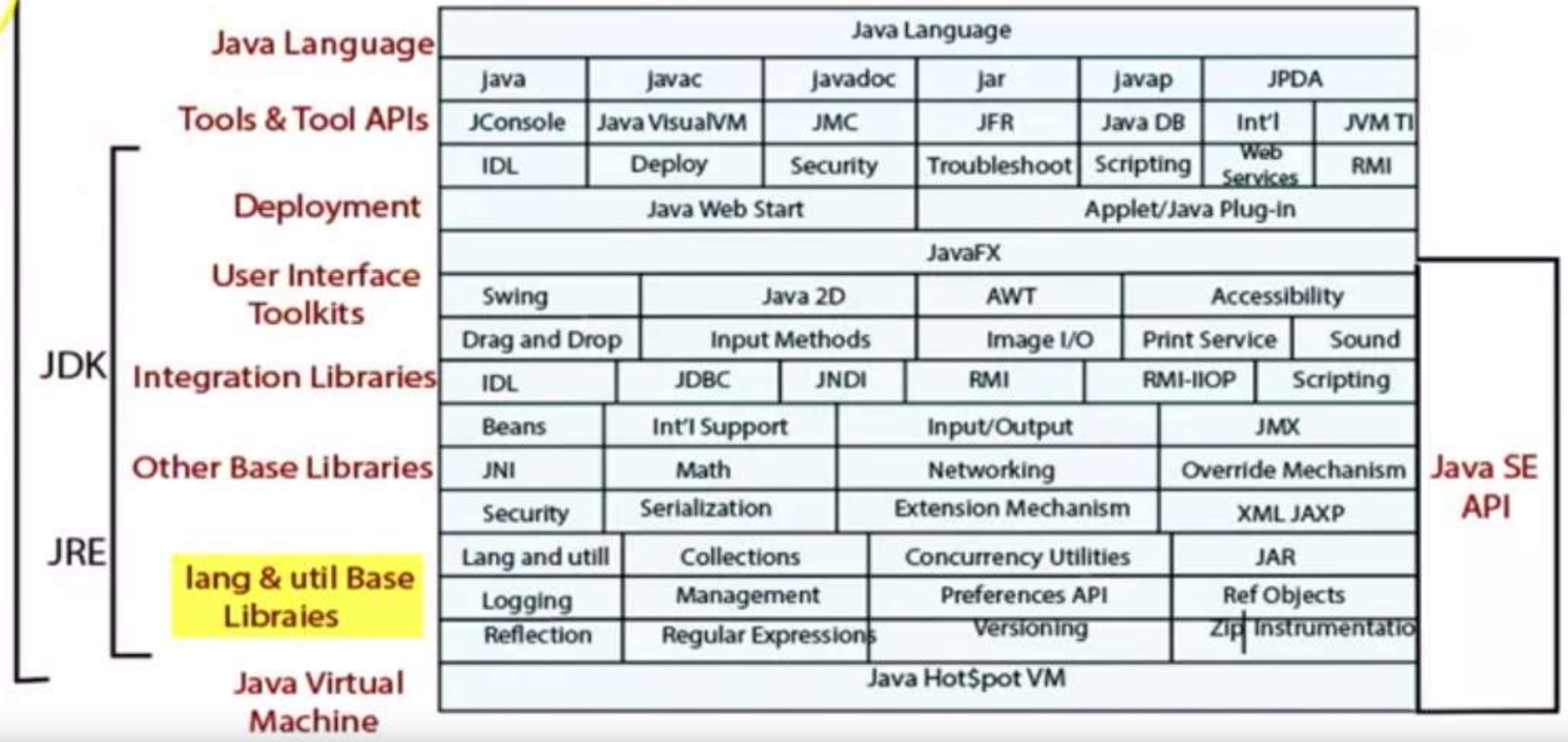





APPLICATIONS
Home
Dialer
SMS/MMS
IM
Browser
Camera
Alarm
Calculator
Contacts
Voice Dial
Email
Calendar
Media Player
Photo Album
Clock
...
APPLICATION FRAMEWORK
Activity Manager
Window Manager
Content Providers
View System
Notification Manager
Package Manager
Telephony Manager
Resource Manager
Location Manager
...
LIBRARIES
Surface Manager
Media Framework
SQLite
WebKit
Libc
OpenGL|ES
Audio Manager
FreeType
SSL
...
ANDROID RUNTIME
Core Libraries
Dalvik Virtual Machine
HARDWARE ABSTRACTION LAYER
Graphics
Audio
Camera
Bluetooth
GPS
Radio (RIL)
WiFi
...
LINUX KERNEL
Display Driver
Camera Driver
Bluetooth Driver
Shared Memory Driver
Binder (IPC) Driver
USB Driver
Keypad Driver
WiFi Driver
Audio Drivers
Power Management


TALK
<CODE>
TO ME

# JVM, JDK and JRE – Deep Drive




Java


Kotlin


Overview of the Java Programming Language
Objective-C
C++
Python
Simula
Scala
Java
J# CLU BETA
JavaScript
CLOS
C#
Smalltalk
Ada95
Eiffel
Swift
Ruby
Modula-3
See en.wikipedia.org/wiki/List_of_object-oriented_programming_languages


TALK
<CODE>
TO ME


EL ECOSISTEMA JAVA
Un lenguaje. Infinitas posibilidades.
Spring Boot
Aplicaciones web empresariales
Hibernate
ORM y gestión de bases de datos
Android SDK
Desarrollo de aplicaciones móviles
Selenium
Automatización y testing de navegadores
Apache Kafka
Streaming de eventos en tiempo real
Maven / Gradle
Gestión de builds y dependencias
Java
Apache Spark
Procesamiento de Big Data
JavaFX / Swing
Aplicaciones de escritorio
Micronaut
Microservicios ligeros
REST APIs
Servicios backend
LibGDX
Desarrollo de videojuegos
JUnit
Testing unitario y aseguramiento de la calidad
UN SOLO LENGUAJE
UN ECOSISTEMA INFINITO
PRESENTE EN MILLONES DE APLICACIONES EN TODO EL MUNDO



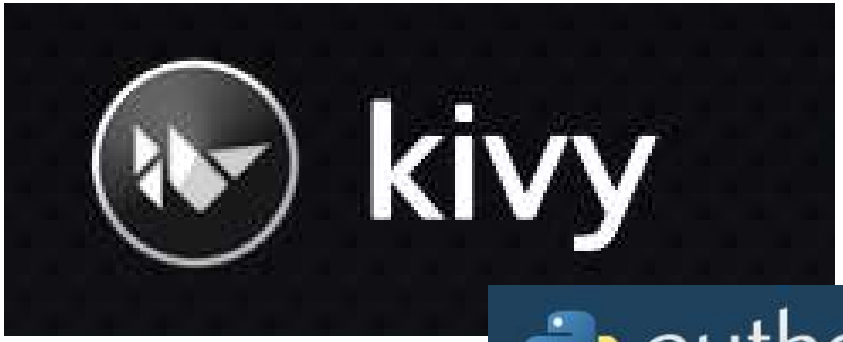

https://reactnative.dev/



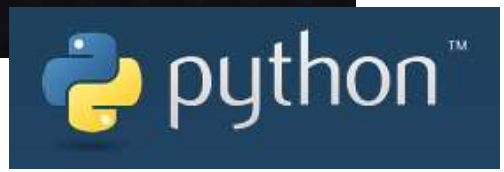




https://flutter.dev/?gclid=Cj0KCQjwuO6WBhDLARIsAIdeyDKeUFptvZSnBUVwlfW9F0HDqDJy8-wi-XvJQh77iqDq1GxuP7lUA8kaAloIEALw_wcB&gclsrc=aw.ds




https://kivy.org/#home

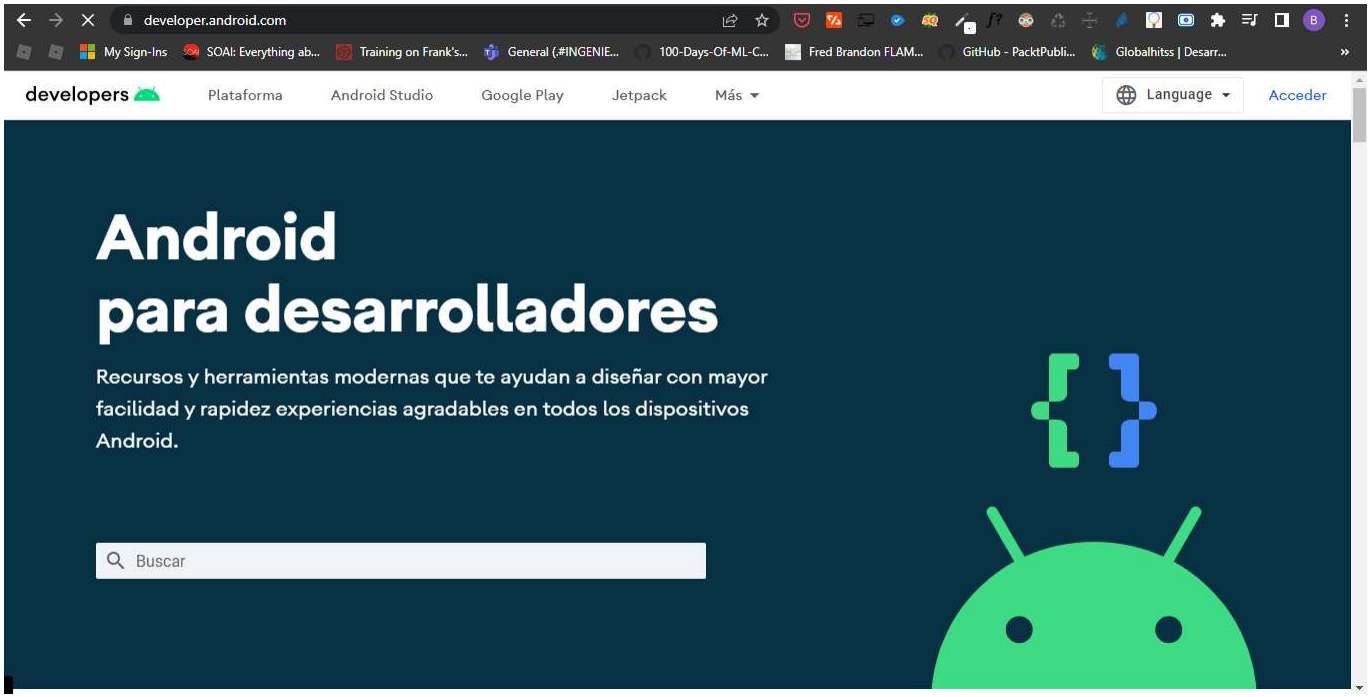








# Plataforma de Uso obligatorio para todo desarrollador Android




https://developers.google.com/



# ¿Por qué vamos a usar Java ?

- Java es Multiplataforma (Mac, Linux, Windows, Rasperry Pi)
- Su curva de aprendizaje es rápida
- Es fácil de usar
- Es open Source y gratis
- Es seguro,  además de rápido y poderoso.
- Cuenta con una gran comunidad que lo soporta
- Es un lenguaje orientado a Objetos
- Permite tener programas con una clara estructura
- Permite reutilizar código , reduciendo los costos de desarrollo.
- Es cercano a los C's : C y C++ . Esto permite que si aprendes Java puedas cambiar a estos fácilmente o Viveversa.
- Es uno de los lenguajes mas populares del mundo



¿Dónde se ubica Java en el top 10?




https://github.blog/2023-11-08-the-state-of-open-source-and-ai/



Java siempre está en el top 5 o Top 10 de los lenguajes mas usados

**En este post encontrarás:** [ ocultar ]


1 5 lenguajes de programación que serán los más usados en 2022

1.1 1.Python

1.2 2. Java

1.3 3. C#

1.4 4. C++

1.5 5. JavaScript


Fuente:

https://keepcoding.io/blog/5-lenguajes-de-programacion-mas-usados-2022/



# El concepto del algoritmo

## ALGORITMO

- Cero o más entradas
- Una o más salidas
- Descripción completa disponible, que es finita
- Cada paso es realizable
- Siempre termina su ejecución dando la respuesta correcta

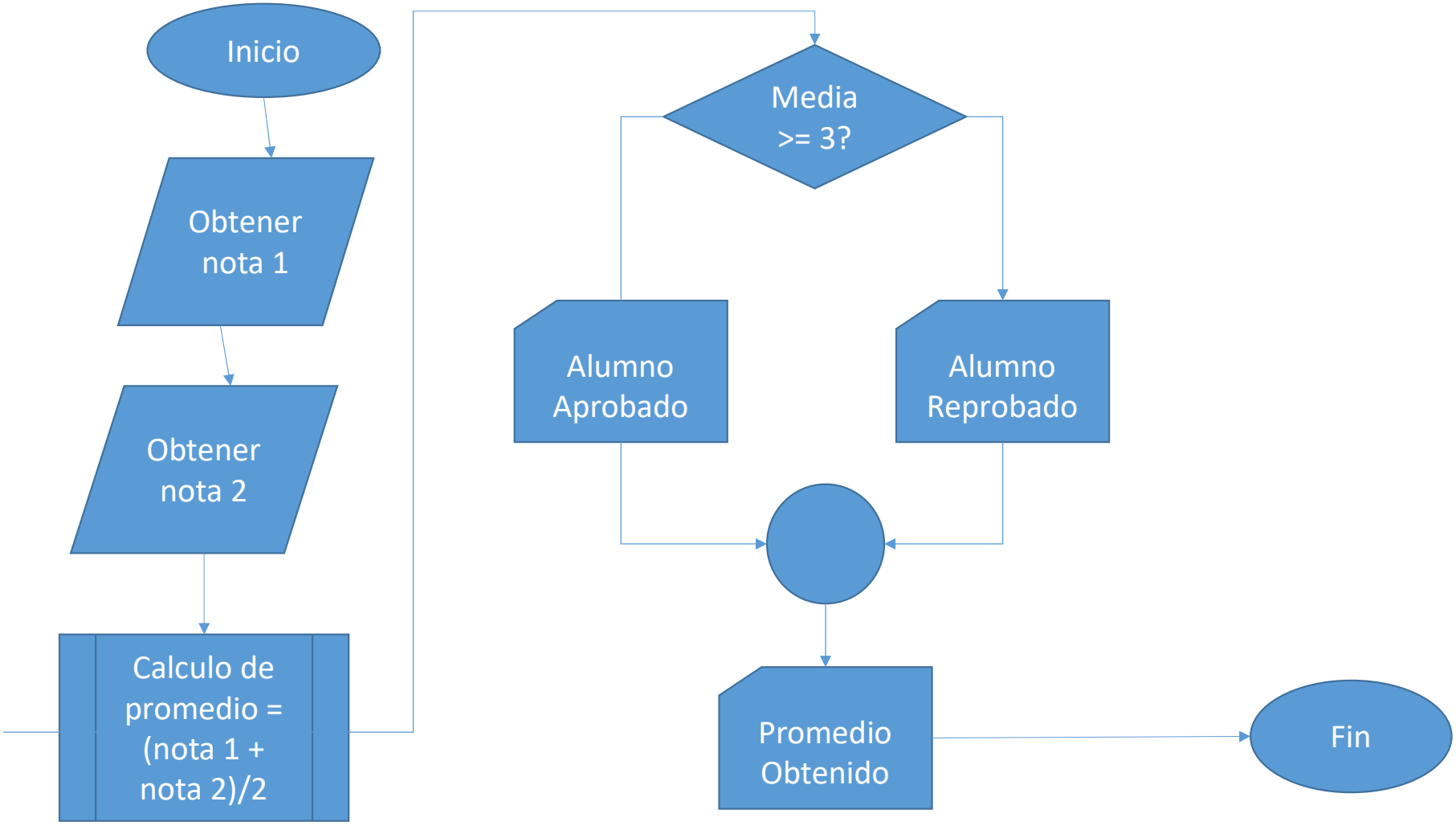
**Algoritmo**

**1.** m. Conjunto ordenado y finito de operaciones que permite hallar la solución de un problema.

**2.** m. Método y notación en las distintas formas del cálculo.



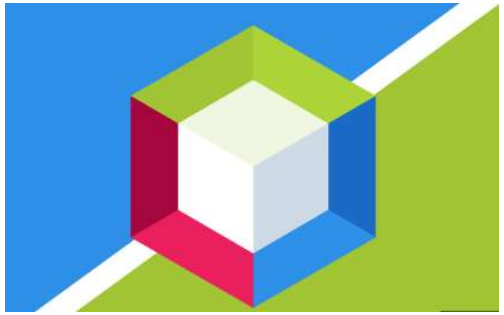
# Algoritmo promedio notas






# Entonos Integrados de desarrollo (IDE) Java

“Un IDE, por otro lado, es un conjunto de herramientas de desarrollo de software diseñadas para facilitar la codificación. En otras palabras, un IDE tiene un editor de código, un depurador, un compilador y otras funciones importantes, todo en una sola herramienta.”



Netbeans




Intellij Idea




Eclipse IDE




BlueJ

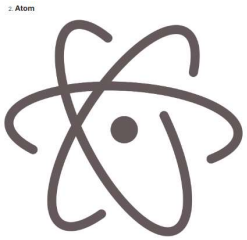
https://www.aluracursos.com/blog/aprenda-todo-sobre-el-ide-entorno-de-desarrollo-integrado



# Editores de código



Sublime Text




Atom




Visual Studio Code

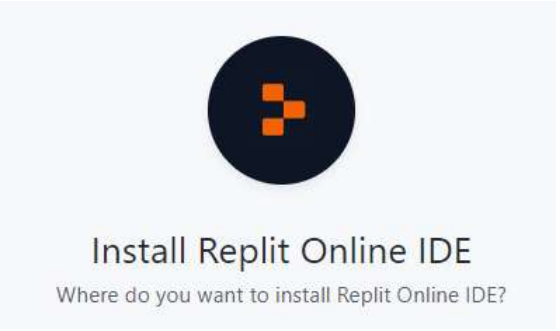
https://www.aluracursos.com/blog/sublime-atom-o-vscode-que-editor-de-codigo-es-mejor



# Editores de código en Línea

THEIA BLUEPRINT

Visual Studio Code



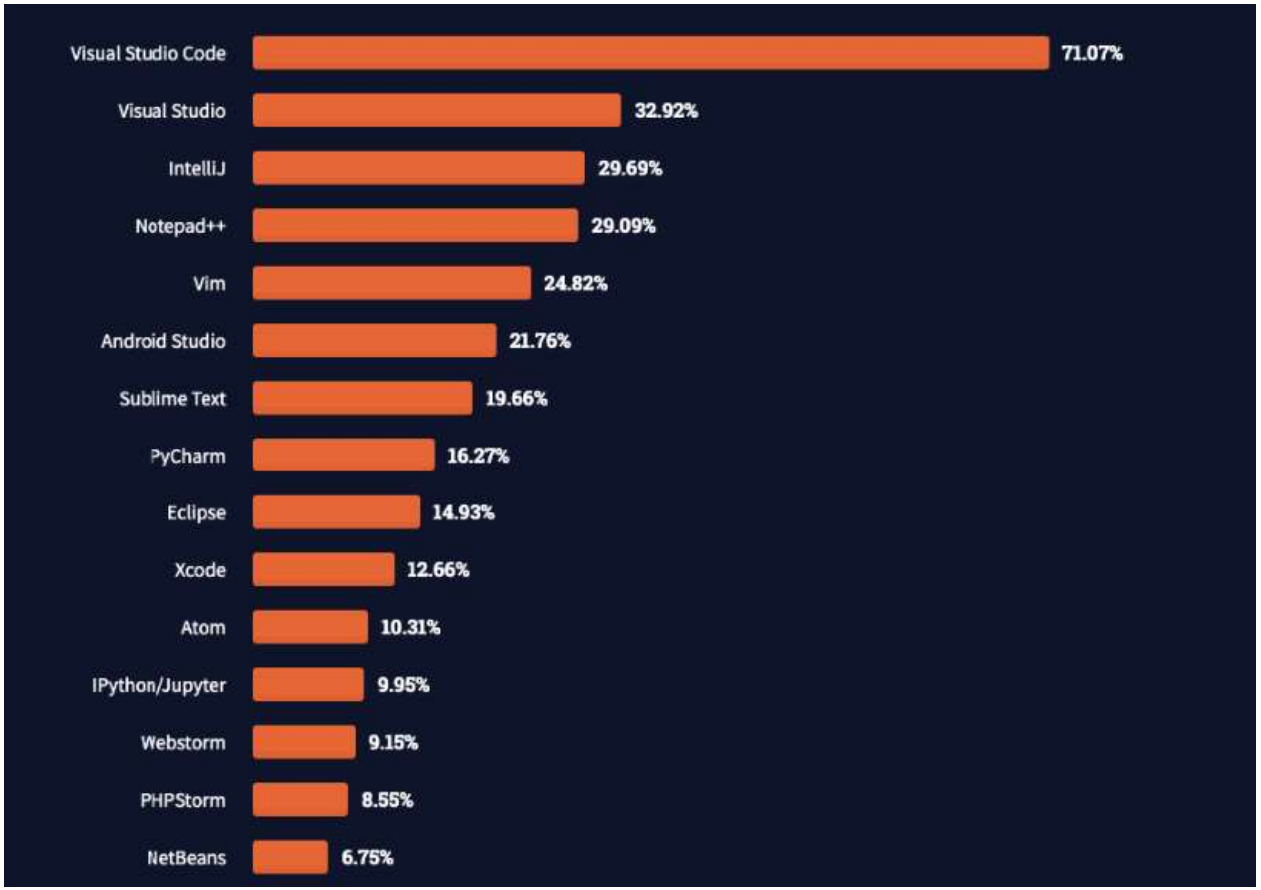

CODEPEN

https://www.aluracursos.com/blog/editores-de-codigo-e-ides



# Uso de IDE´s y editores de código en el mundo

https://insights.stackoverflow.com/survey/2021#most-popular-technologies-new-collab-tools




Fuente: StackOverflow 2021



# Herramientas Case que usaremos en el curso

Jira Software

https://www.atlassian.com/software/jira

https://github.com/

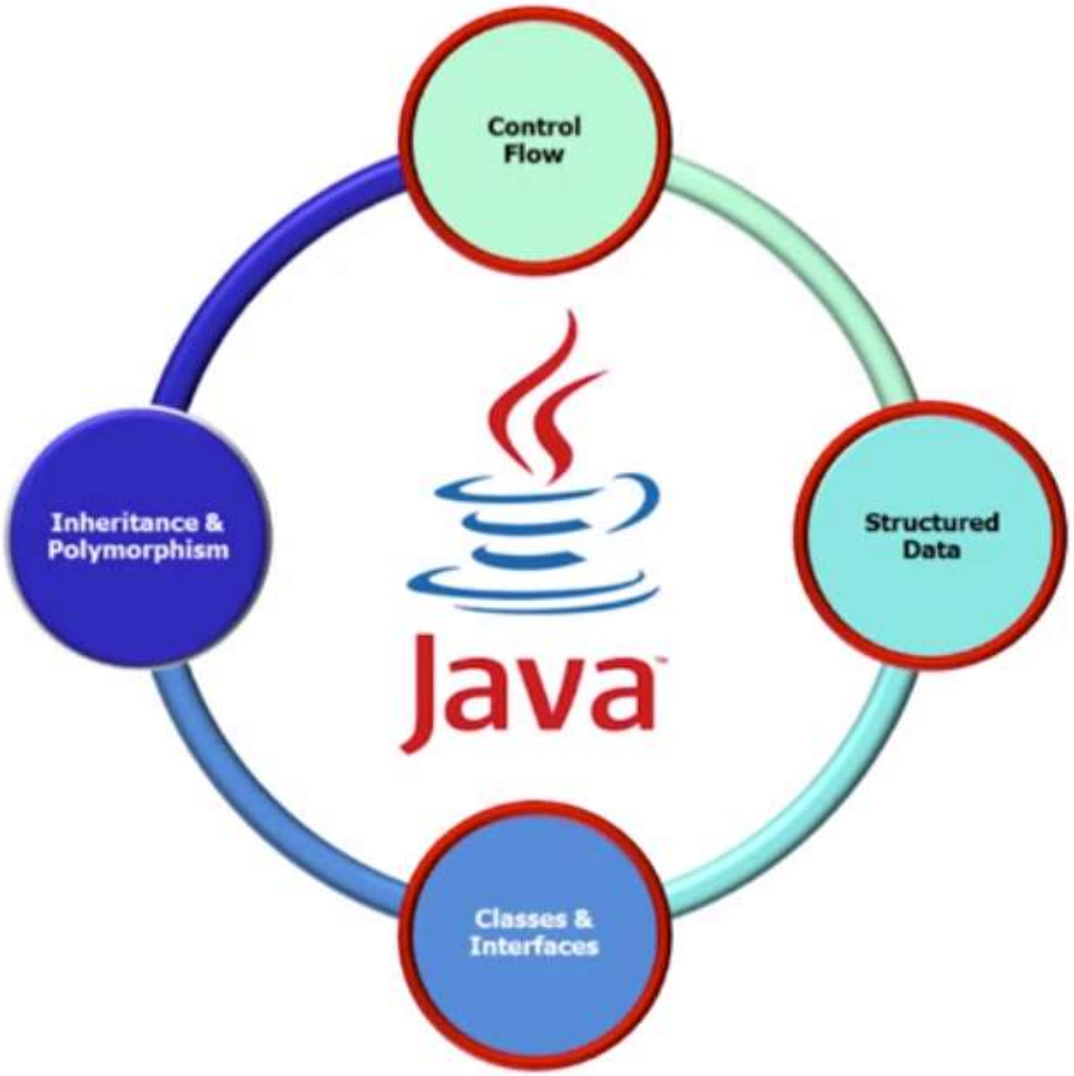
https://app.diagrams.net/

https://www.figma.com/

Consulta los siguientes recursos para ampliar el conocimiento sobre las herramientas CASE
https://zimbronapps.com/educacion/sistemas-computacionales/ingenieria-de-software/importancia-e-historia-las-herramientas-case/#google_vignette

https://youtu.be/1UedW1W2JcU






# Fundamentos de Java



# Generalidades de Java

- Java  pertenece a la familia de los C y comparte similitudes con C y C++
- Es un lenguaje orientado a Objeto por excelencia.
- Originalmente fue diseñado para productos electrónicos.
- Su origen se dio en 1995 por su creador James Goslin.
- Actualmente sus aplicaciones incluyen internet de las cosas, computación en la nube,  dispositivos inteligentes, teléfonos, tables, Smart watchs, computadoras y supercomputadoras.
- Java **WORA** => Write Once, Run Anywhere

https://www.aluracursos.com/blog/caracteristica-destacables-java8-delante



# Historia de Java

Java fue creado en 1995 por James Goslin , quien trabajaba para Sun Microsystems, compañía que mas tarde sería adquirida por Oracle, en el año 2010.





Java es uno de los lenguajes de programación mas populares del mundo y su uso sigue siendo importante , debido a que hay en el planeta mas de 3 billones de dispositivos ejecutándolo.

Su uso abarca áreas como el desarrollo movíl, el desarrollo web, aplicaciones de escritorio , servidores web , aplicaciones de servidores , juegos, conexiones a Bases de datos , sistemas de dispositivos y microcontroladores, etc.







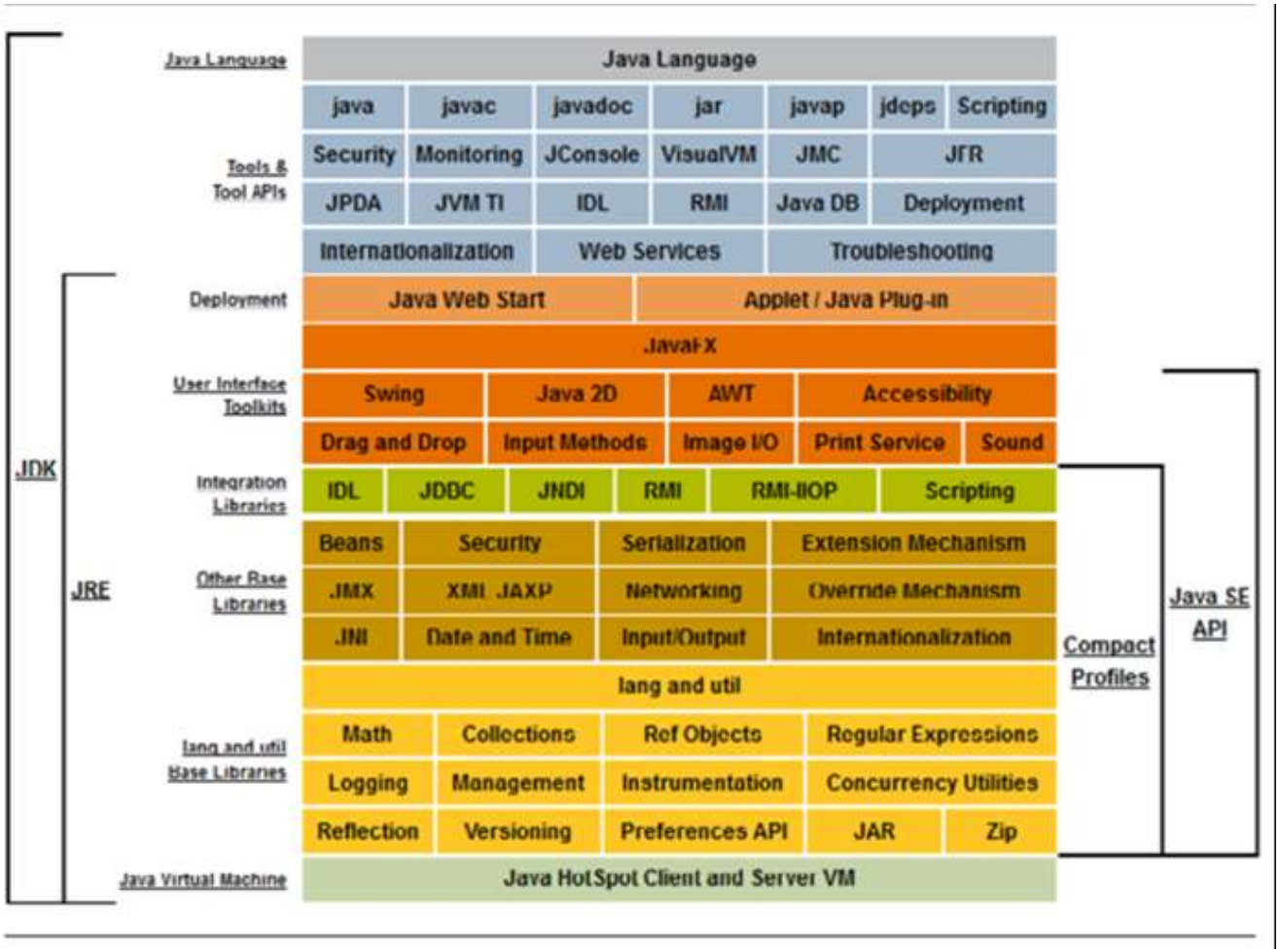

# Versiones de Java a través de los años




[https://www.aluracursos.com/blog/caracteristica-destacables-java8-delante](https://www.aluracursos.com/blog/caracteristica-destacables-java8-delante)

| | |
|---|---|
| JDK Beta | 1995 |
| JDK 1.0 | Enero 1996 |
| JDK 1.1 | Febrero 1997 |
| J2SE 1.2 | Diciembre 1998 |
| J2SE 1.3 | Mayo 2000 |
| J2SE 1.4 | Febrero 2002 |
| J2SE 5.0 | Septiembre 2004 |
| Java SE 6 | Diciembre 2006 |
| Java SE 7 | Julio 2011 |
| Java SE 8 (LTS) | Marzo 2014 |
| Java SE 9 | Septiembre 2017 |
| Java SE 10 | Marzo 2018 |
| Java SE 11 (LTS) | Septiembre 2018 |
| Java SE 12 | Marzo 2019 |

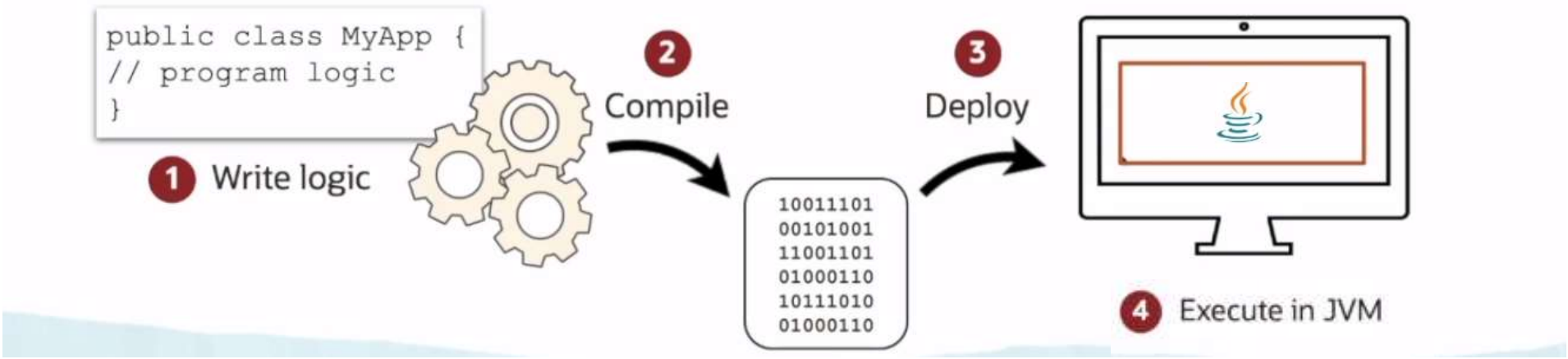





Fuente: Curso Java SE / Conecta empleo – Telefónica



# Como Java funciona




Fuente: [https://mylearn.oracle.com/ou/component/-/79727/110386](https://mylearn.oracle.com/ou/component/-/79727/110386)



# Java Virtual Machine

- Funciona como una computadora , pero es un software
- Administra la memoria a través del Garbage colector
- Permite que Java sea multiplataforma
- Administra la seguridad
- Optimización del uso del procesador
- Librerías con utilidades



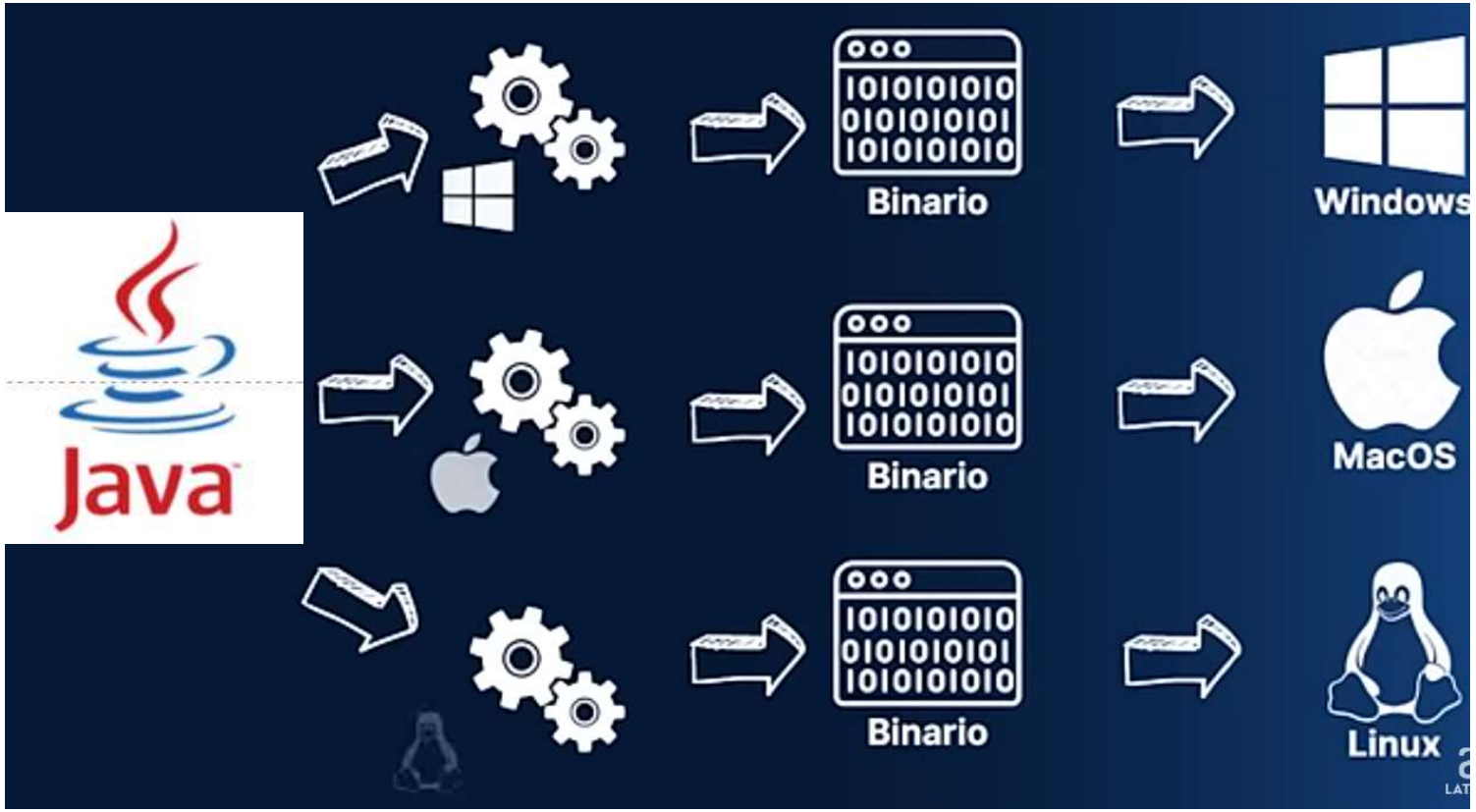

- Existen lenguajes que estan soportados en la Java Virtual Machine como Ruby, Python, Scala , Groovy, Clojure, Kotlin.



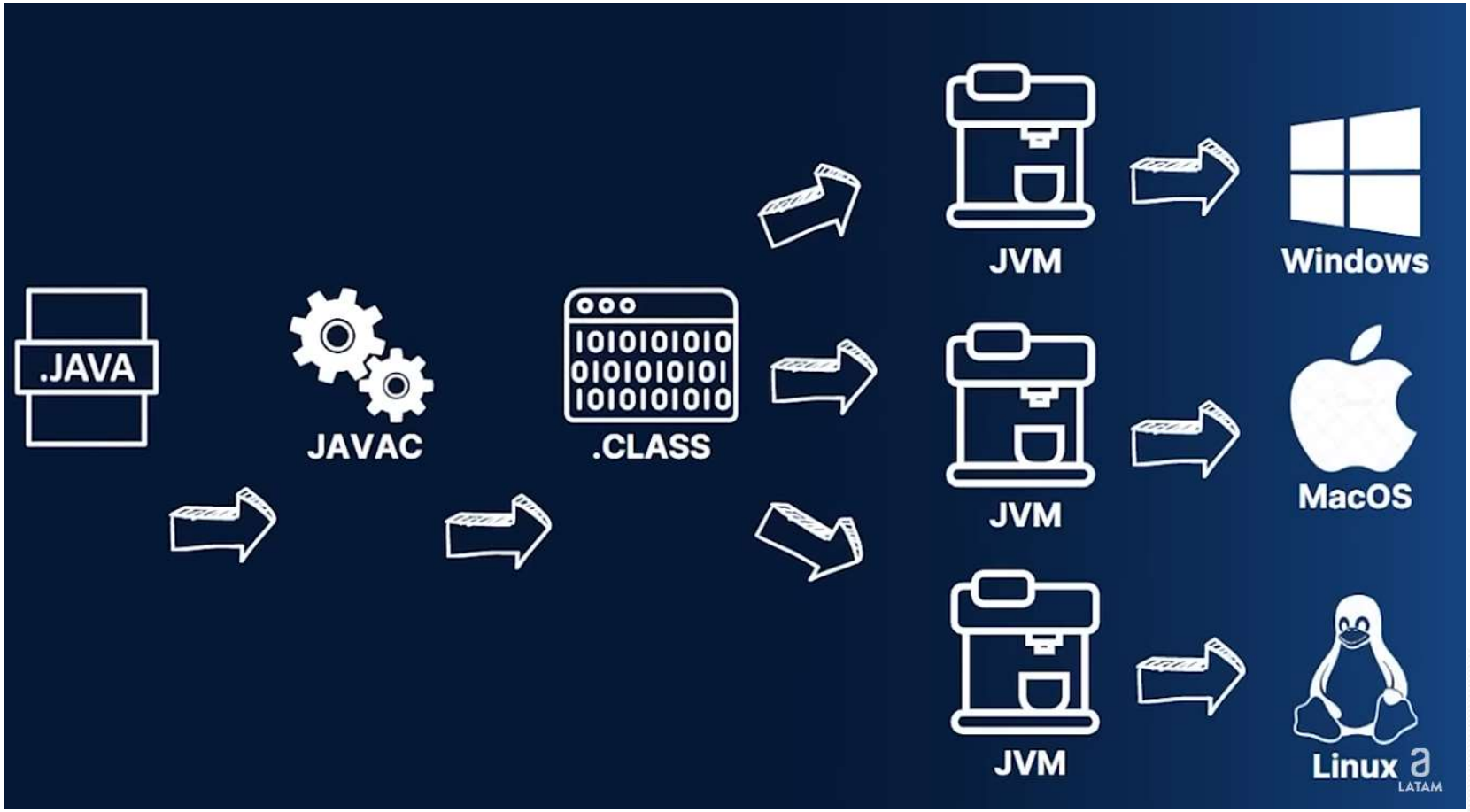

Observa el siguiente video

https://youtu.be/GrEO8nZzyZM



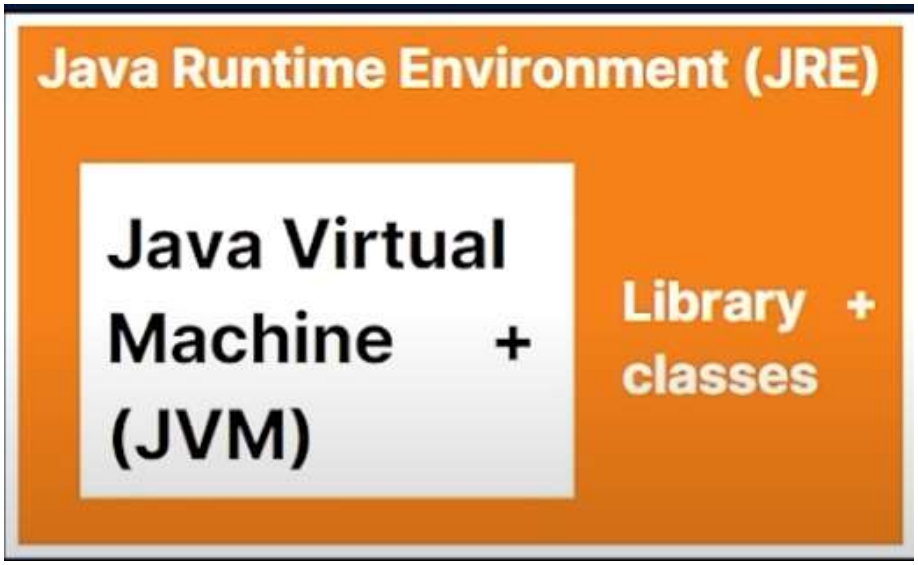

Java
Binario
Windows
Binario
MacOS
Binario
Linux


TALK
<CODE>
TO ME


.JAVA
JAVAC
.CLASS
JVM
JVM
JVM
Windows
MacOS
Linux
LATAM


TALK
<CODE>
TO ME

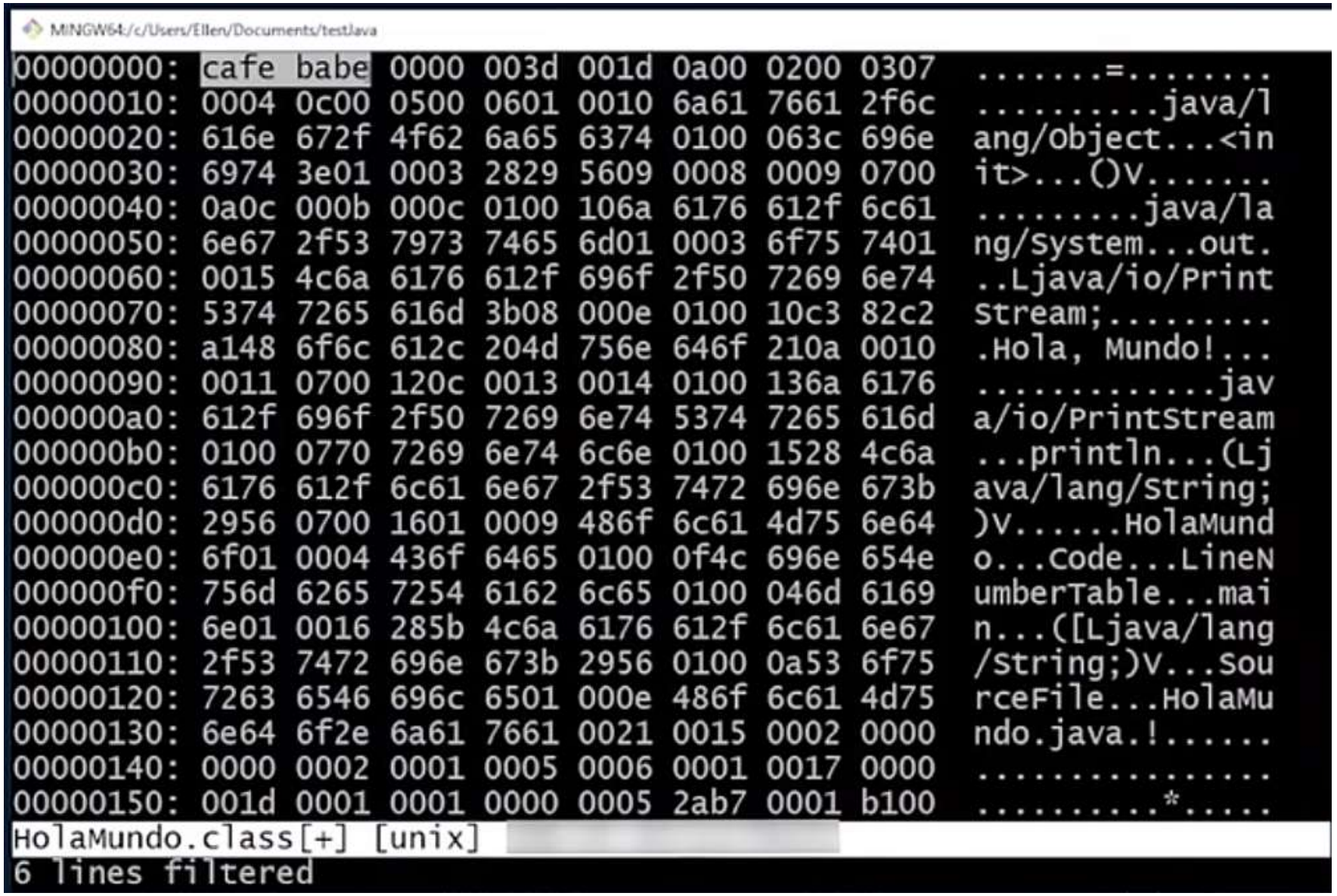
La JVM se encuetra de dentro del Java Runtime Enviroment.




Los que solo desean usar java usaran una JRE

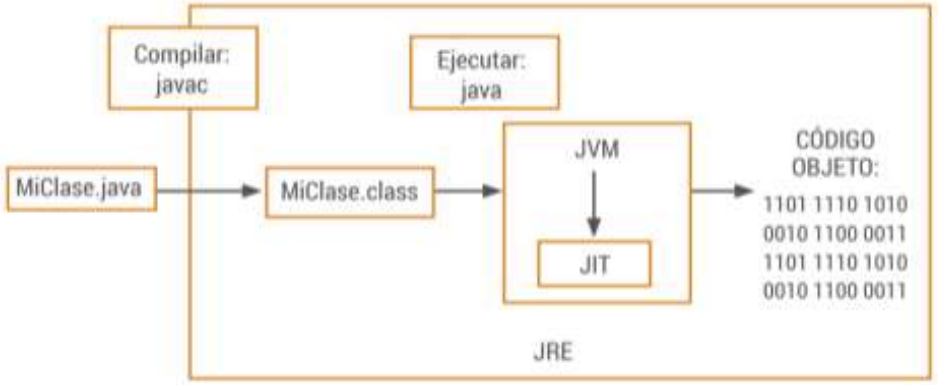
Los que solo desean programar deberán usar un JDK



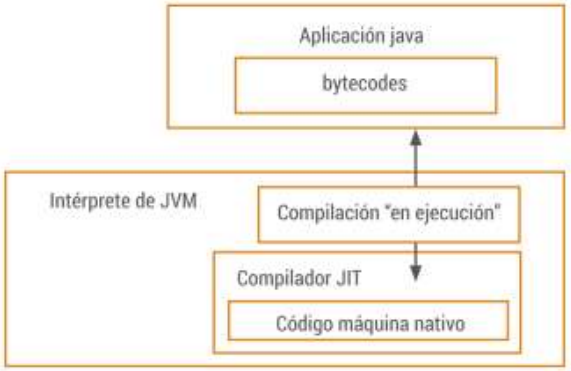

MINGW64:/c/Users/Ellen/Documents/testJava
00000000: cafe babe 0000 003d 001d 0a00 0200 0307 .......=........
00000010: 0004 0c00 0500 0601 0010 6a61 7661 2f6c ..........java/l
00000020: 616e 672f 4f62 6a65 6374 0100 063c 696e ang/Object...<in
00000030: 6974 3e01 0003 2829 5609 0008 0009 0700 it>...()V.......
00000040: 0a0c 000b 000c 0100 106a 6176 612f 6c61 .........java/la
00000050: 6e67 2f53 7973 7465 6d01 0003 6f75 7401 ng/System...out.
00000060: 0015 4c6a 6176 612f 696f 2f50 7269 6e74 ..Ljava/io/Print
00000070: 5374 7265 616d 3b08 000e 0100 10c3 82c2 Stream;.........
00000080: a148 6f6c 612c 204d 756e 646f 210a 0010 .Hola, Mundo!...
00000090: 0011 0700 120c 0013 0014 0100 136a 6176 .............jav
000000a0: 612f 696f 2f50 7269 6e74 5374 7265 616d a/io/PrintStream
000000b0: 0100 0770 7269 6e74 6c6e 0100 1528 4c6a ...println...(Lj
000000c0: 6176 612f 6c61 6e67 2f53 7472 696e 673b ava/lang/String;
000000d0: 2956 0700 1601 0009 486f 6c61 4d75 6e64 )V......HolaMund
000000e0: 6f01 0004 436f 6465 0100 0f4c 696e 654e o...Code...LineN
000000f0: 756d 6265 7254 6162 6c65 0100 046d 6169 umberTable...mai
00000100: 6e01 0016 285b 4c6a 6176 612f 6c61 6e67 n...([Ljava/lang
00000110: 2f53 7472 696e 673b 2956 0100 0a53 6f75 /String;)V...Sou
00000120: 7263 6546 696c 6501 000e 486f 6c61 4d75 rceFile...HolaMu
00000130: 6e64 6f2e 6a61 7661 0021 0015 0002 0000 ndo.java.!......
00000140: 0000 0002 0001 0005 0006 0001 0017 0000 ................
00000150: 001d 0001 0001 0000 0005 2ab7 0001 b100 ..........*.....
HolaMundo.class[+] [unix]
6 lines filtered

# Como Java funciona

Primera fase




Segunda fase







Fuente: Curso Java SE / Conecta empleo – Telefónica

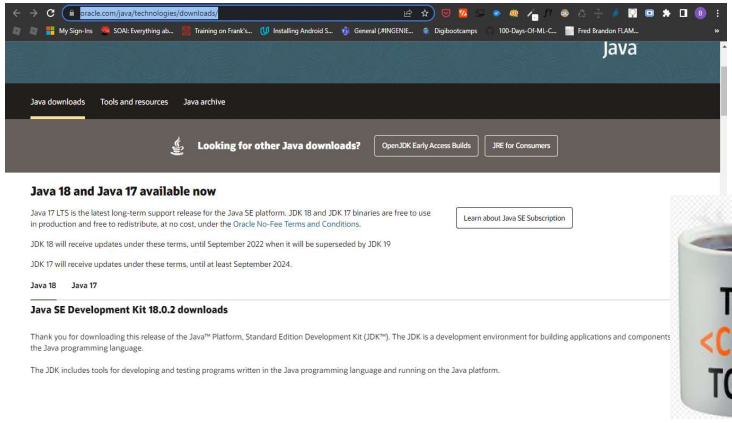
# Java es un lenguaje compilado e interpretado a la vez

Lo que quiere decir que Java se vale de un lenguje intermendio llamado ByteCode que puede ser interpretado usando el JRE (Java Runtime Enviroment https://www.java.com/es/download/ ) Adicional necesitaras tener instalado el JDK (Java Development Kit https://www.oracle.com/java/technologies/downloads/ )







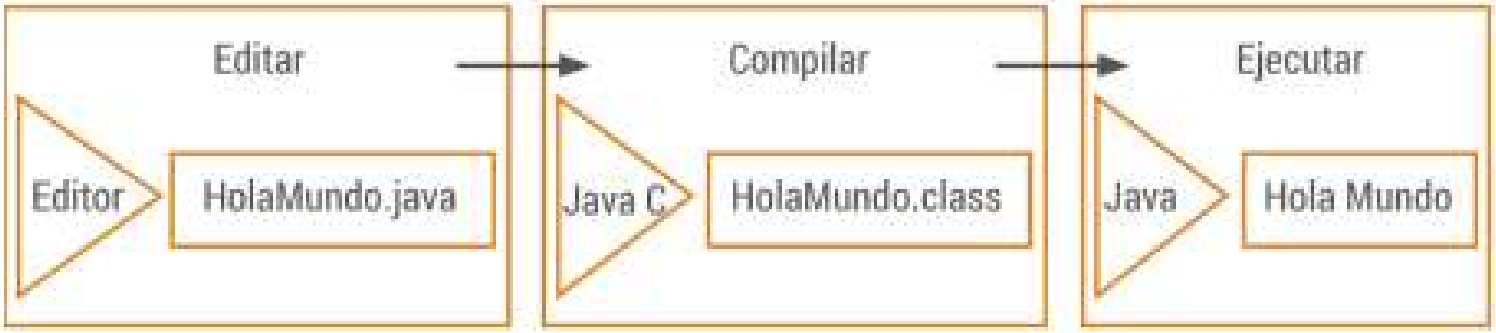






https://blog.makeitreal.camp/lenguajes-compilados-e-interpretados/

Java es un lenguaje compilado e interpretado a la vez




Fuente: Curso Java SE / Conecta empleo – Telefónica



# Estructura del código de Java

**Clases:**
- Unidad básica de código
- Describe los datos y la lógica del programa.

**Paquetes :**
- Agregación lógica intermedia del código
- Agrupa clases

**Módulos:**
- Aparece con JAVA 9
- Agrupa paquetes
- Es una agregación física de código de alto nivel que permite sobrescribir las agregaciones de código de bajo nivel del paquete y escoger explícitamente que partes del código quieres exponer para que ls personas puedan reusar y que partes no.



# Ecosistema de Java

- **Java  Card** => Es usada en SIM Cards
- **Java ME – Micro Edition**  => Usada en Smrt TV ‘s, asi como puede ser embebida en dispositivos electrónicos
- **Java SE – Standard edition** =>  Sirve para introducirse en el mundo de  Java, y que contiene las librerías, el compilador y la Java Virtual Machine que provee las estructuras necesarias para soportar el código que estás creando.
- **Java MP – Micro Profile** => Sirve para crear microservicios de cara al servidor
- **Java EE – Enterprise Edition** => Soporta la creación de aplicaciones para empresas que incluye Web Services ,  Passing Messages a través de Java Messaging Asincrono y JavaBeans Empresarial. Además de crear Servlets y Java Servers Pages, Jav Serves Faces Applications



# Elementos del lenguaje Java



# Notación para escribir los nombre de Clases, objetos, métodos y variables en Java

1 Camel Case (contarElementos)

2 Pascal Case (ContarElementos)

3 Snake Case (contar_elementos)

4 Kebab Case (contar-elementos)

Lo usaremos para las variables, objetos y métodos

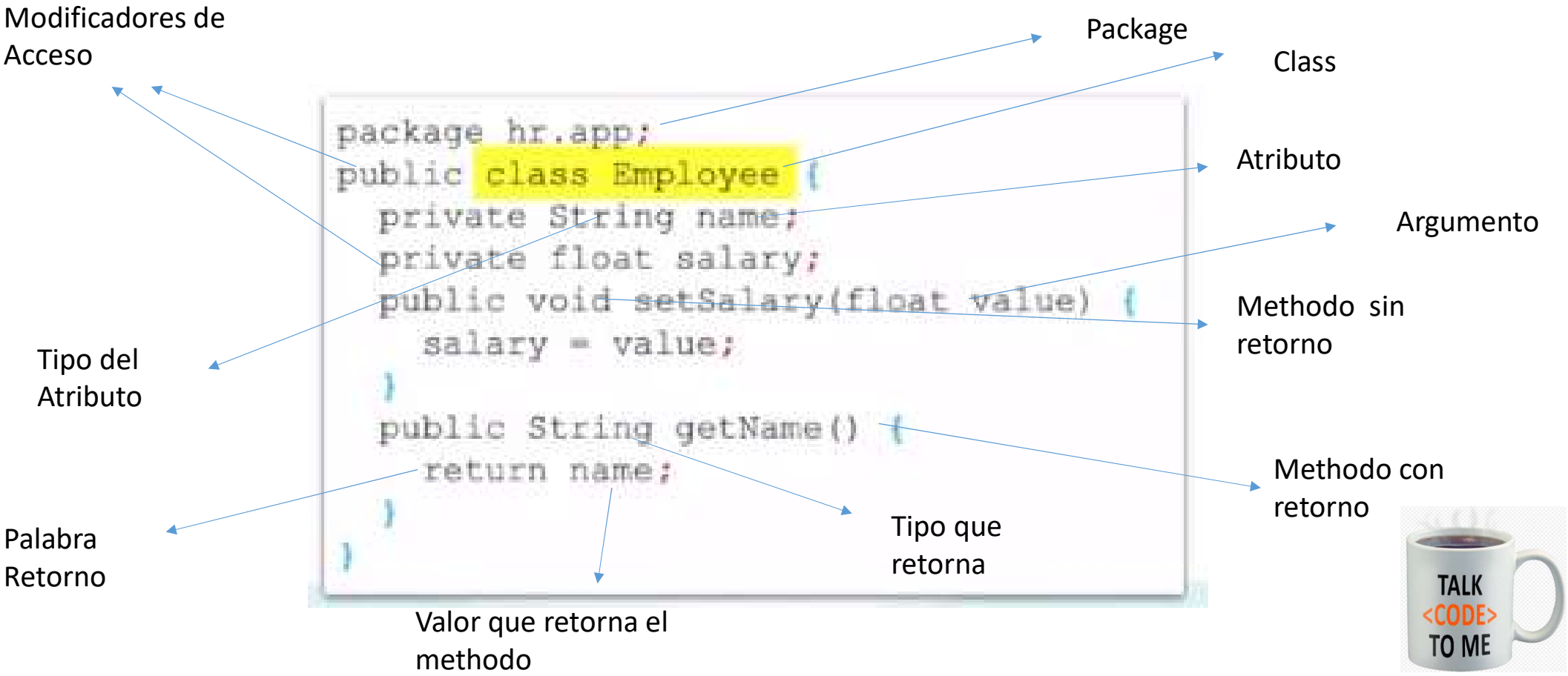
Lo usaremos para las clases , proyecto

Lo usaremos todo en minúscula continua sin underscore para packeges

Tomado de: https://www.neoguias.com/tipos-notacion-nombres/



# Sintaxis de Java



# El método Main en Java

En Java no puedes ejecutar nada que no este representado en una clase  y dentro del método principal



# Comentarios en Java

```java
/**
 *
 * @author bamg3
 */
public class JavaApplication26 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        System.out.println("Hello World");
    }

}
```

Comentario de múltiples líneas

El método Main

Comentario de una sola línea



# Syntax Elements of Programming Languages

Elements of a programming language are known as tokens.

- Identifier is a name the programmer chooses to label things.
- Keyword is a name that is a part programming language.
- Separator is a punctuation character.
- Operator is a symbol that operates on arguments to produce a result.
- Literal is an expression of a numeric, logical, or textual value.
- **Comment** is a line or a block of lines that explains program logic to the reader.

```
let price = 1.99, quantity = 3, total = 0;
// calculate total
total = price * quantity;
/* calculate discount
   based on the quantity */
if (quantity > 2) {
  total = total - price;
}
```



# Tipos de datos primitivos en Java y sus rangos

**Tabla 3.1. Tipos primitivos de variables en Java.**

| Nombre | Tipo | Espacio en memoria | Rango |
|---|---|---|---|
| byte | entero | 1 byte | −128 y 127 |
| short | entero | 2 bytes | −32,768 y 32,767 |
| int | entero | 4 bytes | −2,147,483,648 y 2,147,483,647 |
| long | entero | 8 bytes | −9,223,372,036,854,775,808 y 92233720368547758O7 |
| float | decimal simple | 4 bytes | −3.402823e38 a −1.401298e-45 y 1.401298e-45 a 3.402823e38 |
| double | decimal doble | 8 bytes | ±1.79769313486232e±308 |
| char | caracter simple | 2 bytes | Código UNICODE (incluye ASCII) |
| boolean | valor true o false | 1 byte | true o false |

Fuente: Fundamentos de programación. Alfa Omega. 2018



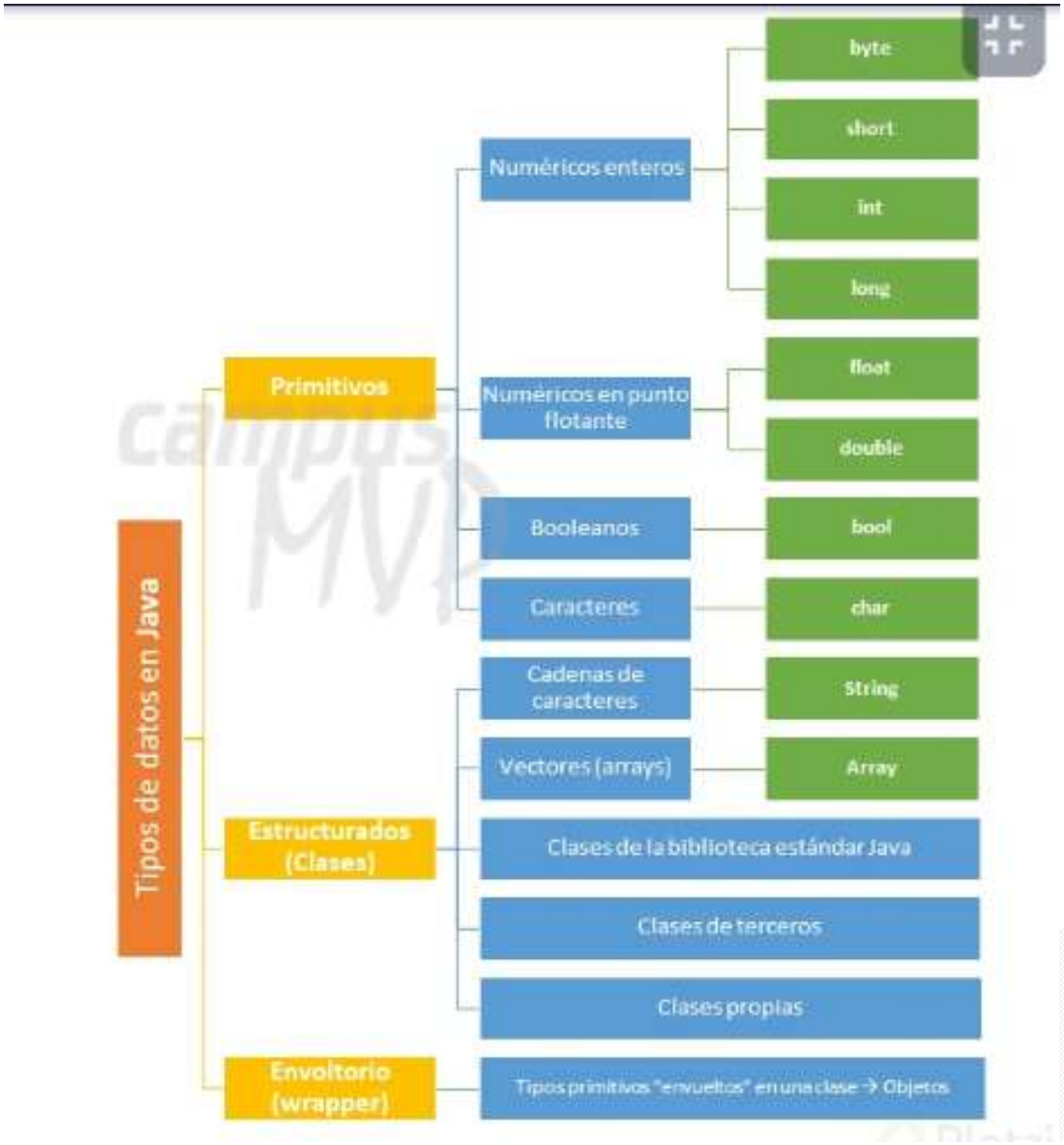
# Tipos de datos primitivos en Java y sus rangos

| Tipo | Representación | Bytes | Rango | Valor por omisión | Clase envoltorio |
|---|---|---|---|---|---|
| **char** | Caracter Unicode | 2 | \u0000 a \uFFFF | \u0000 | Character |
| **byte** | | 1 | -128 a 127 | 0 | Byte |
| **short** | | 2 | -32768 a 32767 | 0 | Short |
| **int** | Entero con signo | 4 | -2147483648 a 2147483647 | 0 | Integer |
| **long** | | 8 | -9223372036854775808 a 9223372036854775807 | 0 | Long |
| **float** | Punto flotante precisión simple | 4 | ±3.4*10-38 a ±3.4*1038 | 0.0 | Float |
| **double** | Punto flotante precisión doble | 8 | ±1.8*10-308 a ±1.8*10308 | 0.0 | Double |
| **boolean** | Lógico | 1 | true o false | false | Boolean |
| **void** | - | - | - | - | Void |



# Tipos de datos primitivos en Java

A partir de Java 10 y posteriores , ya no es necesario indicar el tipo de dato, solo se antepone la palabra reservada VAR y este automáticamente reconoce el tipo de dato.






# Palabras reservadas en Java

Nuestras variables no podrán llamarse de esta manera , ya que estas son palabras reservadas de java .

| Tabla 3.2. Palabras Reservadas en Java. | | | | |
|---|---|---|---|---|
| abstract | boolean | break | byte | case |
| catch | char | class | continue | default |
| do | double | else | extends | false |
| final | finally | float | for | goto |
| if | implements | import | instanceof | int |
| interface | long | native | new | null |
| package | private | protected | public | return |
| short | static | super | switch | synchronized |
| this | throw | throws | transient | true |
| try | void | volatile | while | |

Fuente: Fundamentos de programación. Alfa Omega. 2018



# Palabras reservadas en Java

| | | | | |
|---|---|---|---|---|
| abstract | continue | for | new | switch |
| assert*** | default | goto* | package | synchronized |
| boolean | do | if | private | this |
| break | double | implements | protected | throw |
| byte | else | import | public | throws |
| case | enum**** | instanceof | return | transient |
| catch | extends | int | short | try |
| char | final | interface | static | void |
| class | finally | long | strictfp** | volatile |
| const* | float | native | super | while |

* not used
** added in 1.2
*** added in 1.4
**** added in 5.0



# Operadores aritméticos en Java

**Tabla 3.3. Operadores aritméticos en Java.**

| Operador | Operación | Ejemplo | Significado de la operación |
|---|---|---|---|
| + | Suma | `c = a+b` | Suma el valor de `a` con `b` y asigna el resultado a `c`. |
| − | Resta | `c = a−b` | Al valor de `a` le resta el valor de `b` y el resultado de la operación se lo asigna a `c`. |
| * | Multiplicación | `c = a*b` | Multiplica `a` por `b` y asigna el resultado a `c`. |
| / | División | `c = a/b` | Divide el valor de la variable `a` entre el valor de `b` y asigna el resultado a la variable `c`. |
| / | Módulo residuo | `c = a % b` | El valor de `c` es el residuo de dividir `a` entre `b`. Ejemplos: 5 % 3 = 2; 13 % 8 = 5 |

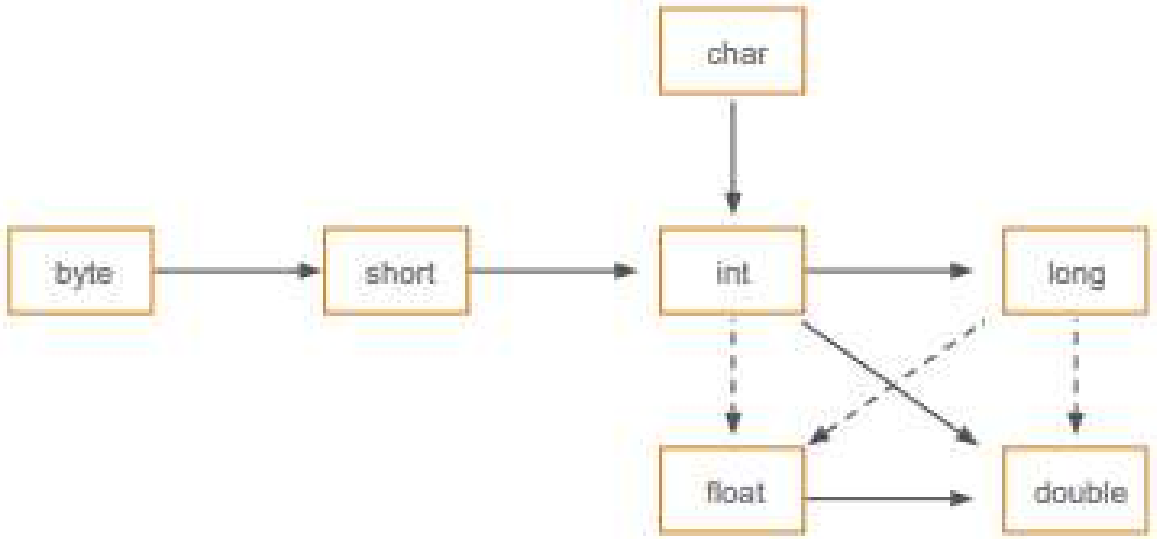


# Jerarquía de los operadores en Java

| Operadores | Tipo | Asociatividad |
|---|---|---|
| [] . () y llamada a método | Especiales | De izquierda a derecha |
| ! ~ ++ -- +(unario) -(unario) (moldeo) new | Unarios, moldeo y creación | De derecha a izquierda |
| * / % | Aritméticos | De izquierda a derecha |
| + - | Aritméticos | De izquierda a derecha |
| << >> >>> | Nivel de bits | De izquierda a derecha |
| < <= > >= | Relacionales | De izquierda a derecha |
| == != | | De izquierda a derecha |
| & | Lógicos | De izquierda a derecha |
| ^ | | De izquierda a derecha |
| \| | | De izquierda a derecha |
| && | | De izquierda a derecha |
| \|\| | | De izquierda a derecha |
| ?: | Ternario | De izquierda a derecha |
| Todos los de asignación | | De derecha a izquierda |



# Casting de variables en Java




Las flechas continuas implican conversiones sin pérdida de información.
Las flechas discontinuas implican conversiones en las que puede haber perdida de información.



# Casting de variables en Java

| DE/PARA | byte | short | char | int | long | float | double |
|---|---|---|---|---|---|---|---|
| byte | ---- | *Impl.* | (char) | *Impl.* | *Impl.* | *Impl.* | *Impl.* |
| short | (byte) | ---- | (char) | *Impl.* | *Impl.* | *Impl.* | *Impl.* |
| char | (byte) | (short) | ---- | *Impl.* | *Impl.* | *Impl.* | *Impl.* |
| int | (byte) | (short) | (char) | ---- | *Impl.* | *Impl.* | *Impl.* |
| long | (byte) | (short) | (char) | (int) | ---- | *Impl.* | *Impl.* |
| float | (byte) | (short) | (char) | (int) | (long) | ---- | *Impl.* |
| double | (byte) | (short) | (char) | (int) | (long) | (float) | ---- |

| TIPO | TAMAÑO |
|---|---|
| boolean | 1 bit |
| byte | 1 byte |
| short | 2 bytes |
| char | 2 bytes |
| int | 4 bytes |
| float | 4 bytes |
| long | 8 bytes |
| double | 8 bytes |




Fuente; Curso Alura Latam – Oracle - Java

# Jerarquía de operaciones en Java

| Tabla 3.4. Jerarquía de operadores en Java. | |
|---|---|
| **Operador** | **Operación** |
| * / % | Los operadores de multiplicación, división y módulo son los que primero se evalúan; si hay varios de este tipo, se evalúan de izquierda a derecha. |
| + − | Los operadores de suma y resta son los que se evalúan después; si hay varios de este tipo, se evalúan de izquierda a derecha. |



# Secuencias en Java

| Tabla 3.6. Secuencias de Escape en Java. | |
|---|---|
| **Secuencia de escape** | **Descripción** |
| `\n` | Salto de línea, coloca el cursor en la línea siguiente. |
| `\t` | Tabulador horizontal, coloca el cursor en la siguiente posición de tabulación. |
| `\r` | Retorno de carro, mueve el cursor al inicio de la línea actual; es decir, no avanza a la siguiente línea. |
| `\\` | Se emplea para imprimir la barra diagonal inversa. |
| `\"` | Se emplea para imprimir un carácter de doble comilla. |



# Operadores simplificados en Java

**Tabla 3.5. Operadores simplificados en Java.**

| Operador | Operación | Ejemplo | Equivalente a | Significado | Si c = 13 el nuevo valor de c es |
|---|---|---|---|---|---|
| += | Suma | c +=3 | c = c + 3 | Incrementa el valor de c en 3. | c = 16 |
| −= | Resta | c -= 3 | c = c − 3 | Decrementa el valor de c en 3. | c = 10 |
| *= | Multiplicación | c *= 3 | c = c * 3 | Multiplica el valor de c por 3 | c = 39 |
| /= | División | c /= 3 | c = c / 3 | Divide el valor de c entre 3 | c = 4 |
| %= | Módulo o residuo | c %=3 | c = c % 3 | El nuevo valor de c es el residuo de dividir c entre 3 | c = 1 |

**Operadores asignación**

| Operador | Aplicación | Desglose |
|---|---|---|
| += | a += b | a = a + b |
| -= | a -= b | a = a - b |
| *= | a *= b | a = a * b |
| /= | a /= b | a = a / b |
| %= | a %= b | a = a % b |



# La clase Scanner en Java

La clase Scanner sirve para importar datos a nuestro programa por parte del usuario. Debemos tener en cuenta que el valor que ingresemos lo vamos a almacenar en una variable y luego la usaremos para realizar distintas operaciones con ella:

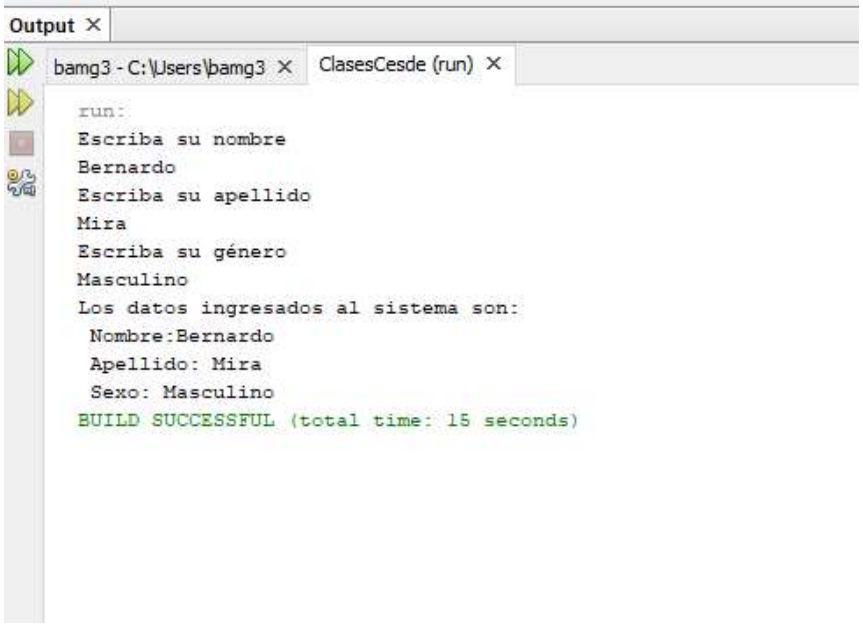
Start Page | Login.java | OperdoresIncrementoDecremento.java | **ClaseScanner.java**

Source | History

```
10      */
11  import java.util.Scanner;
12
13  public class ClaseScanner {
14
15      public static void main(String[] args){
16
17          Scanner leer = new Scanner(System.in);
18
19          String nombres, apellido, sexo;
20
21          System.out.println("Escriba su nombre ");
22          nombres = leer.nextLine();
23
24          System.out.println("Escriba su apellido ");
25          apellido = leer.nextLine();
26
27          System.out.println("Escriba su género ");
28          sexo = leer.nextLine();
29
30
31          System.out.println("Los datos ingresados al sistema son: \n Nombre:" + nombres + "\n Apellido: " + apellido + "\n Sexo: " + sexo);
32
33
34
35      }
36
37  }
38
```



# La clase Scanner en Java






| Tabla 3.11. Clases y métodos para convertir de String a numérico | |
|---|---|
| **Convertir a** | **Clase y método** |
| `double` | `Double.parseDouble` (cadena) ; |
| `float` | `Float.parseFloat` (cadena) ; |
| `long` | `Long.parseLong` (cadena) ; |
| `int` | `Integer.parseInt` (cadena) ; |
| `char` | `cadena.charAt`(posicion); |
| `short` | `Short.parseShort` (cadena) ; |
| `byte` | `Byte.parseByte` (cadena) ; |
| `boolean` | `Boolean.parseBoolean` (cadena) ; |



# La clase Math en Java



# Incremento y decremento en Java

| Posfijo i++ | Prefijo ++i |
|---|---|
| Antes de la variable | Después de la variable |
| Posfijo **se accede primeramente al dato de la variable sin importar que este incrementado o no**, se accede al dato anterior de la variable y **después la incrementa** | **Obtiene el dato incrementado** |

```
//gana un regalo por ganar una vida
int gift = 100+lives++; //Posfijo
System.out.println(gift);
System.out.println(lives);
```

```
//añadirle los puntos + vida que acaba de adquiri
int gift2 = 100 + ++lives; //prefijo
System.out.println(gift2);
System.out.println(lives);
//optienen el dato incrementado
```

**i = 20**

| Operador | Expresión | Ejemplo | Resultado |
|---|---|---|---|
| **Incremento** | i++ | x = i++; | Esta expresión primero asignará el valor de **i** a **x**, y después realizará el incremento de **i**, por lo que el valor de **x** será 20 y el valor de **i** será 21. |
| **Incremento** | ++i | x = ++i; | Esta expresión primero realizará el incremento de **i**, y después asignará el valor de **i** a **x**, por lo que el valor de **x** será 21 y el valor de **i** será 21. |
| **Decremento** | i-- | x = i--; | Esta expresión primero asignará el valor de **i** a **x**, y después realizará el decremento de **i**, por lo que el valor de **x** será 20 y el valor de **i** será 19. |
| **Decremento** | --i | x = --i; | Esta expresión primero realizará el decremento de **i**, y después asignará el valor de **i** a **x**, por lo que el valor de **x** será 19 y el valor de **i** será 19. |

| Instrucción | Equivalencia | a | b |
|---|---|---|---|
| int a = 0; | a = 0; | 0 | |
| a++; | a = a + 1; | 1 | |
| ++a; | a = a + 1; | 2 | |
| a--; | a = a - 1; | 1 | |
| --a; | a = a - 1; | 0 | |
| int b = a++; | b = a; a = a+1; | 1 | 0 |
| b = ++a; | a = a + 1; b = a; | 2 | 2 |
| b = a--; | b = a; a = a - 1; | 1 | 2 |
| b = --a; | a = a - 1; b = a; | 0 | 0 |

Operadores de incremento/decremento

| Ejemplo | Nombre | Efecto |
|---|---|---|
| ++$a | Pre-incremento | Incrementa $a en uno, y luego retorna $a. |
| $a++ | Post-incremento | Retorna $a, y luego incrementa $a en uno. |
| --$a | Pre-decremento | Decrementa $a en uno, luego retorna $a. |
| $a-- | Post-decremento | Retorna $a, luego decrem |



# Incremento y decremento en Java – Prefijo y postfijo

- `++` : `i++` es equivalente a `i = i + 1` .
- `--` : `i--` es equivalente a `i = i - 1` .

Podemos usar estos operadores de forma prefija ( `++i` ) o postfija ( `i++` ). La diferencia está en qué operación se ejecuta primero:

```
// Incremento postfijo:
int vidas = 5;
int regalo = 100 + vidas++;

System.out.println("Regalo: " + regalo + ", vidas: " + vidas);
// Regalo: 105, vidas: 6

// Incremento prefijo:
int vidas = 5;
int regalo = 100 + ++vidas;

System.out.println("Regalo: " + regalo + ", vidas: " + vidas);
// Regalo: 106, vidas: 6
```

Imagen tomada de curso de Platzi de Java SE



# Apuntes de lógica

Estructuras de control:

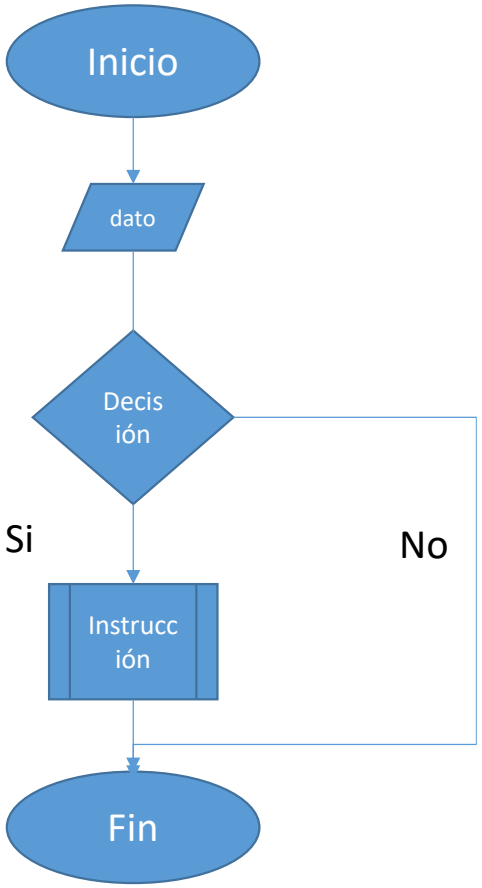
Son las estructuras que un lenguaje emplea para acceder y manipular  los datos en un programa. Podemos encontrar estructuras de tipo secuencial, Selectivas , repetitivas e iterativas.

Java usa las siguientes estructuras de control:

- If – (Simple)
- If – else(doble)
- Else if (multiple)
- Switch (multiple)
- While (repetitiva)
- Do while (repetitiva)
- For (Iterativa)
- For each (Iterativa)
- Try except

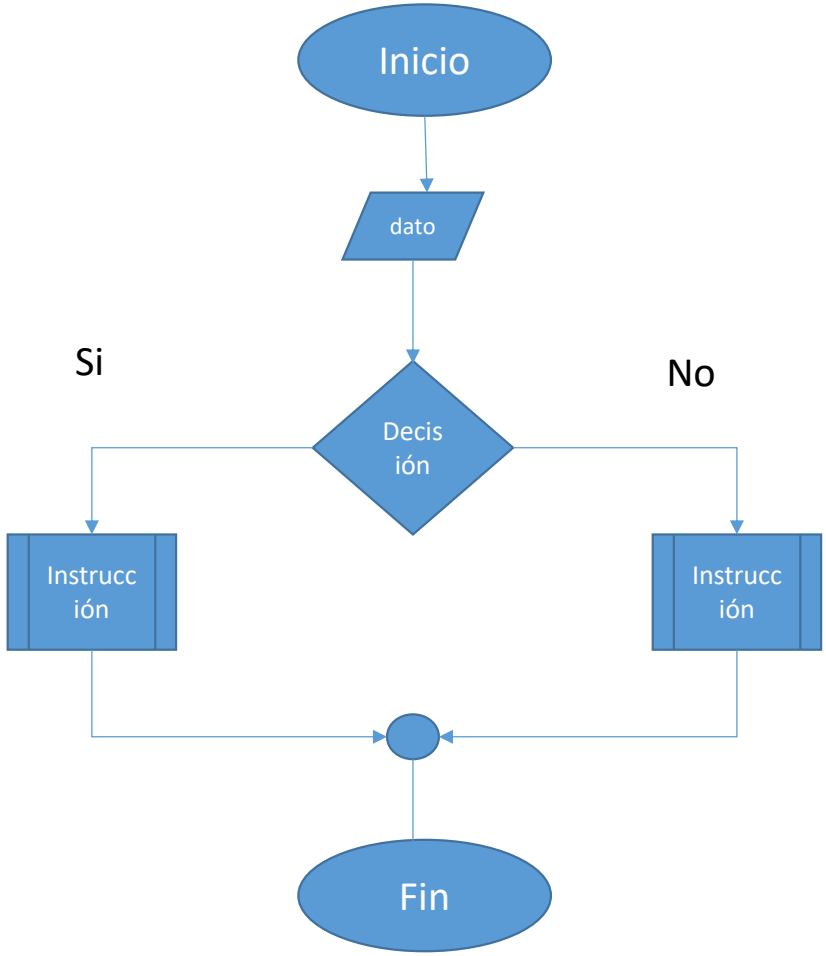


# If - Simple



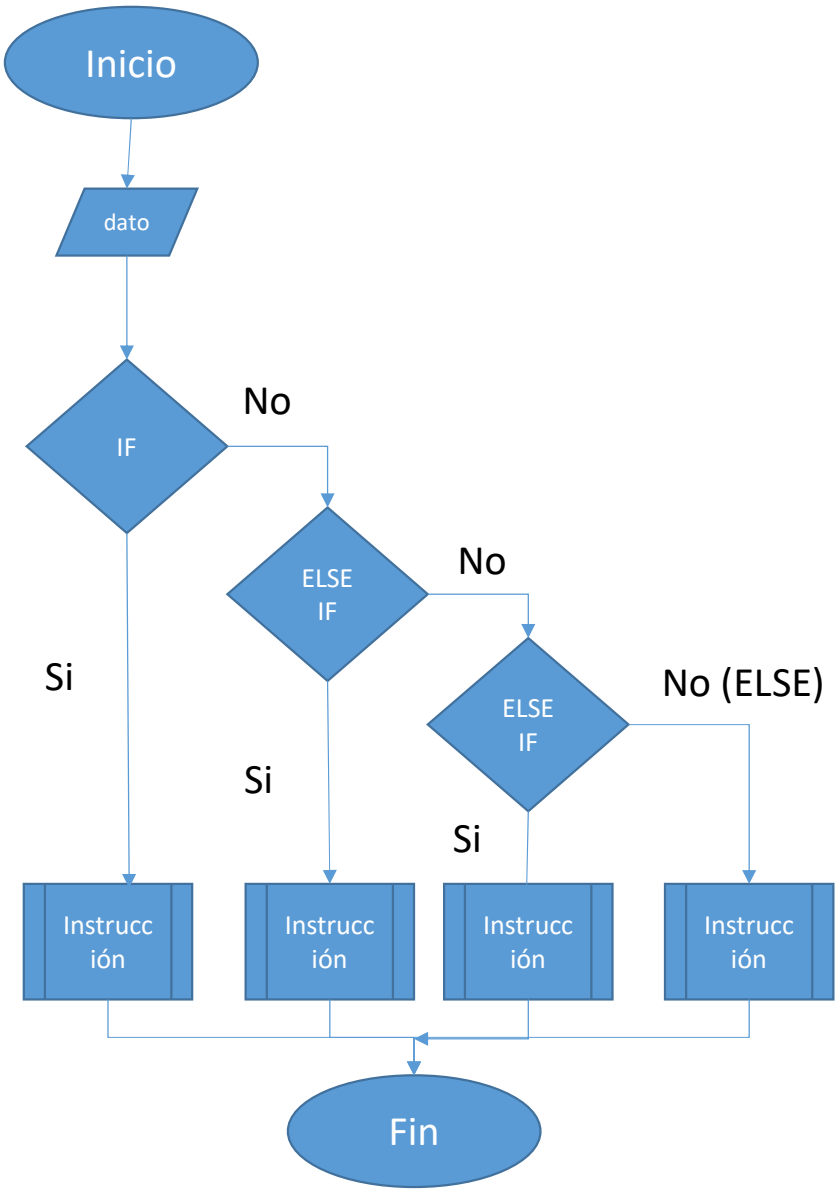



# If - else


Inicio
dato
Si
No
Decis
ión
Instrucc
ión
Instrucc
ión
Fin

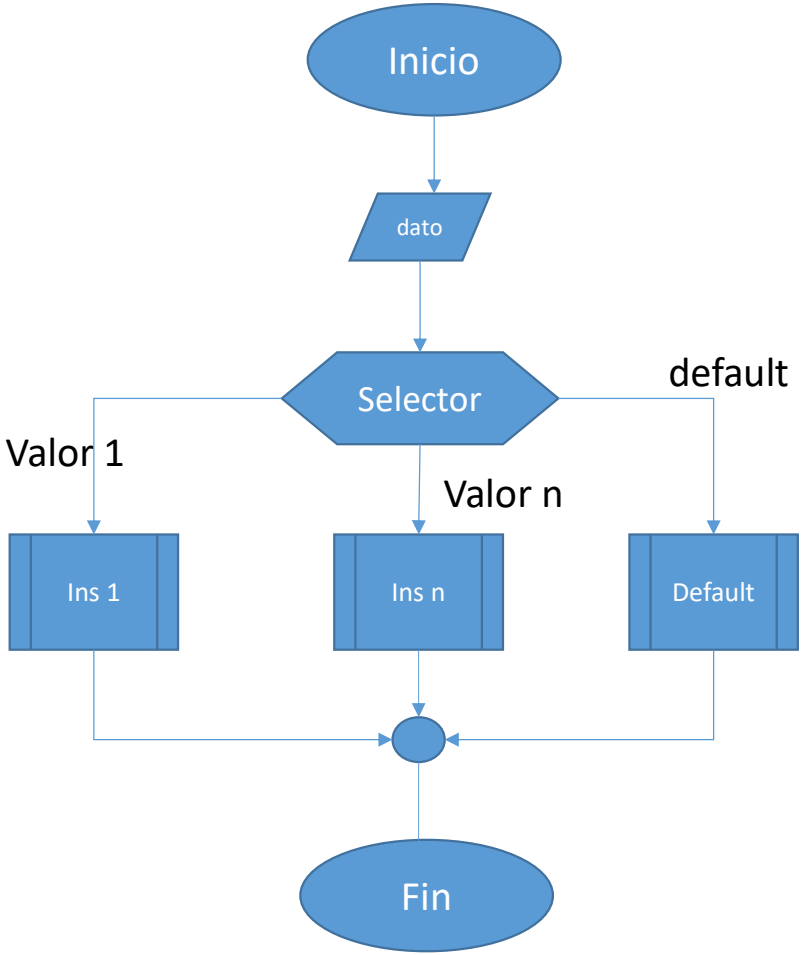

TALK
<CODE>
TO ME

# Else - If


Inicio
dato
IF
No
ELSE IF
No
ELSE IF
No (ELSE)
Si
Si
Si
Instrucción
Instrucción
Instrucción
Instrucción
Fin

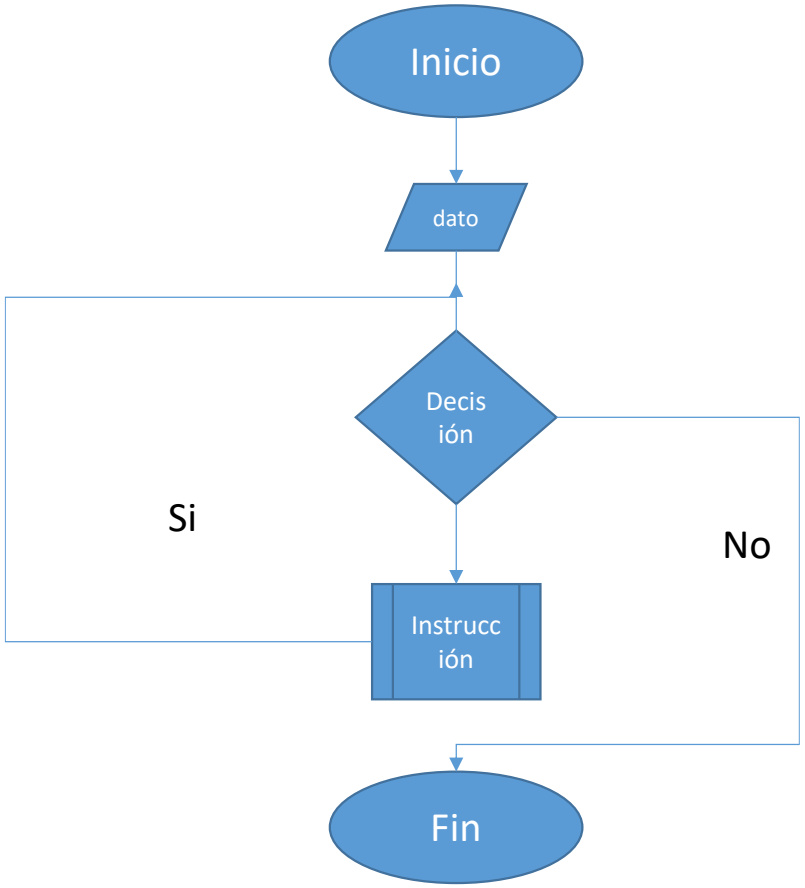

TALK
<CODE>
TO ME

# Switch


Inicio
dato
Selector
default
Valor 1
Valor n
Ins 1
Ins n
Default
Fin

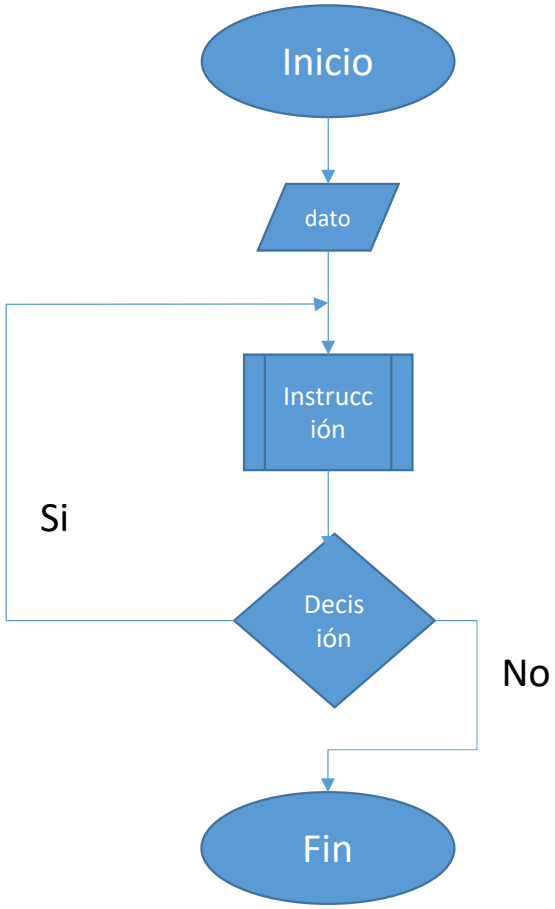

TALK
<CODE>
TO ME

# While



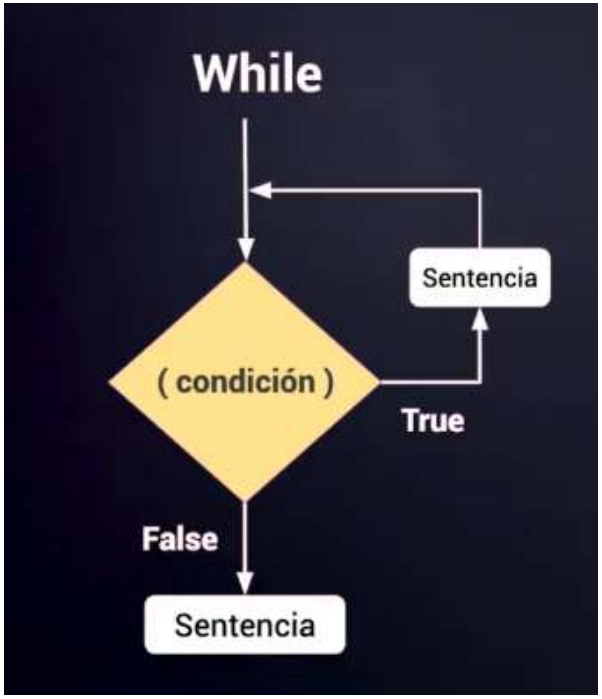



# Do - while






## Bucle while




```
while (condicion) {

    //instrucciones

}
```

En while vamos a entrar al ciclo al menos que la condición se cumpla.

En do while vamos a entrar al ciclo al menos una vez y luego valida si se entra al ciclo una segunda vez.

## Bucle do  while

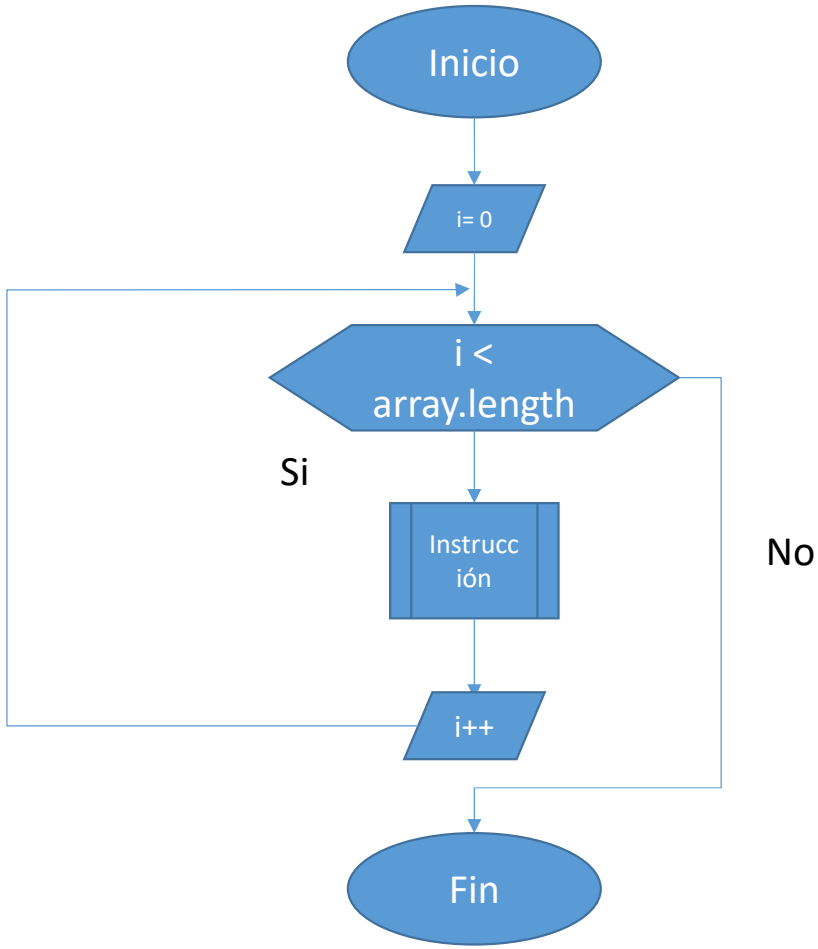
```
do {

    //instrucciones

} while (condicion);
```



# for


Inicio
i= 0
i < array.length
Si
No
Instrucc ión
i++
Fin

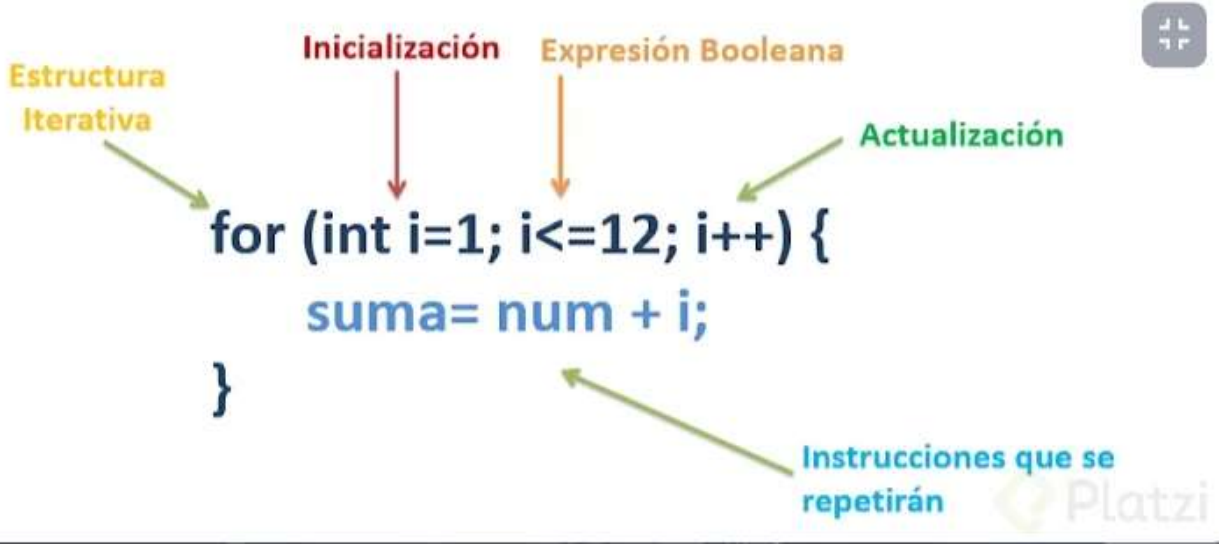

TALK
<CODE>
TO ME

# Ciclo for

```
for(inicializacion; condicion; incremento){

  //instrucciones

}
```






# Ciclo for

```java
*/
public class ForLoop {



    public static void main(String[] args) {

        for(int i=1; i<=10; i++){

          if (i%2 !=0){

              continue;


          }

            System.out.println(i+ " ,Es par");
        }


    }
```

ForLoop > main > for (int i = 1; i <= 10; i++)

Output - ClasesCesde (run)

```
run:
2 ,Es par
4 ,Es par
6 ,Es par
8 ,Es par
10 ,Es par
```

```java
*/
public class ForLoop {



    public static void main(String[] args) {

        for(int i=1; i<=10; i++){

          if (i%2 ==0){

              continue;


          }

            System.out.println(i+ " ,Es impar");
        }


    }
```

ForLoop > main > for (int i = 1; i <= 10; i++) > if (i % 2 == 0)

Output - ClasesCesde (run)

```
3 ,Es impar
5 ,Es impar
7 ,Es impar
9 ,Es impar
BUILD SUCCESSFUL (total time: 0 seconds)
```



# Ejemplo de operadores de incremento en Java usado dentro de un ciclo while y usando como validador el if- else Statement

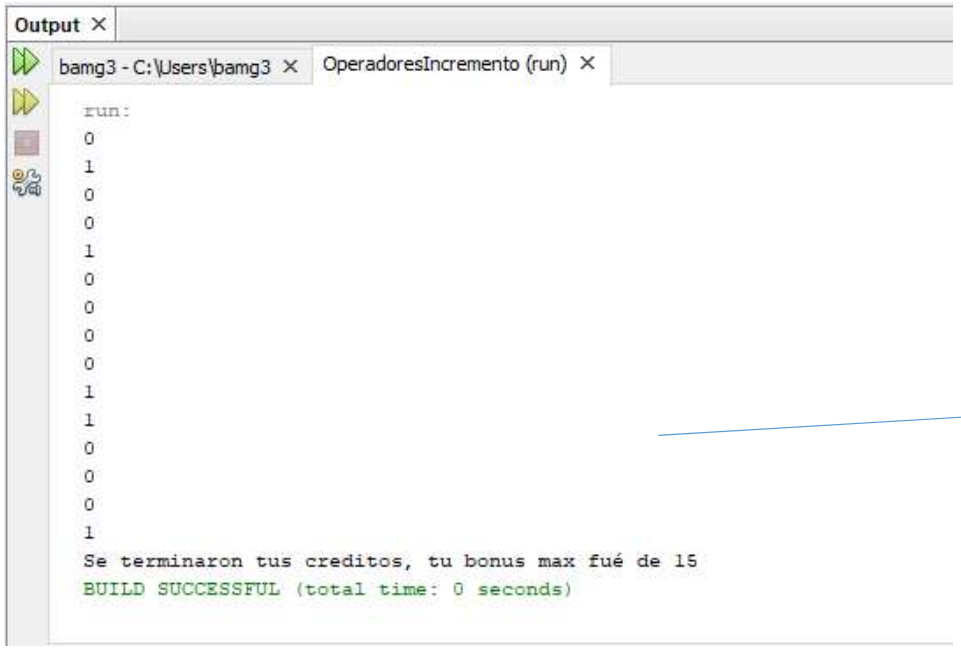
```
10    */
11
12  import java.util.Random;
13
14
15  public class OperdoresIncrementoDecremento {
16
17      public static void main(String[]args){
18
19
20          int credit = 5;
21          int bonus = 5;
22
23          Random rand = new Random();
24          //int horas=5;
25
26          //int pago = salary + horaExtra++;
27
28         //System.out.println(pago);
29
30         while( credit > 0){
31
32
33              int jugadas = rand.nextInt(2);
34
35
36              System.out.println(jugadas);
```

```
34
35
36              System.out.println(jugadas);
37
38              if (jugadas == 1){
39
40                  credit--;
41
42              } else {
43
44                  bonus++;
45
46
47
48              }
49
50          }
51
52          System.out.println("Se terminaron tus creditos, tu bonus max fué de " + bonus);
53
54      }
55
56  }
57
```

OperdoresIncrementoDecremento > main >



# Ejemplo de operadores de incremento en Java usado dentro de un ciclo while y usando como validador el if- else Statement

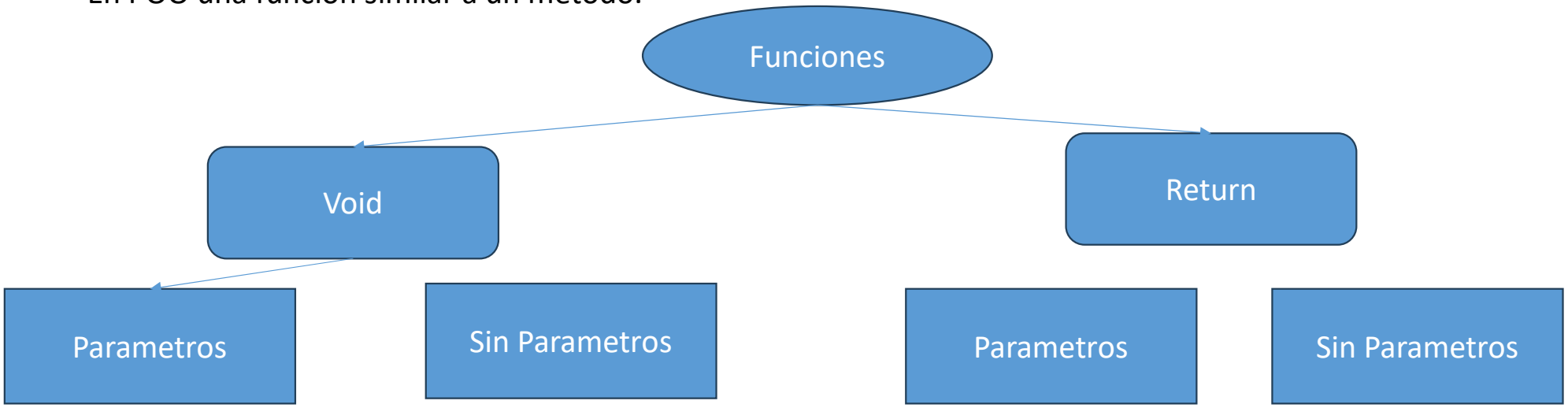



Salida



# Funciones

Una función es un bloque de código que contiene y ejecuta otros bloques de código:
Su principal característica es que puede ser llamado en cualquier parte del código.
Permite la reutilización de código
Facilita el mantenimiento.
Siempre se expresan en verbo infinitivo: (Servir, copiar , imprimir , mostrar)
Su sintaxis es simple.
En POO una función similar a un método.



Sintaxis de una función vacía sin parametros:

```
public  void  servirCopaDeVino(   ){



}


// Ejecucción de una función

servirCopaDeVino();
```

# Java docs

```
/**
* Descripción
*    @param
*    @ return
*/
```

| Syntax | Description | Description |
|---|---|---|
| **@author** | Es el autor de la clase | @author name-text |
| **{@code}** | Muestra texto en formato de código sin que sea interpretado como código HTML | {@code text} |
| **{@docRoot}** | Indica la ruta relativa al directorio raíz del documento generado desde cualquier página generada. | {@docRoot} |
| **@deprecated** | Se pone indicando que esta API no debe usarse más | @deprecated deprecatedtext |
| **@exception** | Se indica cuando es vulnerable a lanzar una excepción, en seguida se ponen las clases de las excepciones posibles | @exception class-name description |
| **{@inheritDoc}** | Indica la herencia o implementación procedentora | |
| **{@link}** | Hace un enlace a la miembro indicado | {@link package.class#member label} |
| **{@linkplain}** | Lo mismo que en anterior excepto que la etiqueta del enlace se muestra en texto plano | {@linkplain package.class#member label} |
| **@param** | Añade parámetros con nombres específicos, seguido de su descripción. | @param parameter-name description |
| **@return** | Asigna un parámetro de retorno, seguido de su descripción | @return description |
| **@see** | Se añade para manejar referencias, o información relacionada | @see reference |
| **@serial** | Se utiliza para indicar campos o clases serializables. | @serial field-description include exclude |
| **@serialData** | Se utiliza para documentar métodos que generan una serialización | @serialData data-description |
| **@serialField** | Se utiliza para documentar objetos serializados | @serialField field-name field-type field-description |
| **@since** | Se añade en el encabezado para especificar desde cuándo fue creado | @since release |
| **@throws** | Es sinónimo con la etiqueta @exception | @throws class-name description |
| **{@value}** | Cuando es usado sin argumento se usa para especificar un campo estático en otro caso se usa para mostrar el valor constante. | {@value package.class#field} |
| **@version** | Se añade en el subtítulo con la versión especificada. | @version version-text |

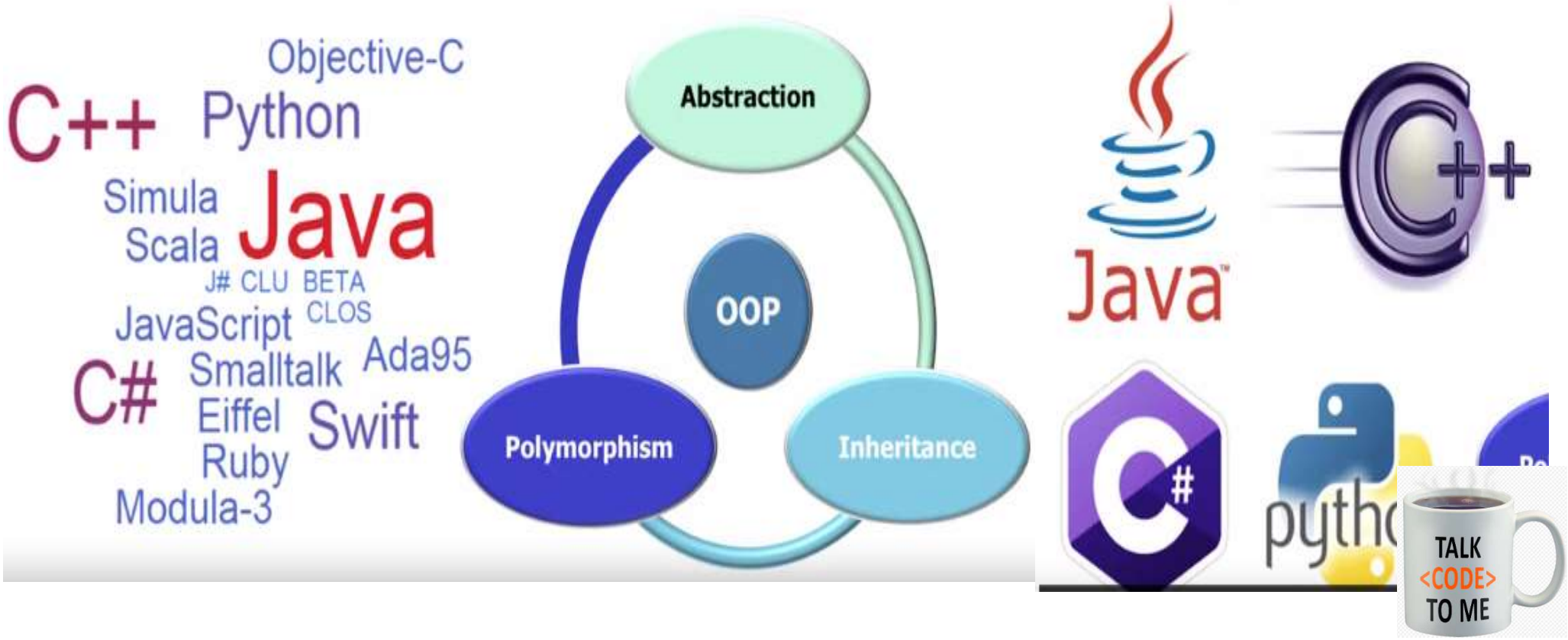


# Manejo de errores en Java
# (try – catch)

```
public class DataLoader {
  public void loadData(Path file) {
    try {
      Files.readLines(file);
      // process information
    } catch(IOException e) {
    // error corrections
    }
  }
}
```



# OOP – Object Oriented Programming Language



# Objetivo de la Programación Orientada a Objetos en Java

El Objetivo de la programación Orientada a Objetos es separar responsabilidades por Clases y Métodos, hacer el código entendible , mantenible y escalable.

También son objetivos de la POO evitar la duplicidad de código y promover la reutilización de código

Es importante tener en cuenta que la POO tiene un principio de alta Cohesión y Bajo Acoplamiento:

En POO los datos y la funcionalidad de una entidad , van de la mano:

Recomiendo realices  la siguiente lectura complementaria para comprender mejor los temas:

https://www.aluracursos.com/blog/poo-que-es-la-programacion-orientada-a-objetos



# ¿Que es una Clase ?

*“Una clase o class es un tipo de datos definido por el usuario”* (Ceballos, 2018, p. 14). También se puede definir como *“una plantilla para la creación de objetos de datos a partir de un modelo establecido. Estos tipos de dato son abstractos y contienen atributos y métodos, a través de una clase se implementa un concepto abstraído de la realidad”* (Flórez, 2012, p. 25).




- Una clase es una entidad abstracta.
- Es un tipo de clasificación de datos.
- Define el comportamiento y atributos de un grupo de estructura y comportamiento similar.

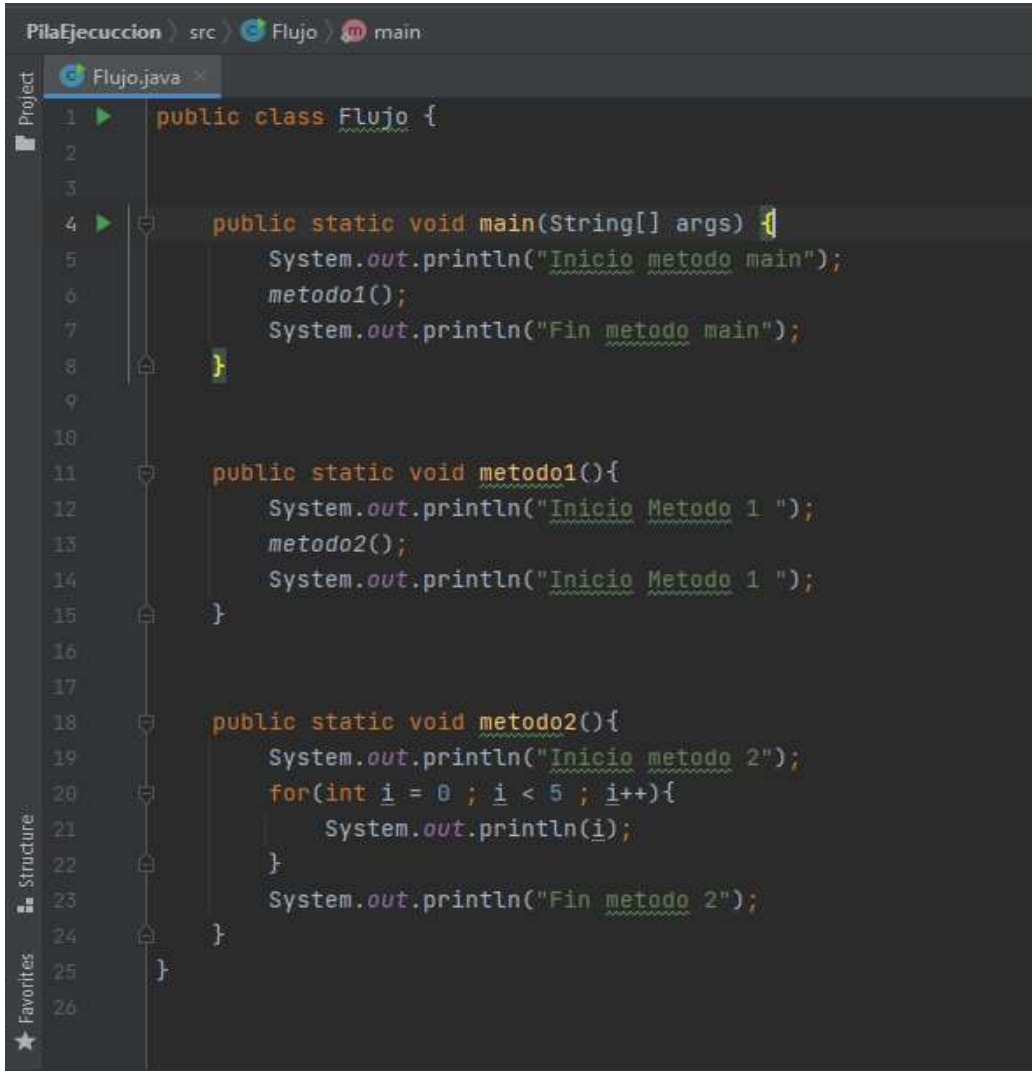
En Java se debe implementar cada clase en un archivo independiente con extensión **.java**. Para el siguiente ejemplo, el archivo debe denominarse **MiClasejava.**

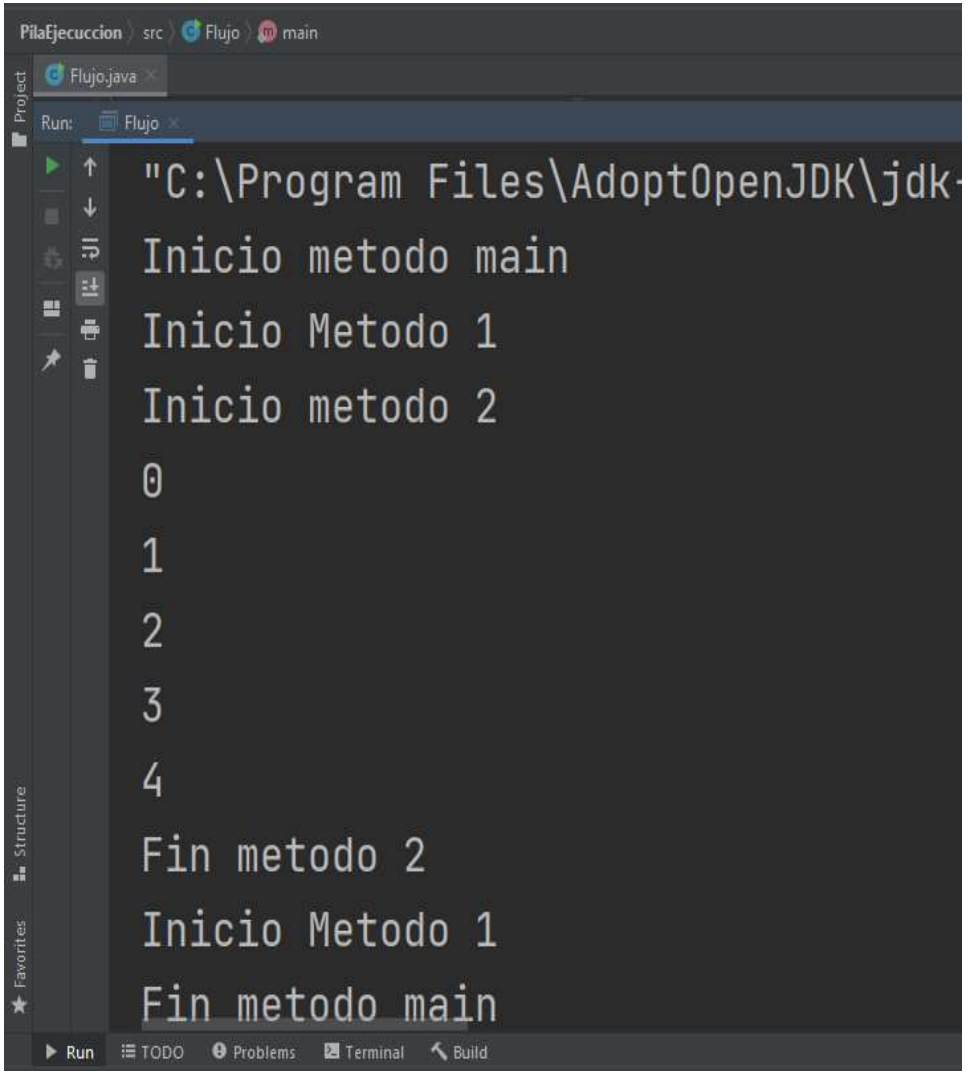
Sintaxis de la clase:

```
public class MiClase{
   //Definición de atributos
   //Definición de métodos
```

| **Ejemplo:** | |
|---|---|
| Clase Coche<br>Métodos: arrancar, avanzar, parar,...<br>Atributos: color, velocidad, etc. | Nombre de la clase<br>Métodos (funciones)<br>Atributos (datos) |

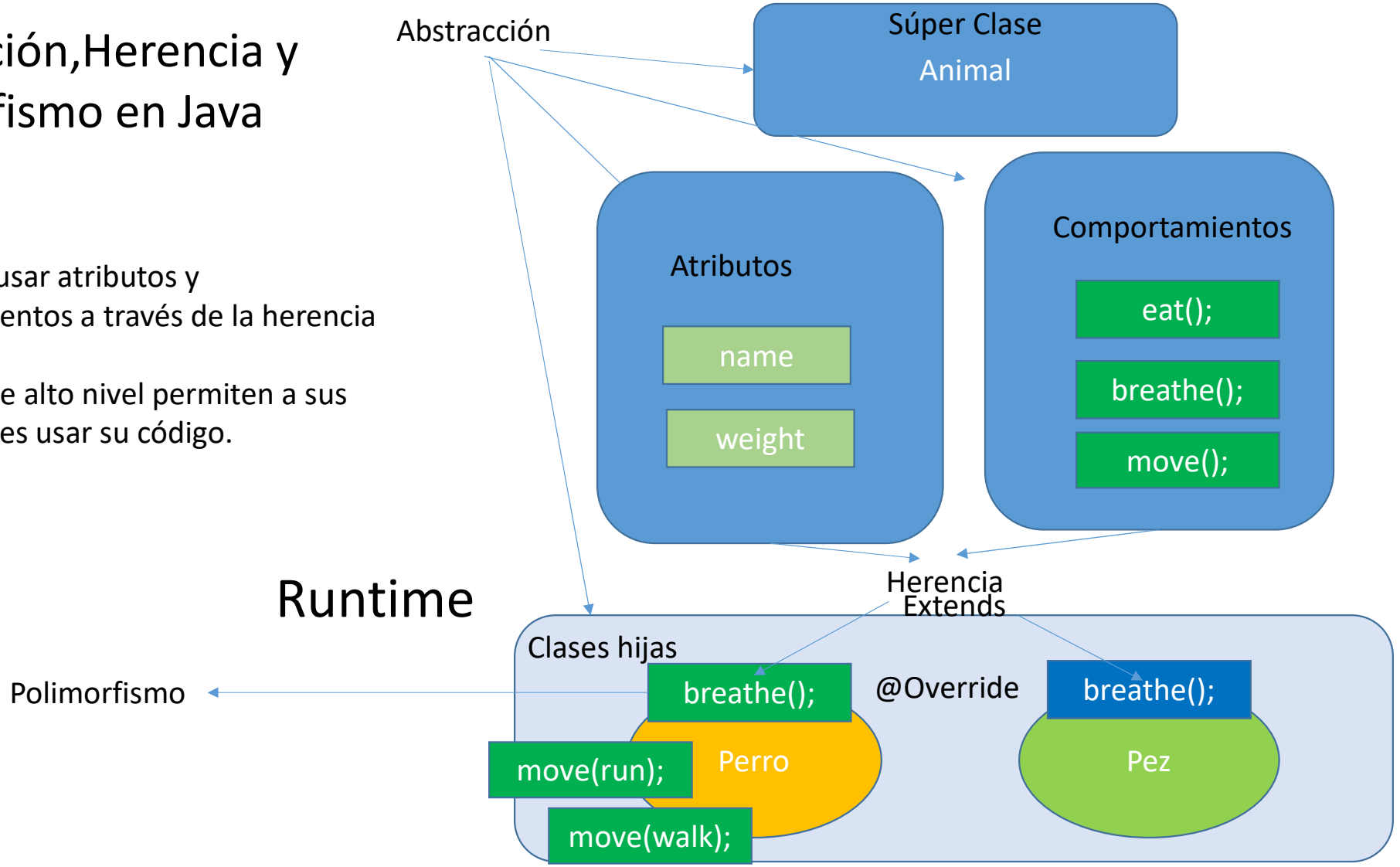
# La pila de ejecucción en Java



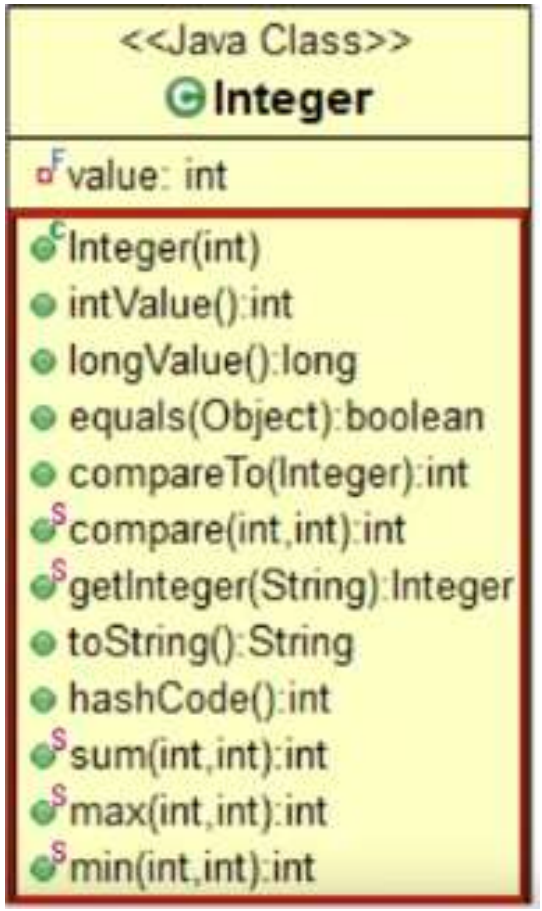



# Abstracción,Herencia y polimorfismo en Java

- Permite reusar atributos y comportamientos a través de la herencia de clase .
-Las clases de alto nivel permiten a sus descendientes usar su código.

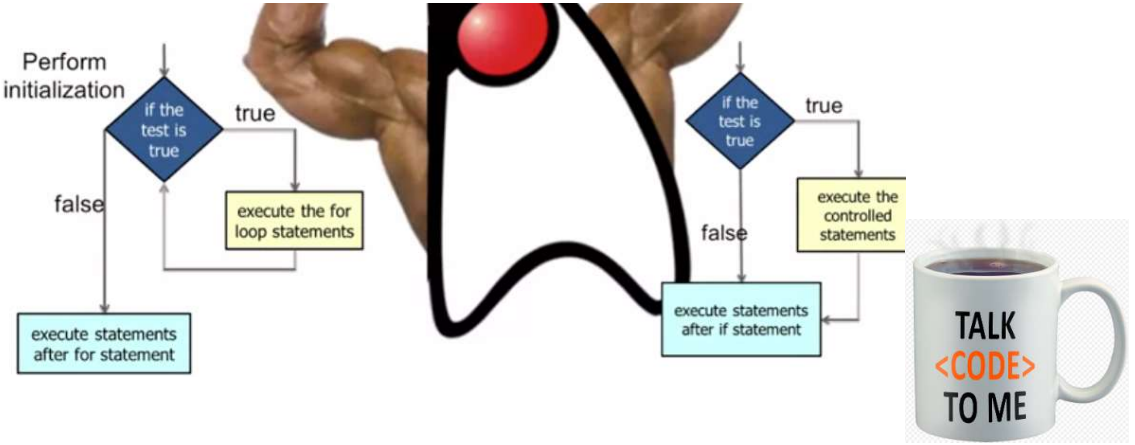


La abstracción en JAVA

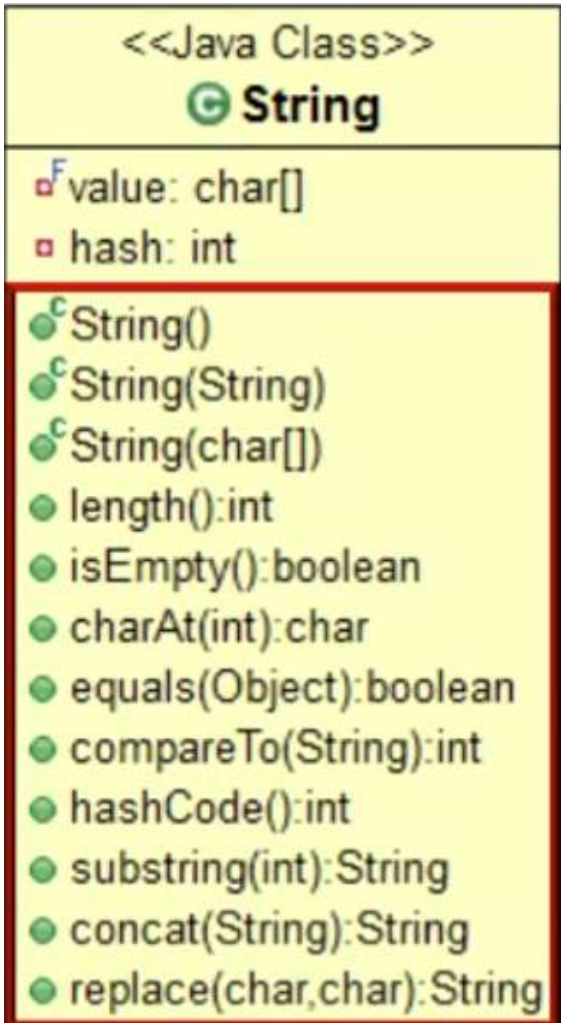


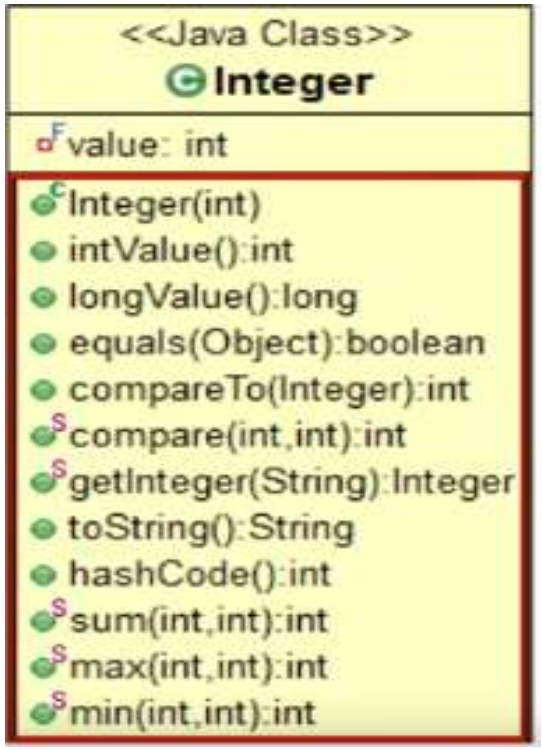

Esta permite definir los aspectos importantes de un objeto , sin distraerse en otros menos importantes, partiendo de un valor , en este caso un Integer , que contiene unos métodos, esto es conocido como abstracción de datos.

Por otro lado tenemos Abstracciones de control, que determinan el orden en el que se evalúan o se ejecutan las sentencias individuales:

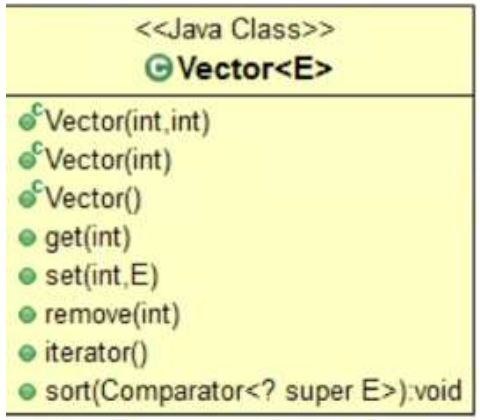


# Ejemplos de Abstracción de datos



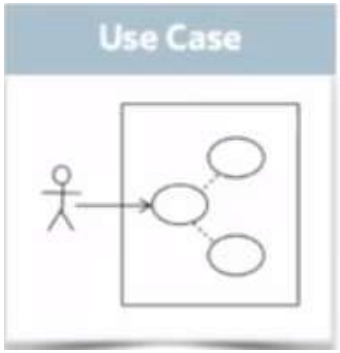



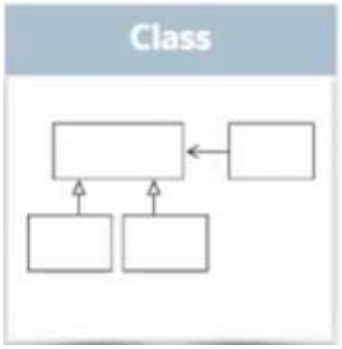

En términos concretos , la abstracción permite identificar las generalidades de una entidad desde el contexto del problema a resolver y determinar sus características (Atributos) y comportamientos ( métodos) para modelar la clase.

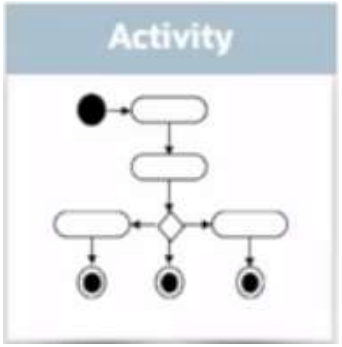


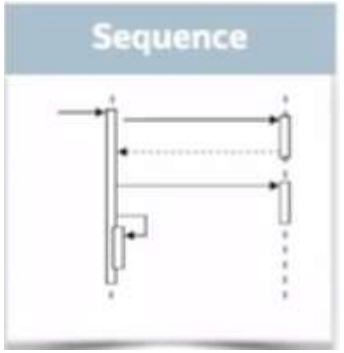


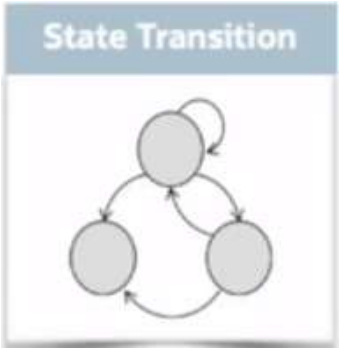

# Uso de UML para el diseño de aplicaciones en Java



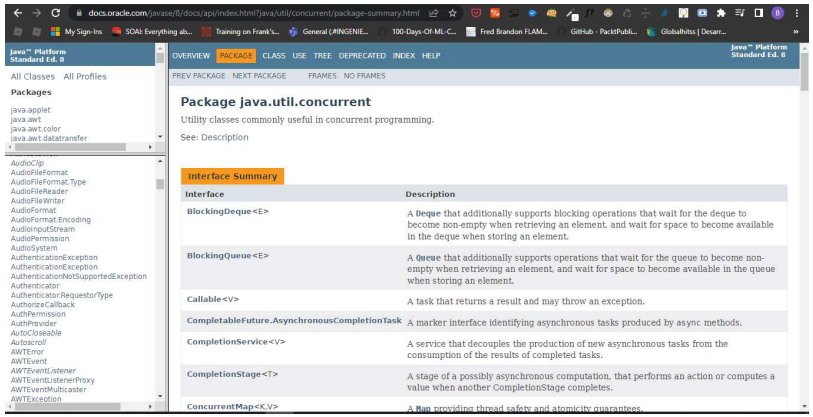









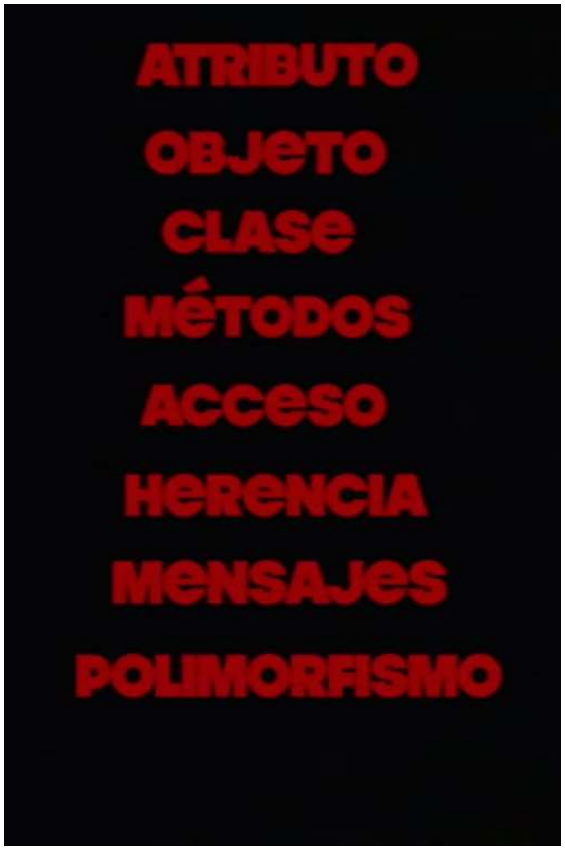
# Java – Documentación Oficial




https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html



# Conceptos bases de Programación Orientada a Objetos






# Composición de una clase en Java

En Java el nombre de la clase debe ser exactamente igual al nombre del archivo

NombreClase.java

```java
//Librerias o archivos Java y paquetes

//Nombre de la Clase
public class NombreClase {

    //Atributos o Variables de la Clase Globales

    //Constructor
    public NombreClase() {

    }

    //Metodo
    public void metodo(){

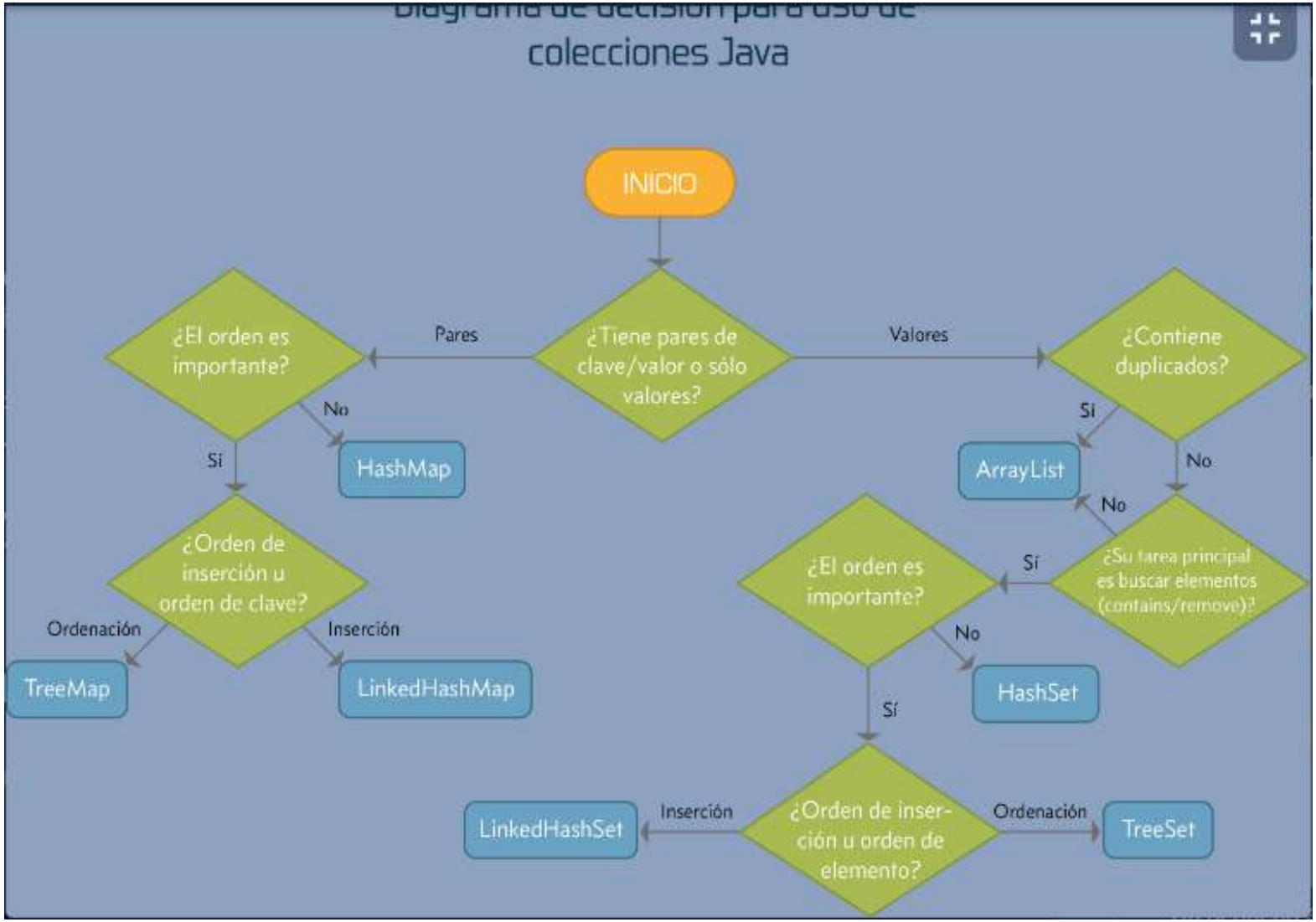
    }

    //Metodo Principal
    public static void main(String[] args) {

    }
}
```

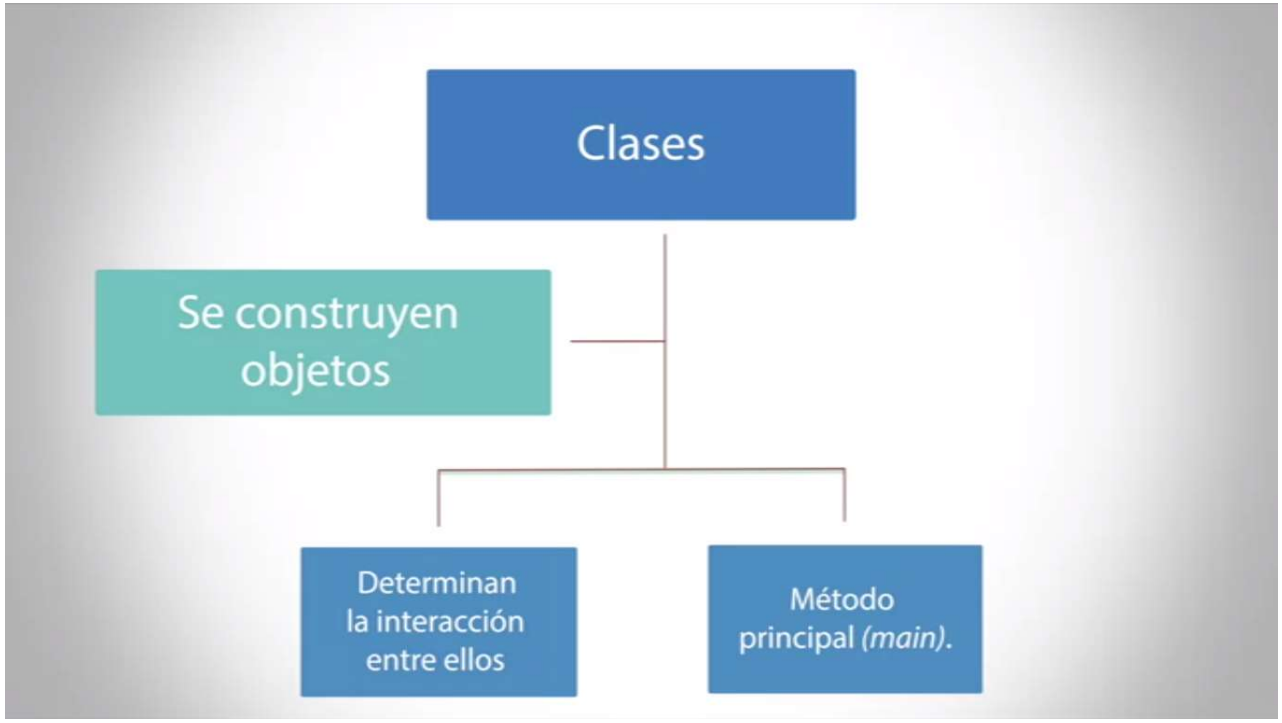


# Estructura de datos en Java






# Programación Orientada  a Objetos

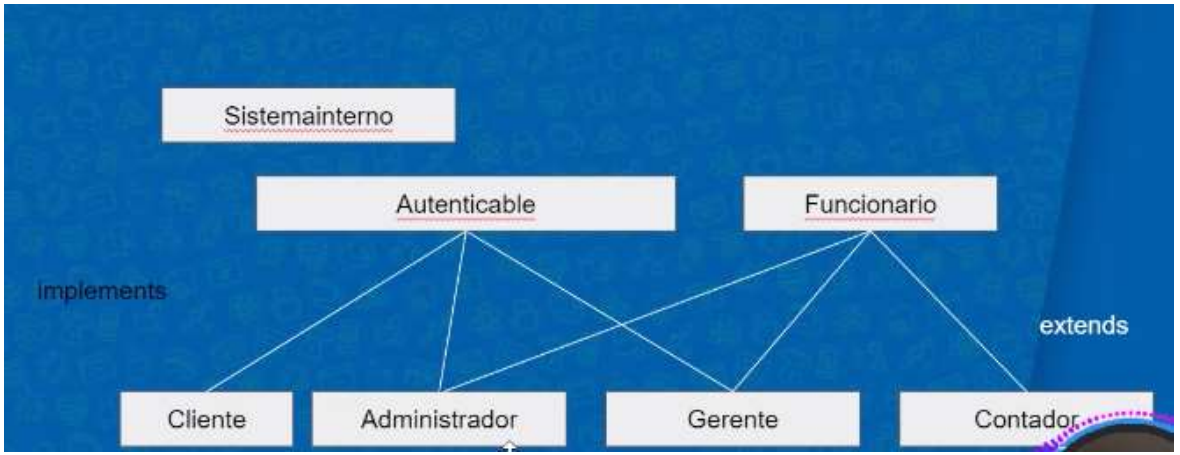





# Herencia

Los pilares de la herencia son la reutilización de código y el polimorfismo.  Para esto usamos extends e implements.

La herencia captura lo que es común y aísla lo que es diferente entre clases.






# Constructores de la clase

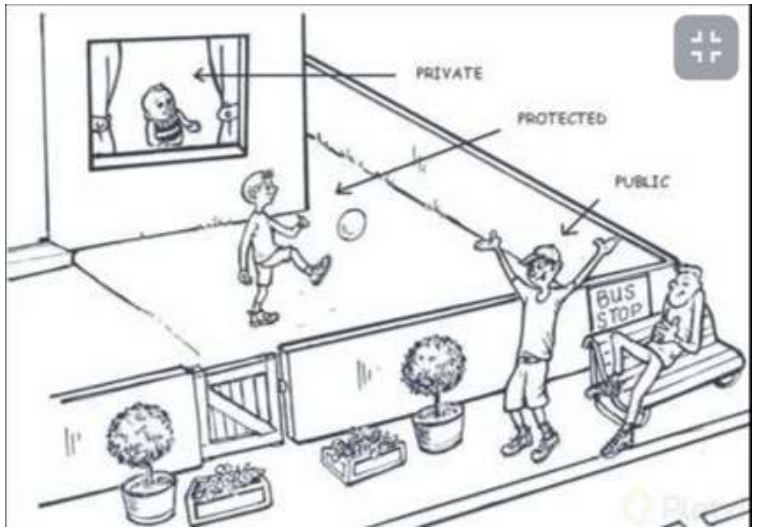
- Los constructores no se heredan , pero si se pueden llamar en la clase hija, usando la palabra reservada super;

- El constructor por default de java deja de existir en el momento que se declara uno , pero este puede ser agregado manualmente si así se desea.



# Modificadores de acceso : Encapsulamiento

| Modificador | Clase | Package | Subclase | Otros |
|---|---|---|---|---|
| public | ✓ | ✓ | ✓ | ✓ |
| protected | ✓ | ✓ | ✓ | ● |
| default | ✓ | ✓ | ● | ● |
| private | ✓ | ● | ● | ● |

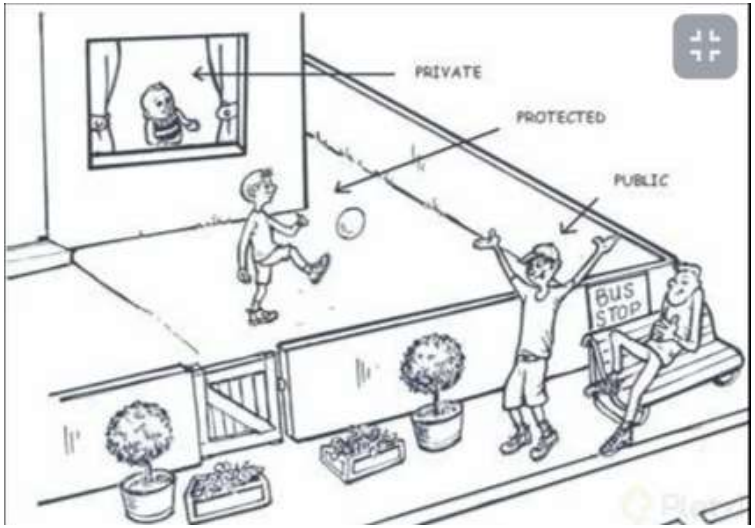





# Modificadores de acceso : Encapsulamiento

## Sobrecarga de Constructores

| Modificador | Clase | Package | Subclase | Otros |
|---|---|---|---|---|
| public | ✓ | ✓ | ✓ | ✓ |
| protected | ✓ | ✓ | ✓ | ● |
| default | ✓ | ✓ | ● | ● |
| private | ✓ | ● | ● | ● |

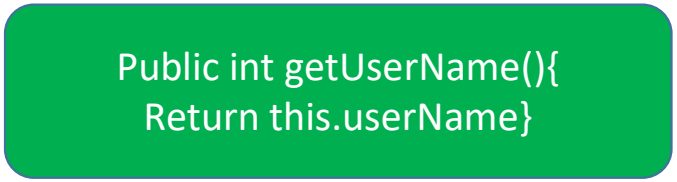


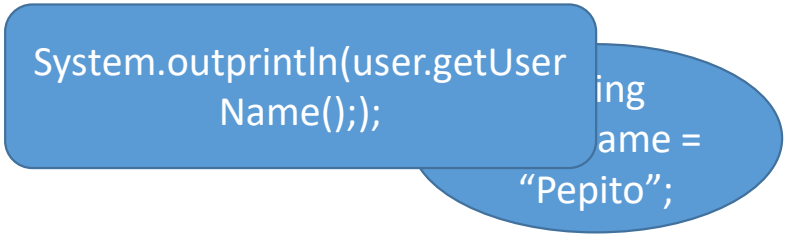




private String userName;


public void setUserName(string
name){ this.userName = name;
}


Public int getUserName(){
Return this.userName}

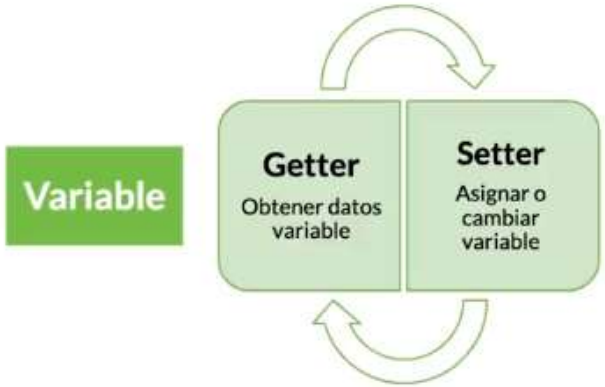

System.outprintln(user.getUser
Name(););
ing
ame =
“Pepito”;

Getters and Setters : accediendo a las variables encapsuladas de una manera eficiente sin afectarlas directamente.

## Getters y Setters

**Leer/Escribir** específicamente los valores de las **variables** miembro.

## Getters y Setters






# Variables vs objetos

**Clases Wrapper / Objeto primitivo**

Byte
Short
Integer
Long
Float
Double
Character
Boolean

**String**

Poseen métodos que les permite manipular esta cadena de variables.

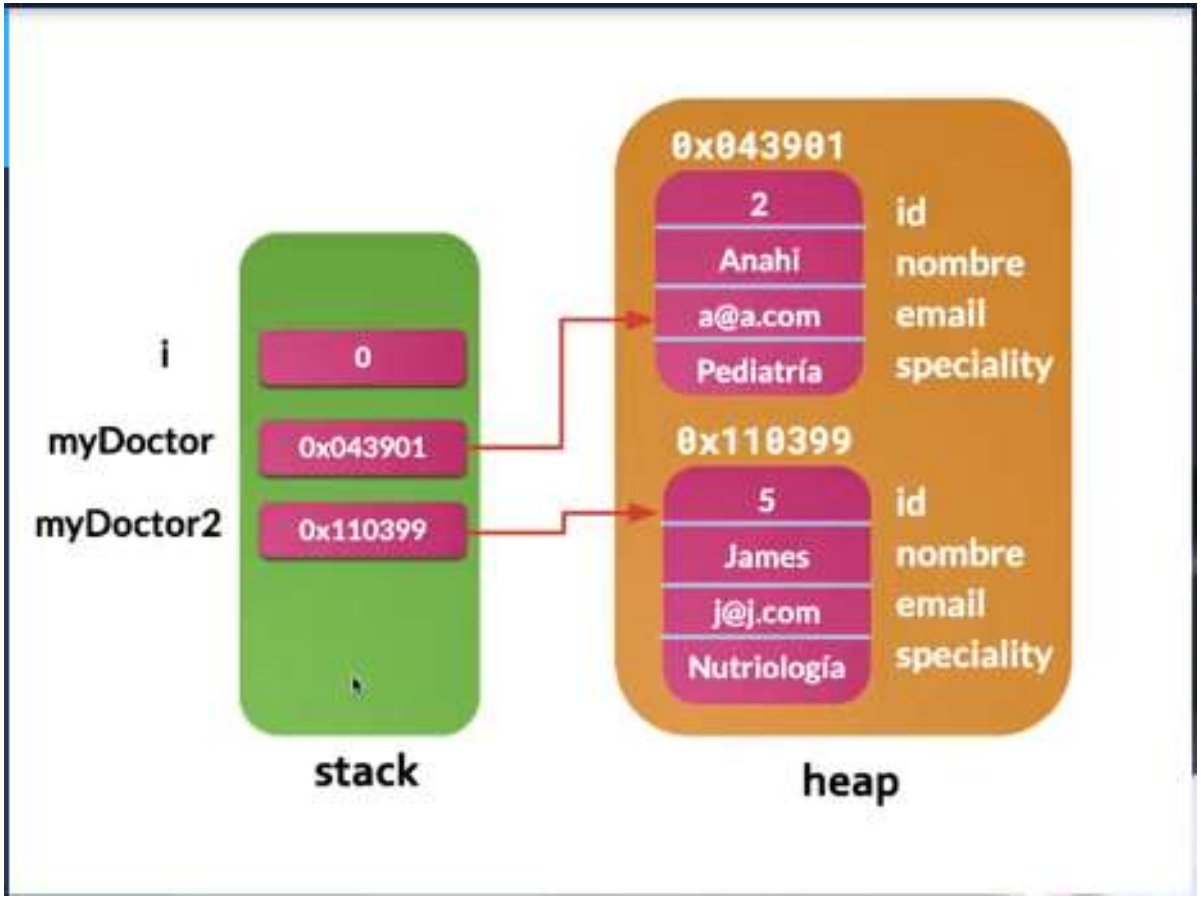
En el caso de los objetos , puedo hacer parseos entre objetos.

Tabla 3.11. Clases y métodos para convertir de String a numérico

| Convertir a | Clase y método |
|---|---|
| `double` | `Double.parseDouble` (cadena) ; |
| `float` | `Float.parseFloat` (cadena) ; |
| `long` | `Long.parseLong` (cadena) ; |
| `int` | `Integer.parseInt` (cadena) ; |
| `char` | `cadena.charAt`(posicion); |
| `short` | `Short.parseShort` (cadena) ; |
| `byte` | `Byte.parseByte` (cadena) ; |
| `boolean` | `Boolean.parseBoolean` (cadena) ; |



# Ubicación en memoria de Variables vs objetos






| this | super |
| --- | --- |
| La palabra clave **this** se refiere al objeto actual en un método o constructor. El uso más común de este término es eliminar la confusión entre los atributos de clase y los parámetros del mismo nombre (porque un atributo de clase está sombreado por un parámetro de método o constructor). Además de este uso, esta palabra se puede utilizar para:<br>• Invoca al constructor de la clase actual;<br>• Invocar el método de clase actual;<br>• Devuelve el objeto de clase actual;<br>• Pase un argumento en la llamada al método;<br>• Pase un argumento en la llamada al constructor; | La palabra clave **super** se refiere a objetos de superclase (madre). Se utiliza para llamar a los métodos de la superclase y para acceder al constructor de la superclase. El uso más común de la palabra clave super es eliminar la confusión entre superclases y subclases que tienen métodos con el mismo nombre.<br>• super:<br>◦ utilizado para referirse a la variable de instancia de la clase inmediatamente superior (clase madre);<br>◦ se usa para invocar métodos de la clase inmediatamente superior (clase madre);<br>• super():<br>◦ se utiliza para invocar al constructor de la clase inmediatamente superior (clase madre); |



# Polimorfismo

El polimorfismo es la capacidad del objeto de ser referenciada por varios tipos de variables: Por ejemplo, si hay una clase *Gerente* que sea hija de *Funcionario*. Un objeto de tipo *Gerente* puede ser referenciado como tipo *Funcionario* también.

Tenemos polimorfismo cuando una clase hace un extends a otra o cuando implementa una interfaz: es decir , tenemos polimorfismo por herencia o por interfaz.



# Interfaces y clases Abstratas

- En la interfaz es necesario implementar todos los métodos.
-

Clases abstratas es una combinación de herencia con interfaces . No requiere que se le implenten todos los métodos y no requiere ser instanciada, además no se puede crear instancias de una clase abstrata. Tampoco se pueden crear objetos. Lo que si se puede hace con la clase abstrata es heredarla (Extends)



# clases Abstractas

Estas no pueden ser instanciadas , para crear una instancia , primero debemos crear una clase hija no abstracta. Esto, porque lo abstracto (La clase) , no puede volverse concreto (objeto).

Esta puede tener métodos abstractos , pero también métodos concretos, es decir con implementación.

Son útiles cuando queremos utilizar comportamientos y atributos basados en clases con comportamientos  comunes . Los beneficios de la herencia siguen siendo validos.

En el caso de las clases abstractas podemos  usarla para referencia un objeto a partir de una clase mas especifica que herede de esta:

Funcionario funcionario = new Gerente();

Esto es considerado desacoplamiento de clases , donde usamos la clase mas general para referenciar los objetos , pero usamos el constructor de la clase mas especifica para instanciar el objeto.



# Métodos Abstractos

Los métodos abstractos al igual que clases abstractas , son puramente conceptuales, es decir estos no tiene implementación , solo definen la firma ( visibilidad , retorno, nombre del método y parámetros).

```
public abstract  boolean  calcularBono(int valor);
```

Usamos métodos abstractos cuando queremos forzar a un hijo concreto (clase concreta )  a implementar un método.



# INTERFAZ

La interfaz es la manera de JAVA de implementar la herencia múltiple, de una manera limpia, usando polimorfismo.

La interfaz tiene métodos sin implementación , por ende todos sus métodos son abstractos. A diferencia de las clases abstractas que puede contener métodos concretos y abstractos.

La clase que implementa una interfaz “firma” un contracto que lo obliga a implementar los métodos de esta.

Las interfaces no tienen atributos , no tienen constructores , ni los métodos tienen implementación. Los métodos por defecto son abstractos y por ende toda interfaz es abstracta.

Una ventaja de usar una interfaz es que los métodos se pueden llamar de manera segura en la clase que la implementa.



## Clase Abstracta e Interfaz

| Interfaz | Clase Abstracta |
|---|---|
| Palabra clave: **implements** | Palabra clave: **extends** |
| La interfaz admite herencia múltiple | La clase abstracta no admite herencia múltiple |
| Proporciona una abstracción absoluta y no puede tener implementaciones de métodos. | Puede tener métodos con implementaciones. |
| La interfaz no contiene constructor | La clase abstracta contiene constructor |
| La interfaz no puede tener modificador de acceso, por defecto todos los accesos son públicos | La clase abstracta puede tener modificadores de acceso. |
| Los miembros de la interfaz no pueden ser static | Solo los miembros completamente abstractos pueden ser static |

https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html

https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html



# ENUMS en Java

EnumExample.java

```java
package enums;

public enum EnumExample {

    LUNES,MARTES,MIERCOLES,JUEVES,VIERNES,SABADO,DOMINGO

}
```

Main.java

```java
package enums;

public class Main {

    public static void main(String[] args) {

        for ( EnumExample en: EnumExample.values()){
            System.out.println("El dia de la semana es: " + en);
        }


        EnumExample domingo = EnumExample.DOMINGO;

        System.out.println(domingo.name());
        System.out.println(domingo.ordinal());
        System.out.println(domingo.toString());




    }
}
```

https://youtu.be/EoPvlE85XAQ



# Diferencia entre anulación y Sobrecarga de métodos en Java

| Anulación de método | Sobrecarga de métodos |
|---|---|
| Los métodos deben tener el mismo nombre y firma | Los métodos deben tener el mismo nombre pero una firma diferente |
| Es un ejemplo de polimorfismo en tiempo de ejecución. | Es un ejemplo de polimorfismo en tiempo de compilación. |
| El tipo de retorno de todos los métodos debe ser el mismo | El tipo de devolución puede ser el mismo o no |
| La anulación ocurre entre dos o más clases | La sobrecarga ocurre en la misma clase |
| Los parámetros deben ser los mismos | Los parámetros deben ser diferentes |




Fuente : https://www.delftstack.com/es/howto/java/override-java/

# Casos de polimorfismo en Java

En java la clase mas genérica (padre) puede ser adaptada a la clase mas especifica (Clase hija) .

```
public class Moto extends Vehiculo{


}

Vehiculo vehiculo = new Vehiculo();

Moto moto = new Moto();

Vehiculo vehiculo = new Moto();
```



Caso de polimorfismo: Todas las Motos son Vehiculos, pero no todos los Vehiculos son Motos

Esto es posible por las referencias , es decir , el nuevo objeto vehiculo apunta a las referencia de la clase hija Moto. Debemos tener en cuenta que Moto extiende de Vehiculo. Adicional , en el caso que sea al contrario, no funciona.



# Pasar un objeto como argumento de un método en Java

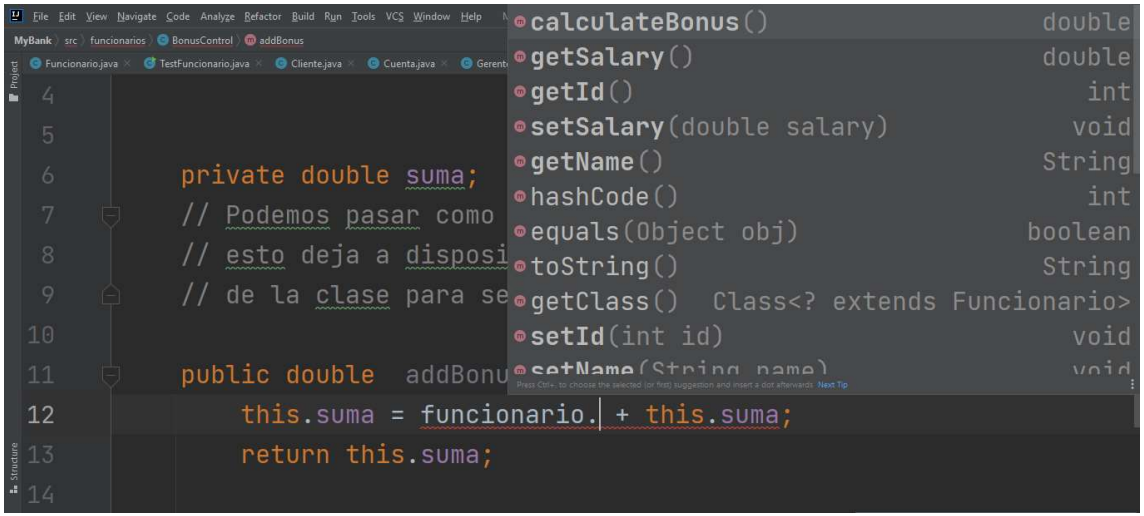
Java nos permite pasar un objeto como argumento de un método, esto deja disponible para el método el llamado de los atributos y los métodos del objeto dentro de la función.

```
public double ventasNetas( Vehiculo vehiculo){

        this.suma = vehiculo.getPrecio() + this.suma;

        return this.suma;

}
```






# Bajo acoplamiento

Con composiciones de clase e interfaces tendremos mas flexibilidad con el código, ya que no será necesario estar apegados al acoplamiento que propone la herencia.

Cuenta.java | Autenticable.java | Cliente.java | SistemaInterno.java | TestFuncionario.java | Contador.java | AdminSys.java | AutenticacionUtil.java | Gerente.java | TestGerente.java

```java
package funcionarios;

import org.w3c.dom.ls.LSOutput;

public class AdminSys extends Funcionario implements Autenticable{

    //private int password = 1234;
    private AutenticacionUtil util;


    public AdminSys(){
        this.util = new AutenticacionUtil();
    }

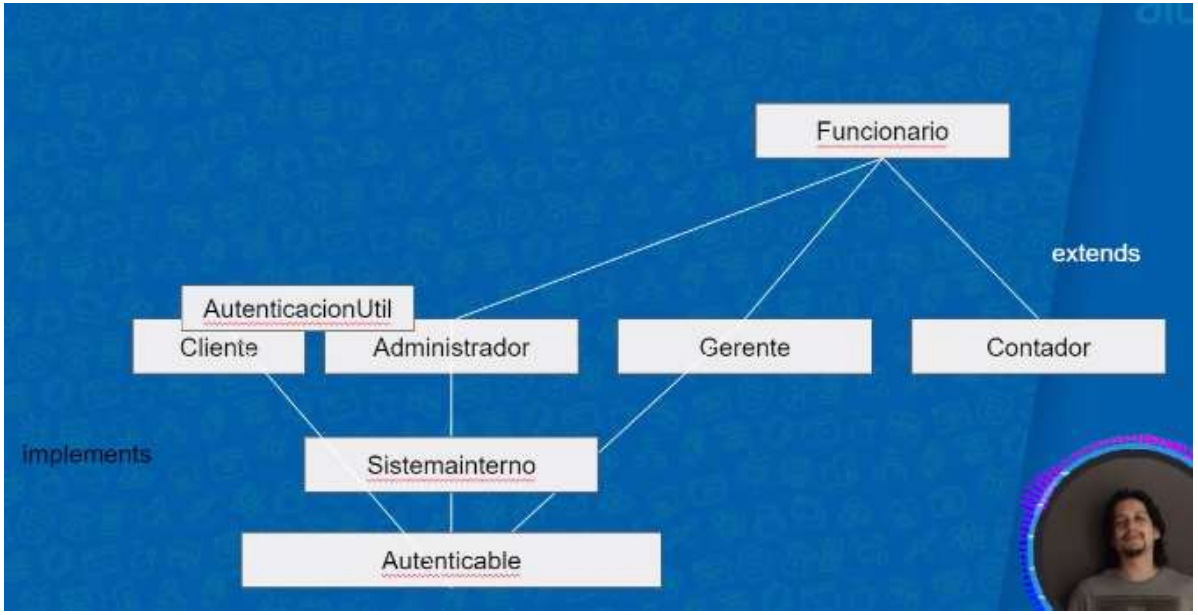
    public AdminSys(String name, int id, double salary, int tipo){
        super(name, id, salary, tipo);
    }


    @Override
    public void setPassword(int password) {
        this.util.setPassword(password);
    }

    @Override
    public boolean sesionInit(int password){
        return this.util.initSesion(password);
```



# Bajo acoplamiento



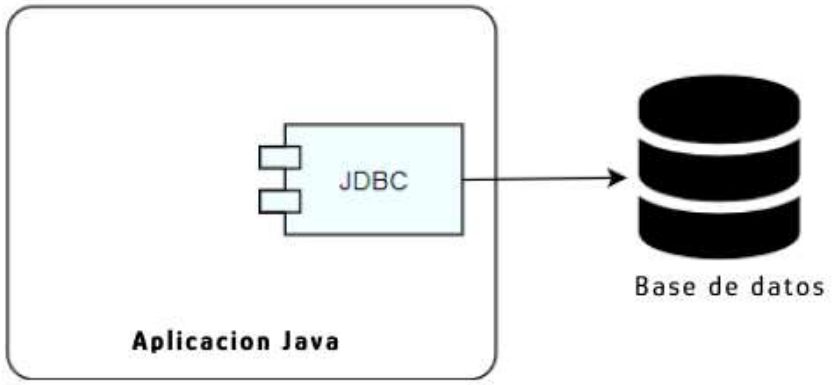



# Crear anotaciones en Java

https://www.aluracursos.com/blog/crear-anotaciones-en-java



# JDBC Java




https://www.aluracursos.com/blog/conociendo-el-jdbc

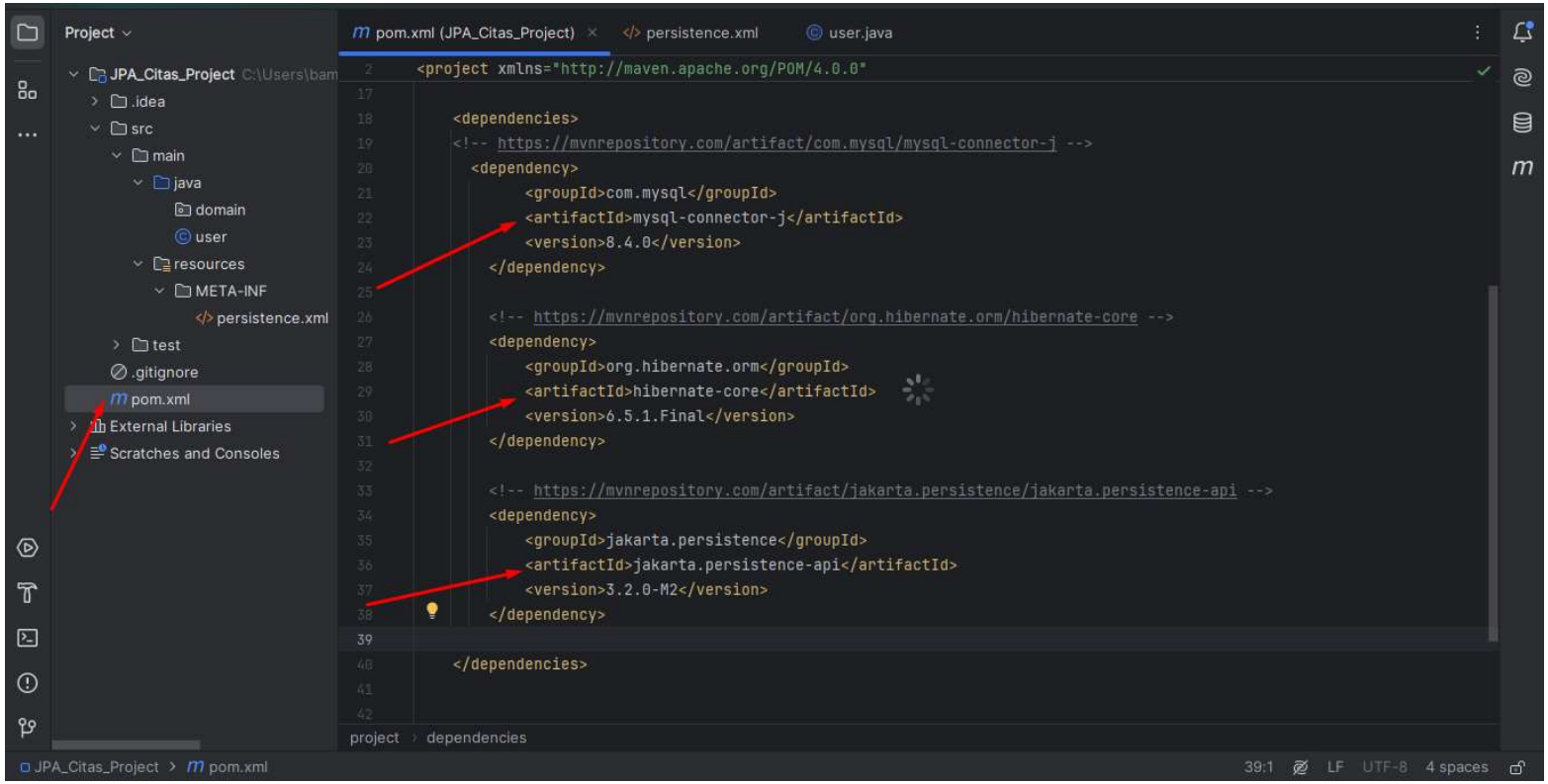


# Diferencias entre JDBC  y JPA - Java

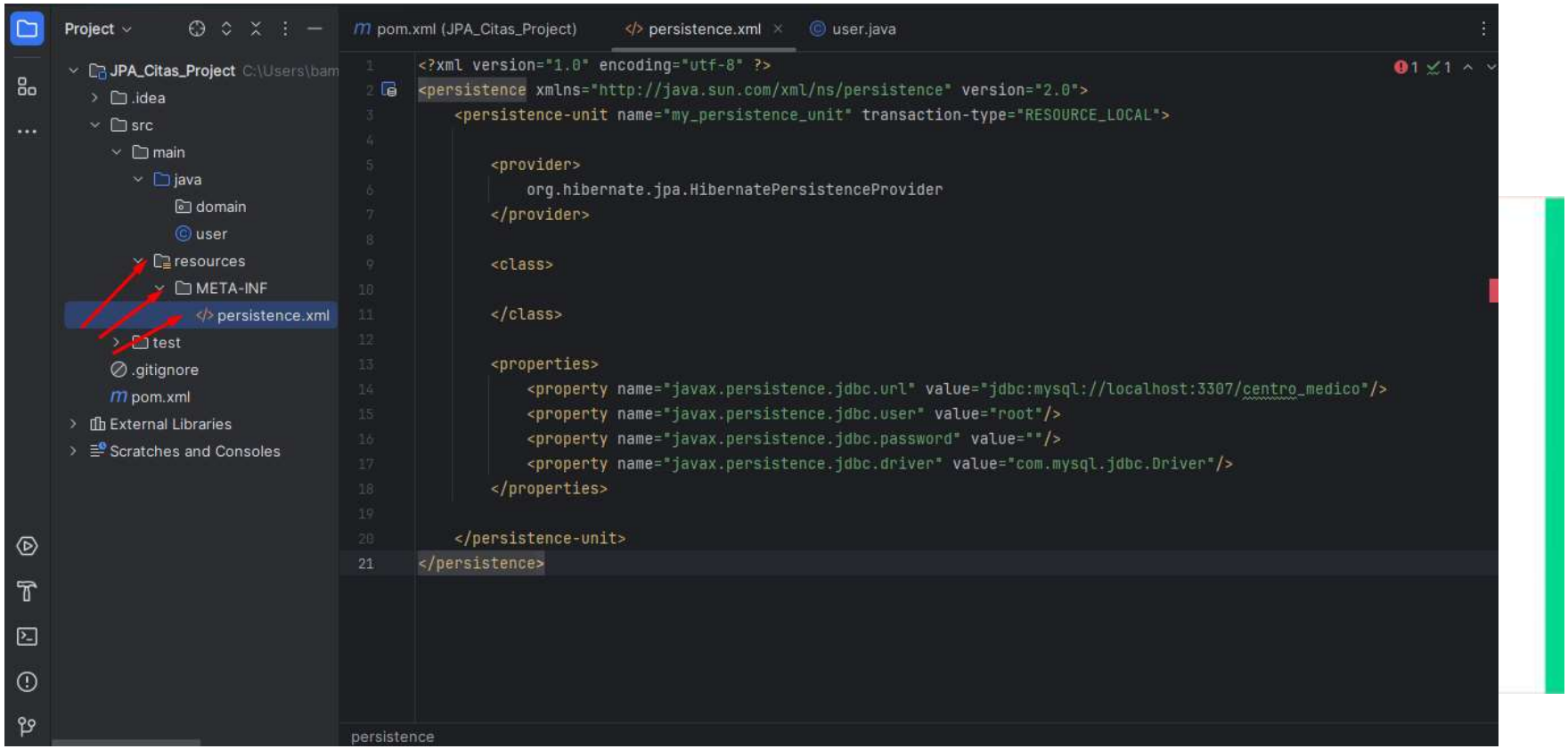
| Aspecto | JDBC | JPA |
| --- | --- | --- |
| Nivel de Abstracción | Bajo nivel, requiere código manual para operaciones de base de datos. | Alto nivel, proporciona abstracción de objetos y mapeo objeto-relacional. |
| Mapeo Objeto-Relacional | No ofrece soporte nativo para mapeo objeto-relacional. Requiere mapeo manual. | Proporciona un mecanismo automático de mapeo objeto-relacional utilizando anotaciones o XML. |
| Productividad y Mantenibilidad | Puede ser tedioso y propenso a errores, requiere escribir más código y cambios en la base de datos pueden requerir modificaciones en el código. | Mejora la productividad al reducir la cantidad de código necesario y facilita el mantenimiento debido al mapeo automático. |
| Portabilidad | Ampliamente compatible con sistemas de gestión de bases de datos (DBMS) debido a ser una API estándar de Java. | Requiere un proveedor de persistencia específico (como Hibernate o EclipseLink), lo que puede introducir algunas diferencias en características y rendimiento. |

# Creando un proyecto con JPA

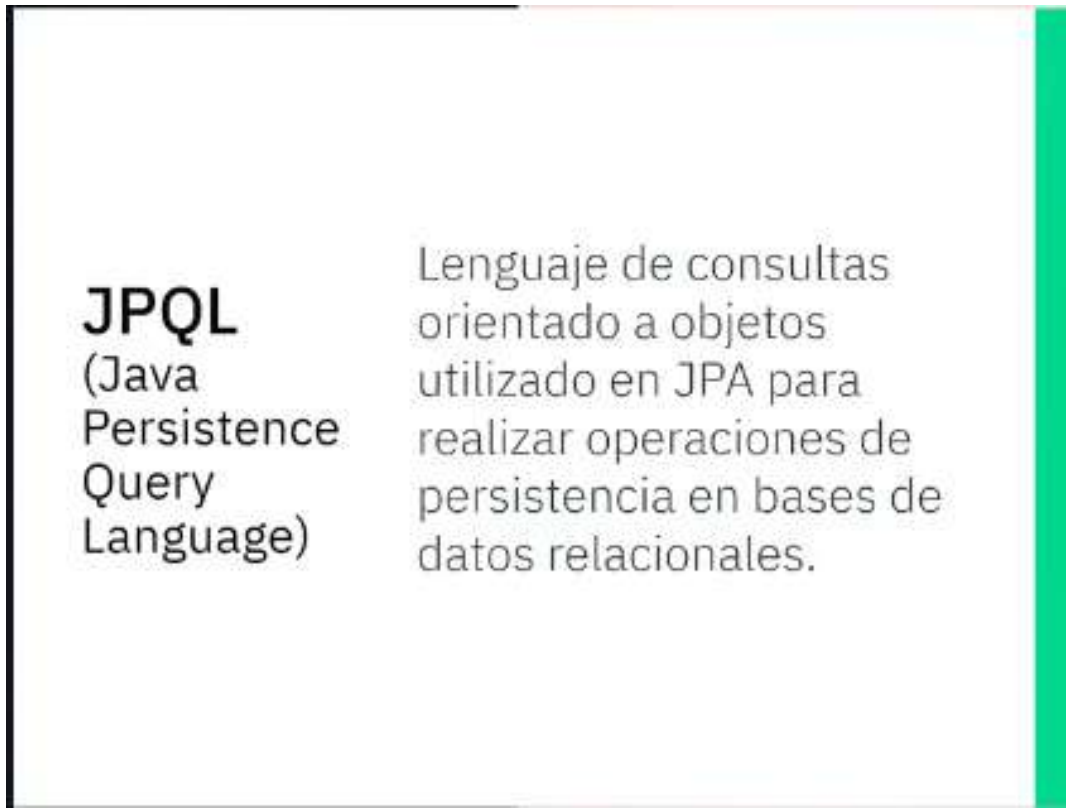
# Implementando JPA e hibernate en las dependencias



# Creando el archivo Persistence.xml



# Creando el archivo Persistence.xml

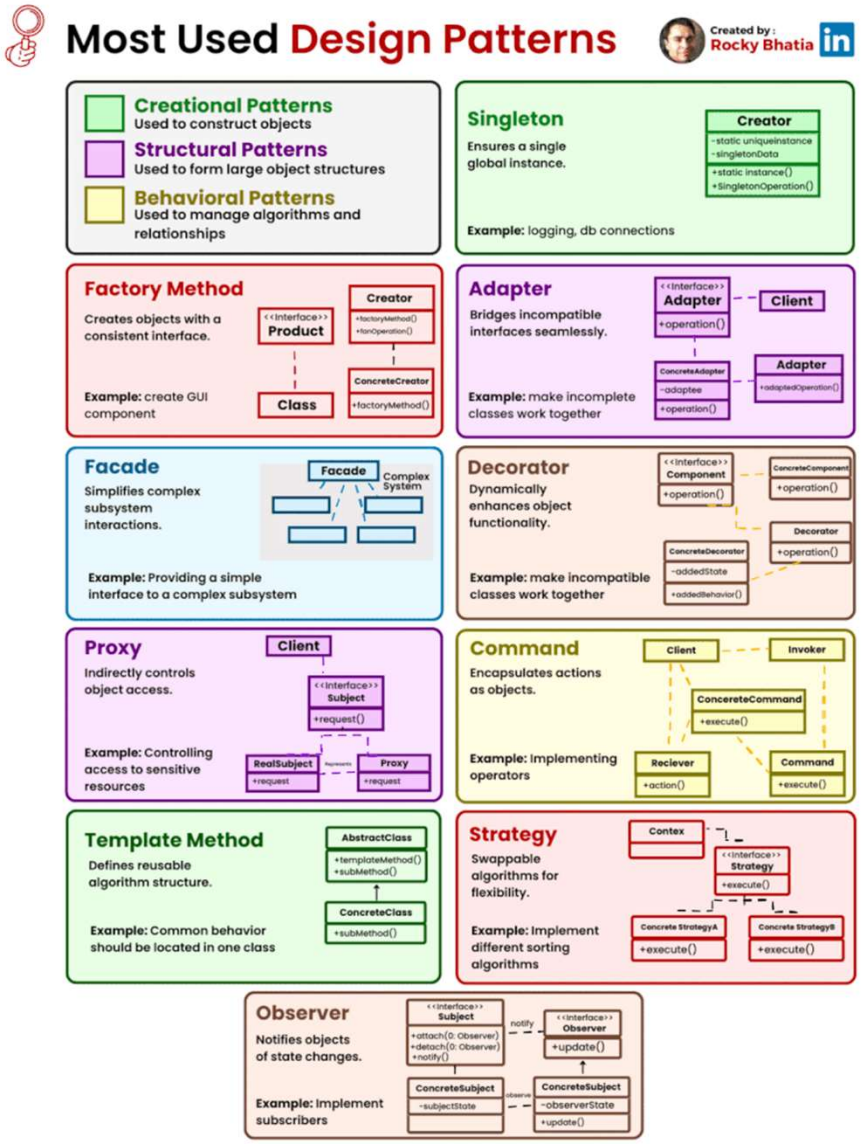


# Creando el archivo Persistence.xml

| EntityManagerFactory | EntityManager |
| --- | --- |
| Fábrica de instancias de EntityManager | Representa una "unidad de trabajo" en la aplicación. |
| Se encarga de crear y configurar los EntityManager | Utilizado para realizar operaciones de persistencia. |
| Configurado a través del archivo de persistencia (persistence.xml) | No requiere configuración adicional, ya que se crea a partir del EntityManagerFactory. |
| Administra las conexiones a la base de datos y Proporciona un método createEntityManager() | Proporciona métodos para realizar operaciones de persistencia, como persist(), merge(), remove() y find(). |

| | |
|---|---|
| **.find( )** | Recupera una entidad por su identificador único |
| **.persist( )** | Guarda una nueva entidad en la base de datos |
| **.merge( )** | Combina los cambios de una entidad gestionada con la base de datos |
| **.remove( )** | Elimina una entidad de la base de datos |
| **.createQuery( )** | Crea una consulta JPQL personalizada |
| **.getTransaction( )** | Obtiene la transacción actual asociada al EntityManager |

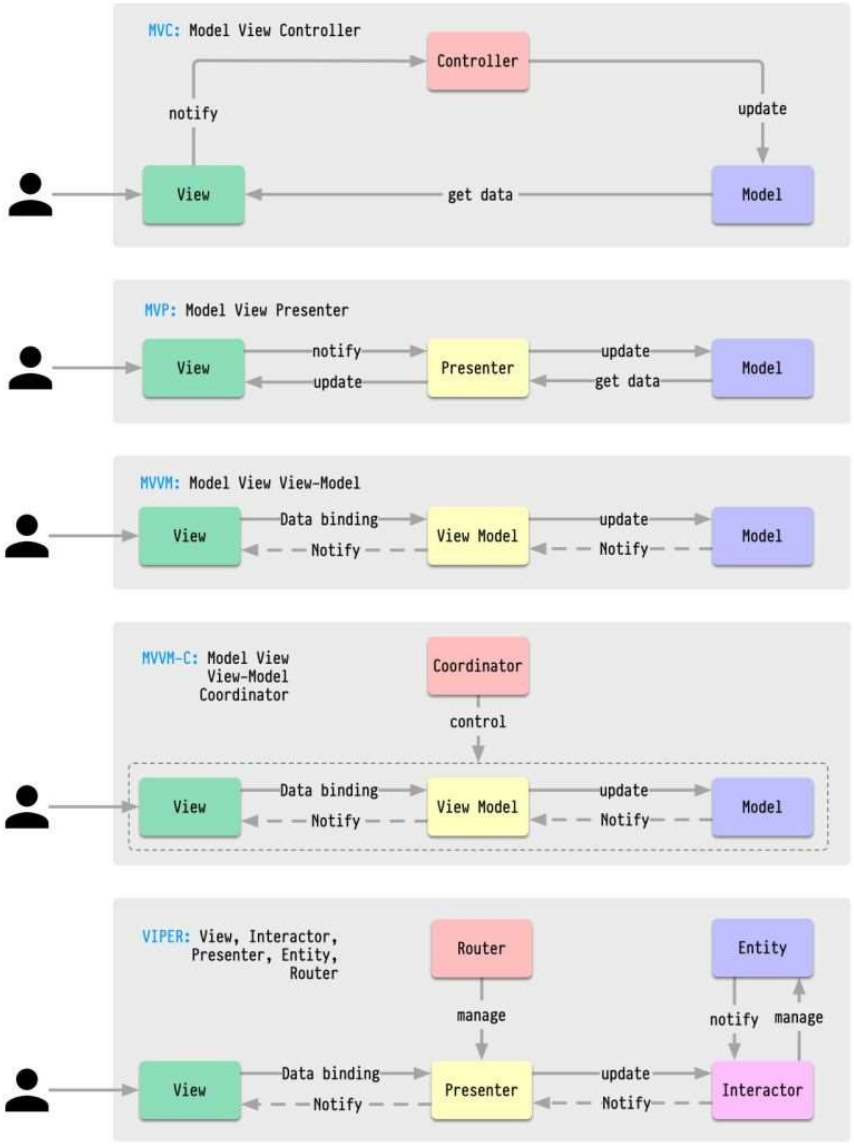
Patrones de diseño en JAVA



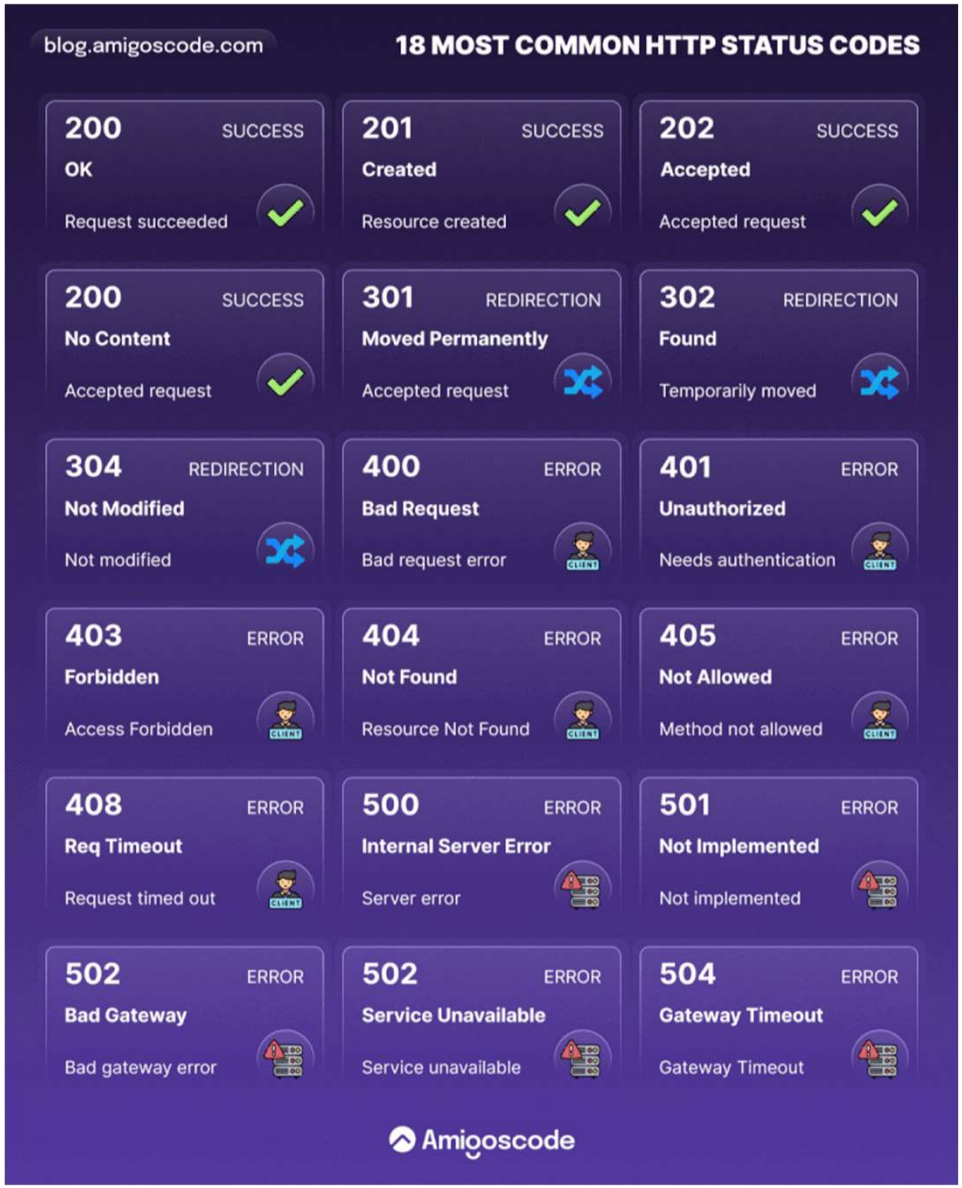



## MVC, MVP, MVVM, VIPER patterns

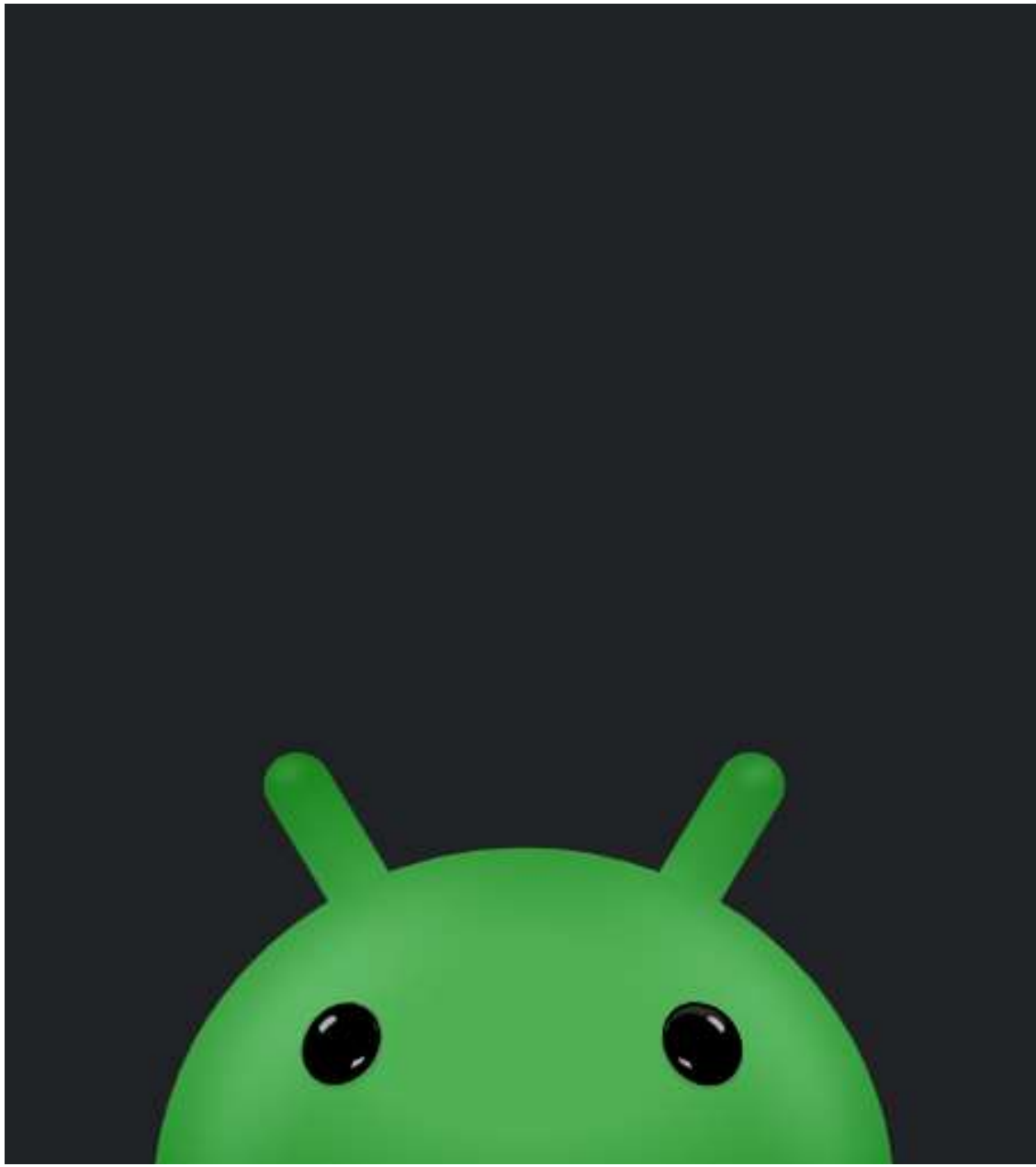
ByteByteGo.com



Respuestas mas comunes de API bajo el protocolo HTTP



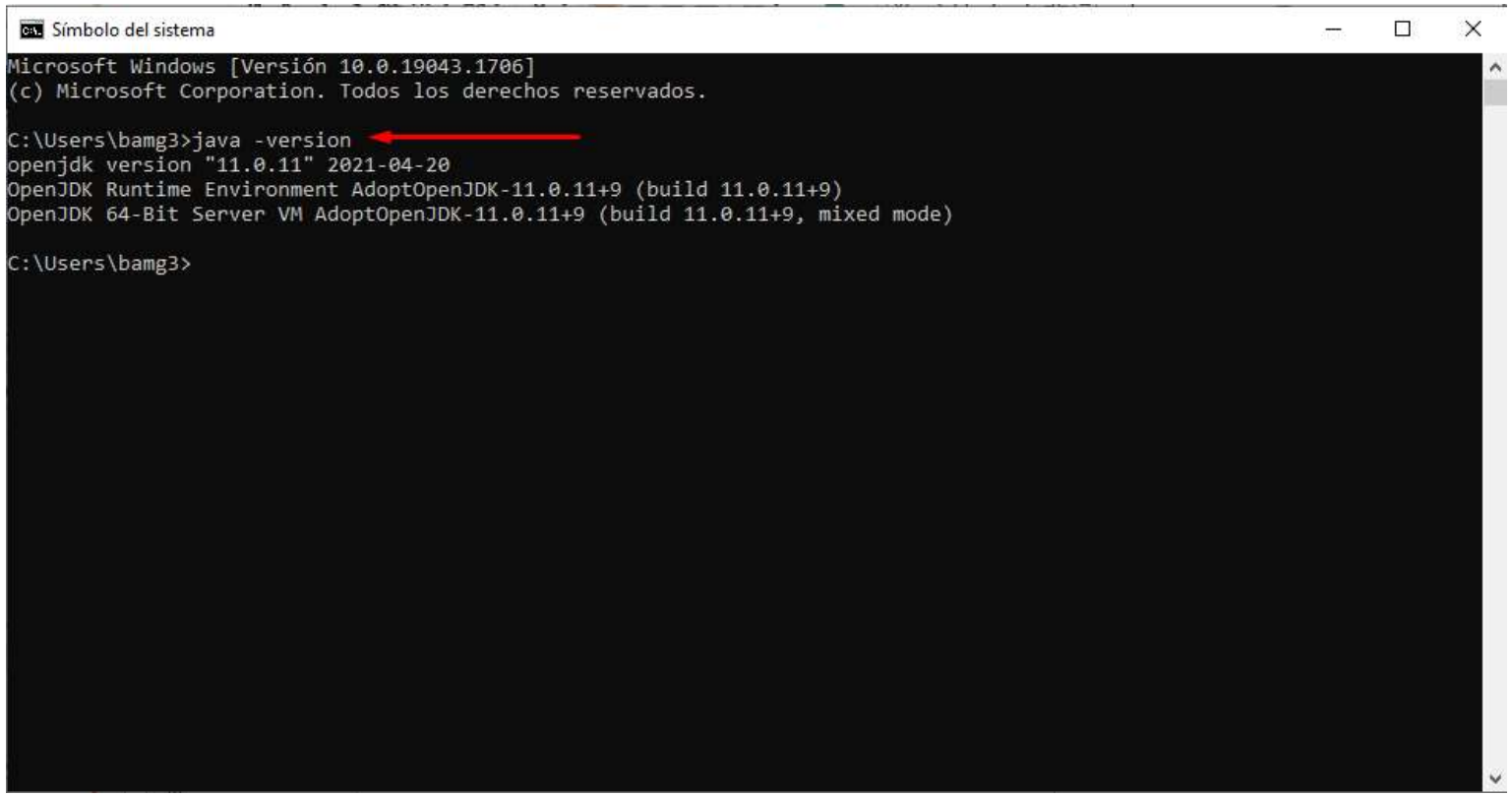
# Desarrollo Android en Android Studio con JAVA



# Instalación de Android Studio

Lo primero es validar si tenemos instalado el JDK de Java en nuestro equipo.

1. Abre la consola y escribe : java –versión
2. Si la versión e menor a 7 debes instalar JDK , si tienes unas superior, tu PC esta listo






# Instalación de Android Studio



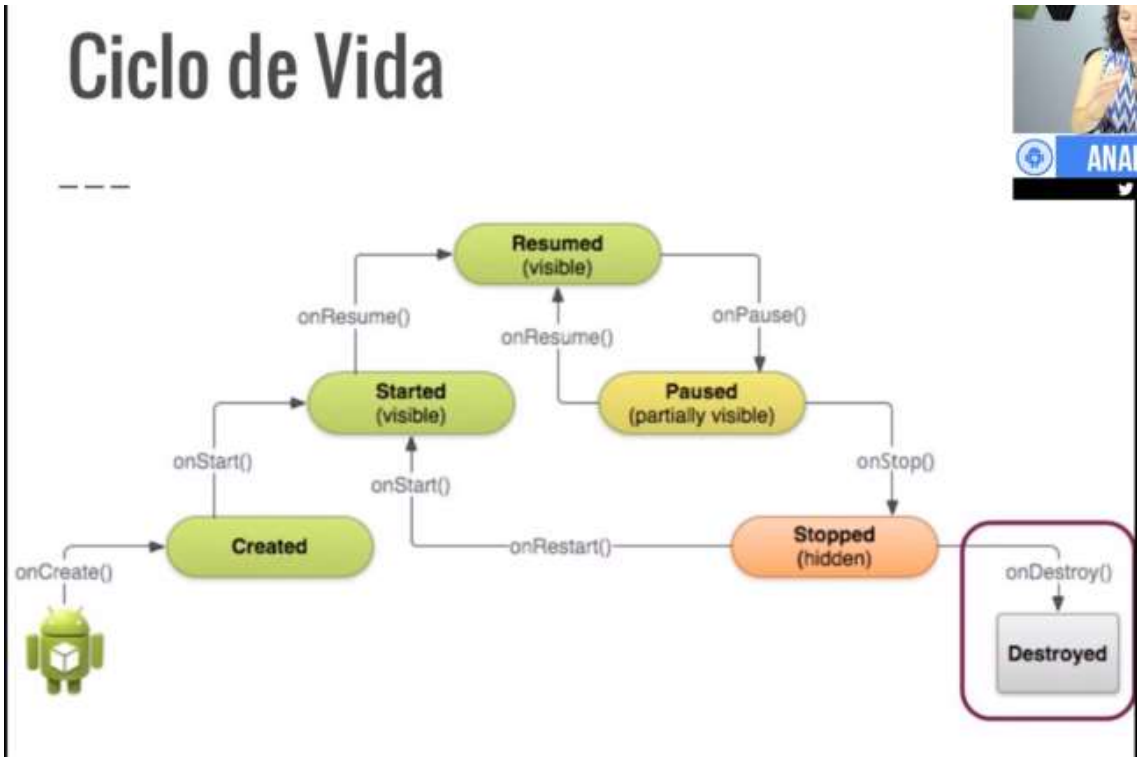
# Créditos

Imagen Vaso:

Image by <a href="https://pixabay.com/users/janbaby-3005373/?utm_source=link-attribution&utm_medium=referral&utm_campaign=image&utm_content=2680204">Jan Alexander</a> from <a href="https://pixabay.com//?utm_source=link-attribution&utm_medium=referral&utm_campaign=image&utm_content=2680204">Pixabay</a>



# Ciclo de Vida



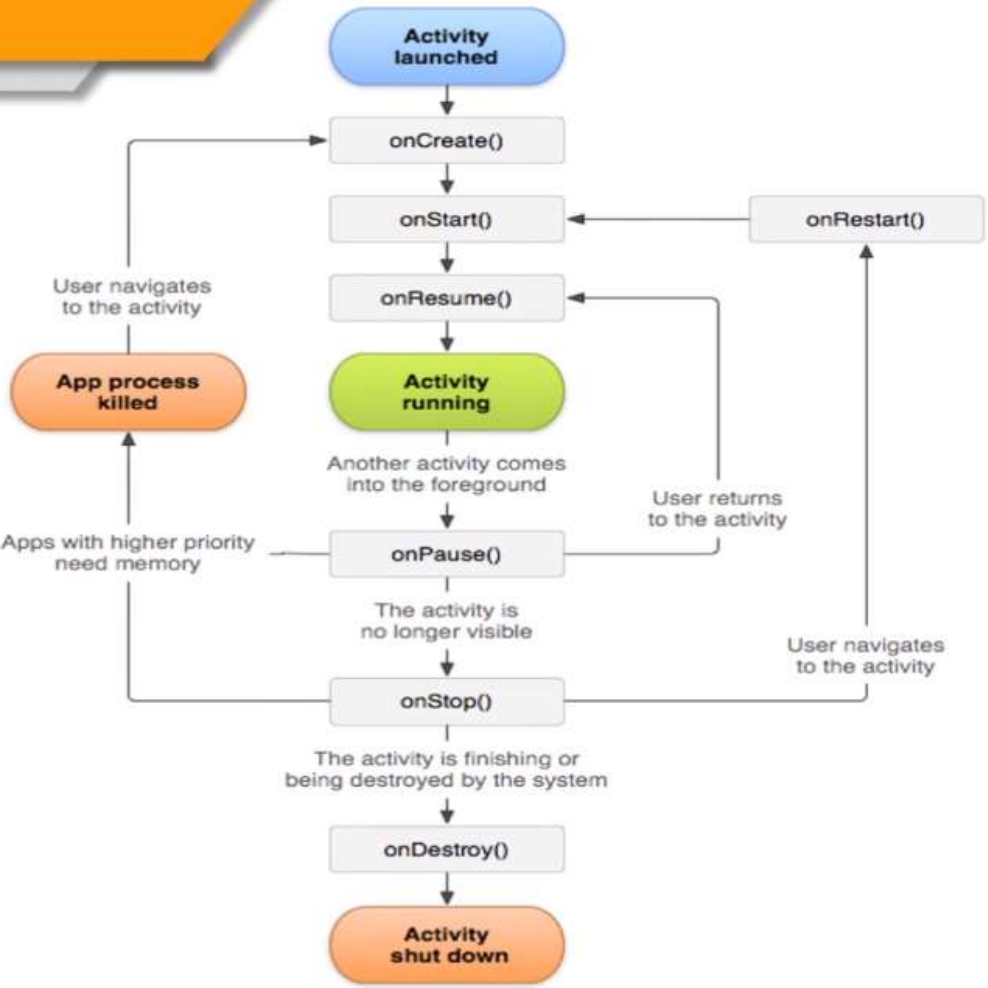



```java
public class ExampleActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // The activity is being created.
    }
    @Override
    protected void onStart() {
        super.onStart();
        // The activity is about to become visible.
    }
    @Override
    protected void onResume() {
        super.onResume();
        // The activity has become visible (it is now "resumed").
    }
    @Override
    protected void onPause() {
        super.onPause();
        // Another activity is taking focus (this activity is about to be "paused").
    }
    @Override
    protected void onStop() {
        super.onStop();
        // The activity is no longer visible (it is now "stopped")
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        // The activity is about to be destroyed.
    }
```





# Application Lifecycle






Un activity es una pantalla de una aplicación.
Un activity se compone de una clase de Kotlin/Java y un archivo layout .xml
Un activity tiene un ciclo de vida.
El primer paso en el ciclo de vida es el onCreate el cual tiene el estado de la actividad. Aquí va todo lo que queremos que se ejecute cuando se crea la actividad.
Luego pasamos al estado onStart. Es parte de la inicilización de la actividad. Aquí la actividad es visible en el dispositivo.
Después pasamos al estado onResume. Sucede cuando nuestra actividad ya termino de hacer el render de sus elementos. Aquí la actividad es visible y su estado es que la aplicación está corriendo.
También existe el estado onPause. Es cuando la aplicación está en pausa. Aquí la actividad es parcialmente visible. Puede que otra actividad esté corriendo, por lo que la anterior tomaría el estado de pausa.
El estado onStopped. Es cuando la actividad está escondida. Cuando otra actividad está tomando prioridad y está encima de.
El estado onDestroy. ****Sucede cuando se presiona el botón de back, la aplicación se cerraría o se destruye. Para recuperar el estado, antes tendría que repetir todo el proceso, es de cir pasar por el onCreate. Si la actividad estaba escondida, pasará por el método onRestart.



Un buen articulo de lo que son los fragments

🤘 http://www.sgoliver.net/blog/fragments-en-android/

Los fragments son contenedores que muestran el contenido de una porción de interfaz. Viven en la activity , por lo tanto dependen del ciclo de vida de la actividad.



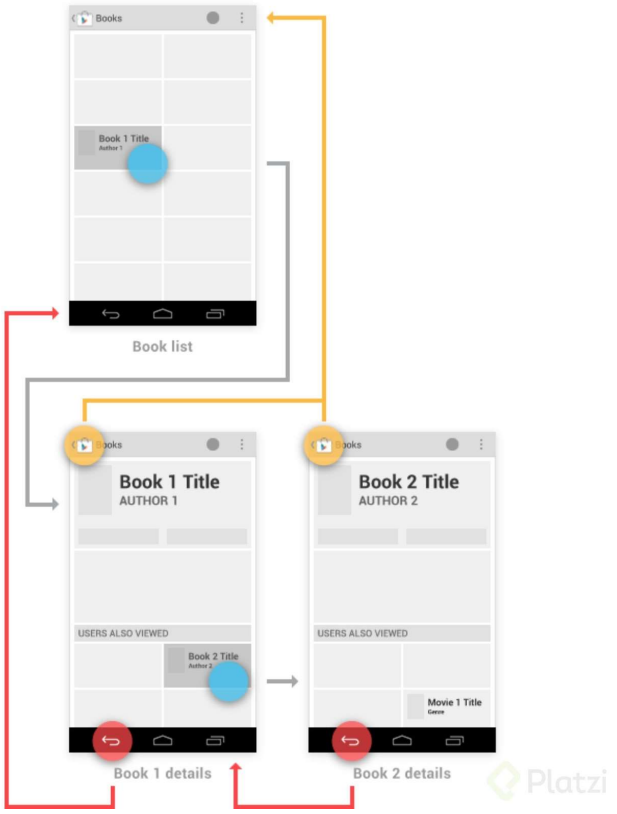



Los Intents nos servirán para unir componentes de una aplicación, y podemos tener dos casos específiamente:

El que ya mencionamos, unir Activity's dentro de la aplicación

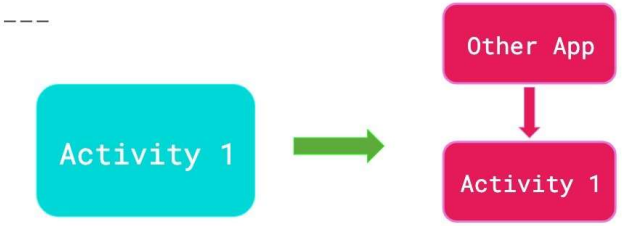



Explícitos





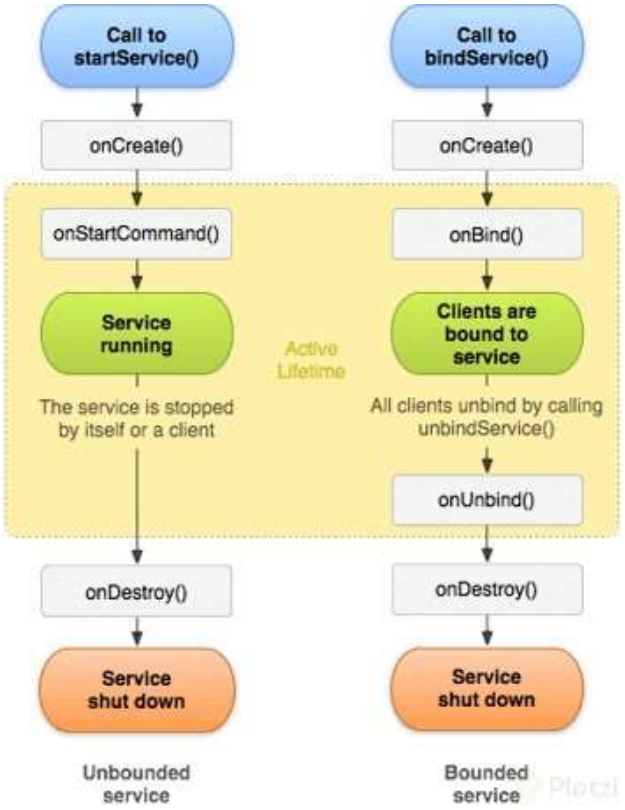





Implícitos

Services:

Ejecutan parte de una aplicación en Background.  Los servicios se ejecutan en las actividades , teniendo un servicio que la detone y la mantenga en funcionamiento: ejemplo , escuchar la radio. O escuchar música , etc.




Broadcast reciever: Son receptores de transmisión y están atentos a lo que sucede en la aplicación, como por ejemplo mensajes, batería baja , llamadas entrantes , nivel de wifi, etc.



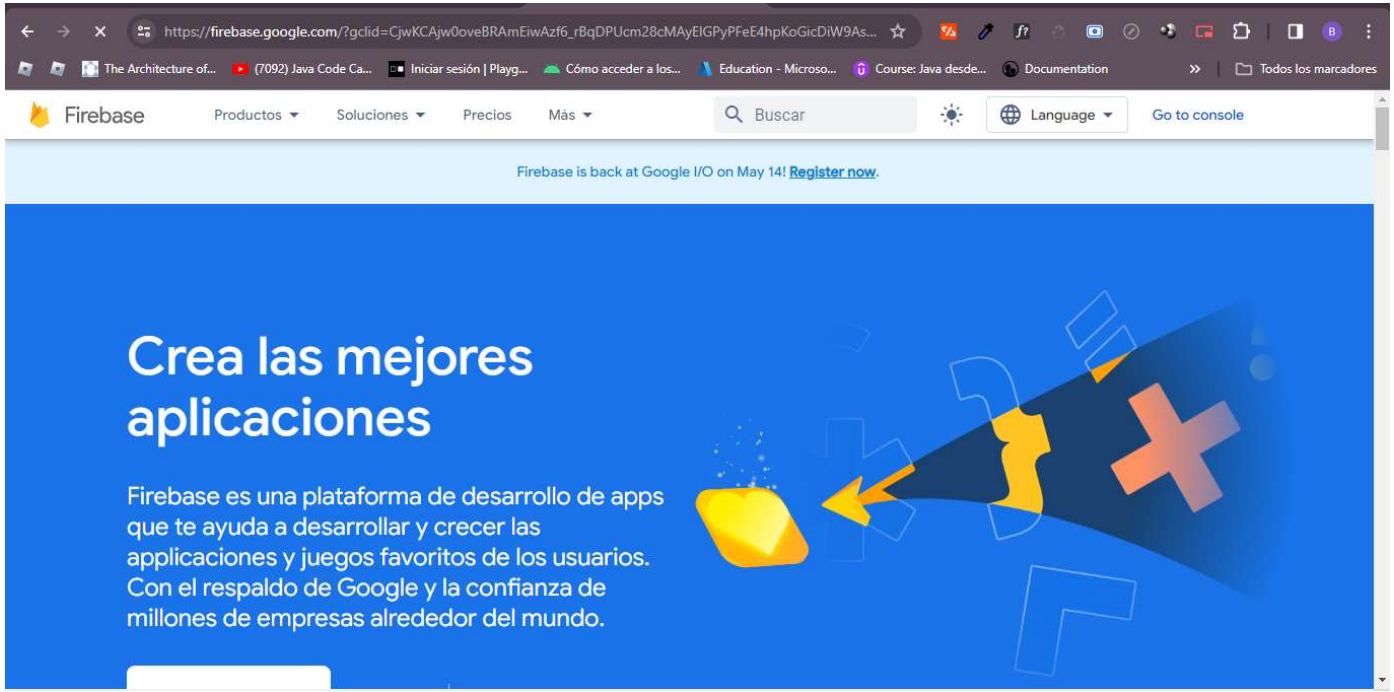
# Que es Firebase

FireBase es una base de datos NoSQL , que funciona en la nube y tiene integrado funcionalidades que potencian las aplicaciones mobiles al integrar en um mismo servicio base de dato en tiempo real, base de datos para datos no relacionales , autenticación y métricas.



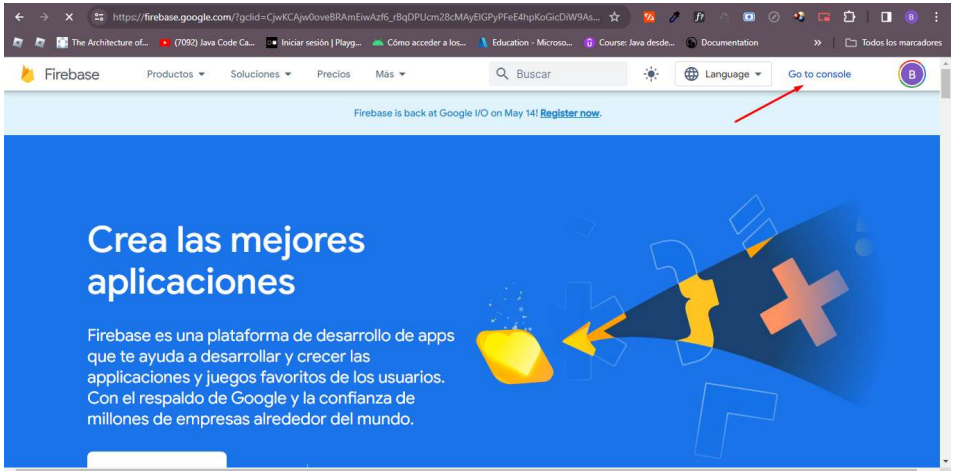
# Conectar Android Studio a Firebase

1. Proceda a crear su cuenta de FireBase usando una cuenta de google

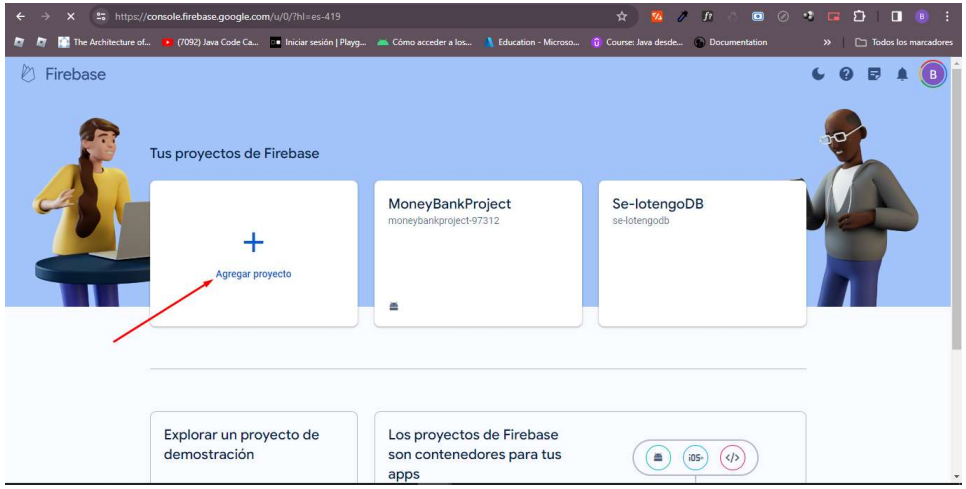


# Conectar Android Studio a Firebase

Una vez logeado vaya a la consola

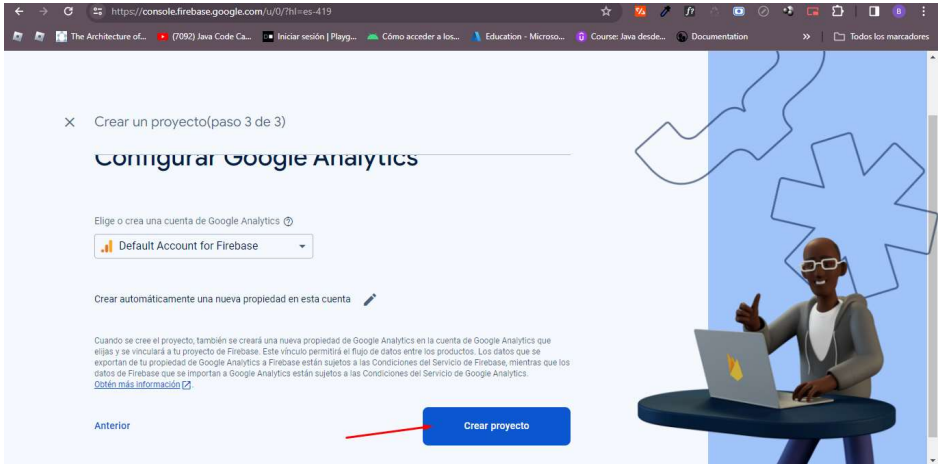



Dentro de la consola agregue un nuevo proyecto



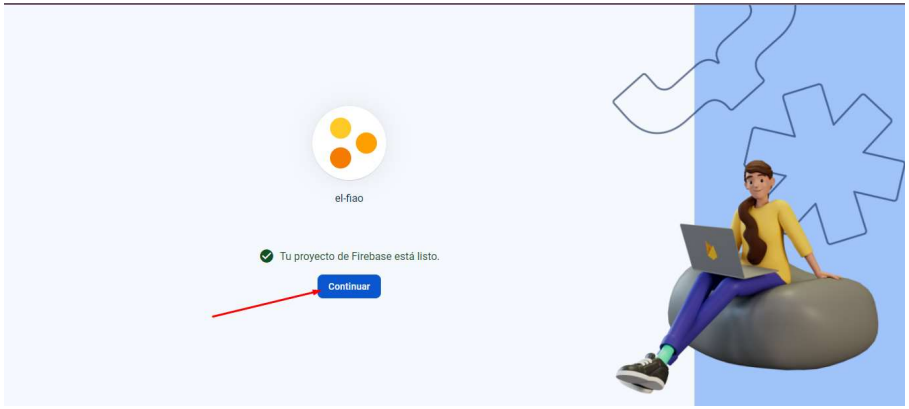
# Conectar Android Studio a Firebase



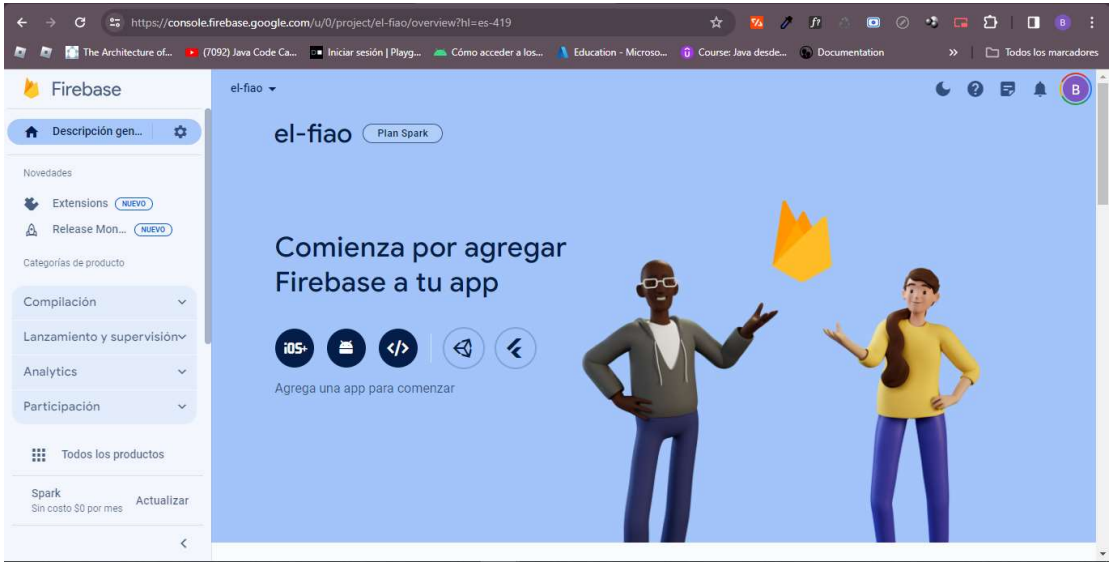






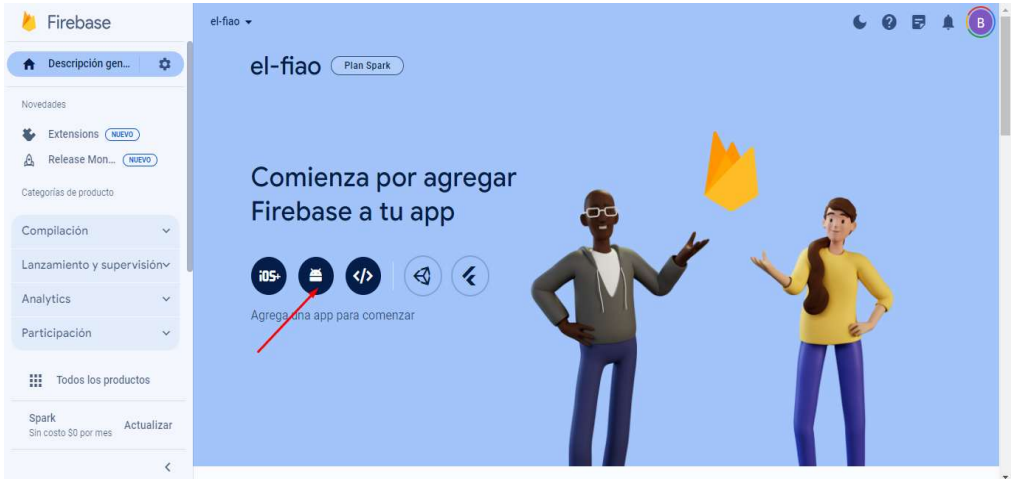
# Conectar Android Studio a Firebase

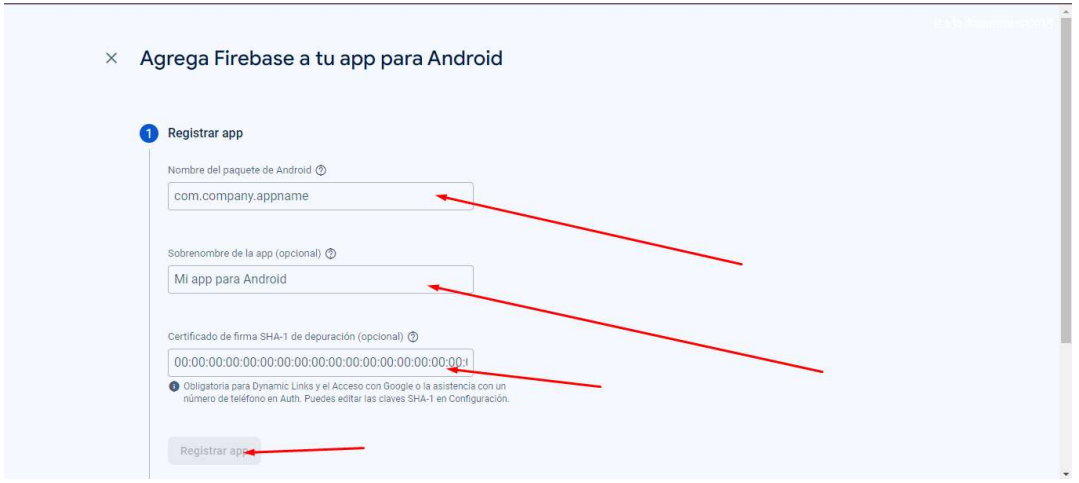
Ya tienes tu base de datos creada, es hora de integrarla con nuestro proyecto en Android Studio



# Conectar Android Studio a Firebase

Ya tienes tu base de datos creada, es hora de integrarla con nuestro proyecto en Android Studio

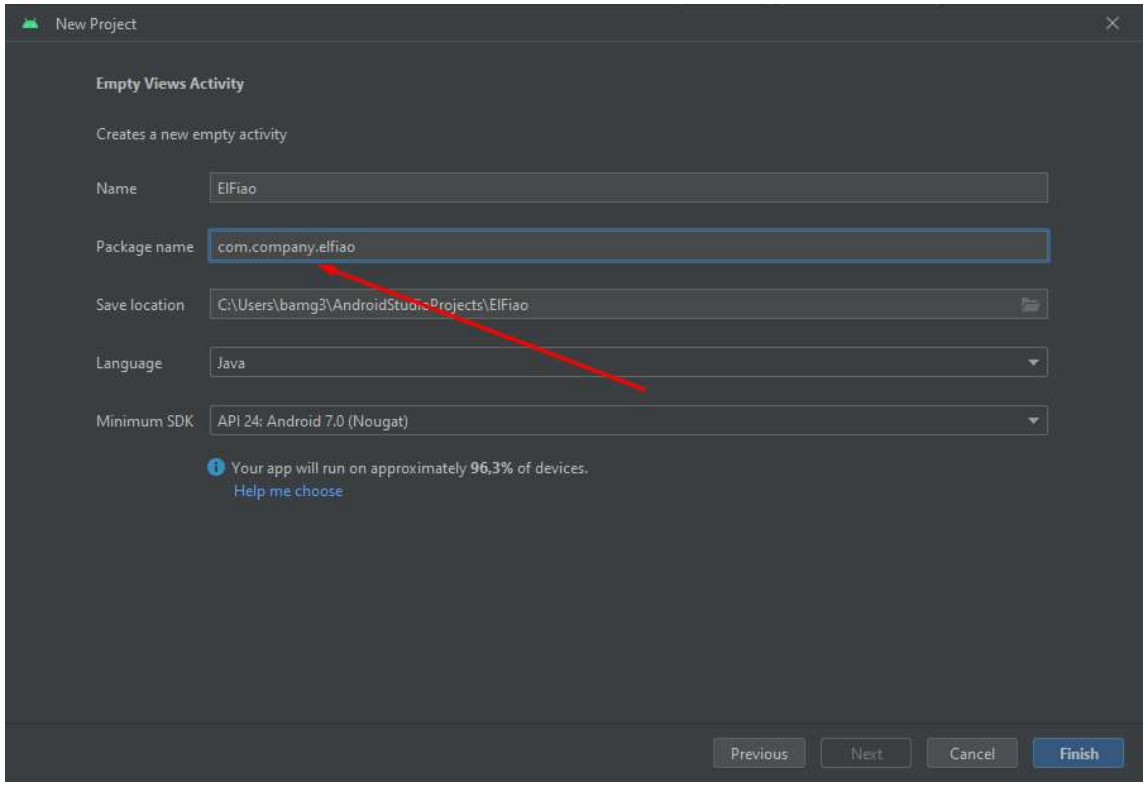


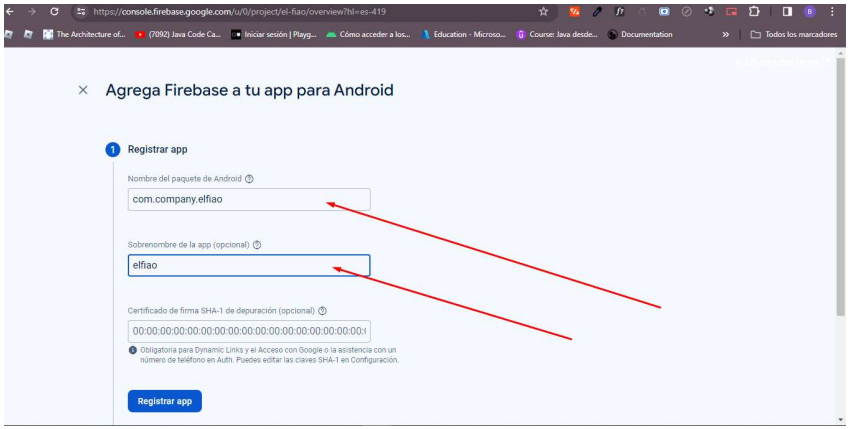




Da click en el icono de Android para proseguir a la configuración de la base de datos, para obtener estos datos procederemos de la siguiente manera:

# Conectar Android Studio a Firebase

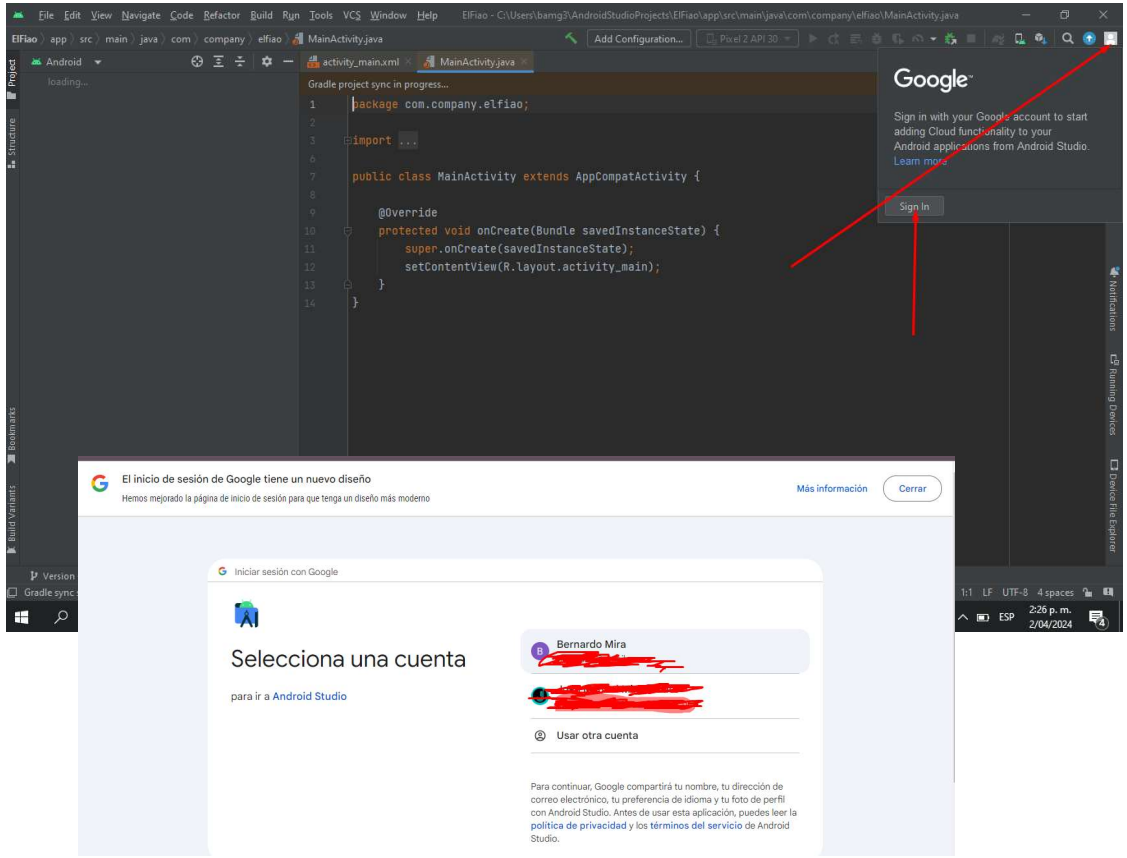
El nombre del paquete que te pide firebase, es el nombre del paquete que le diste a tu app

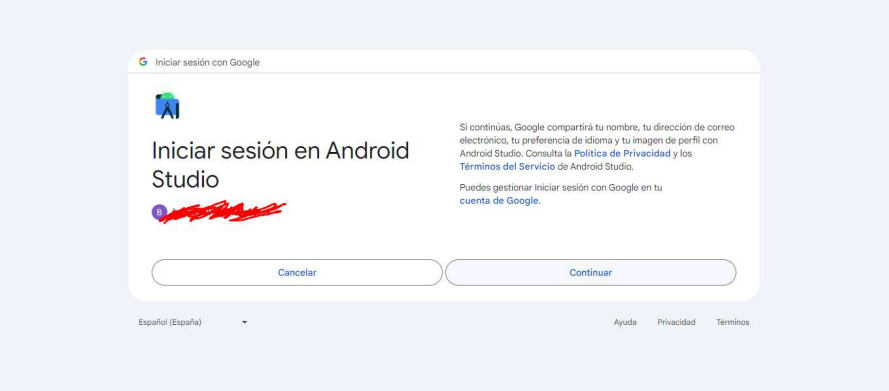


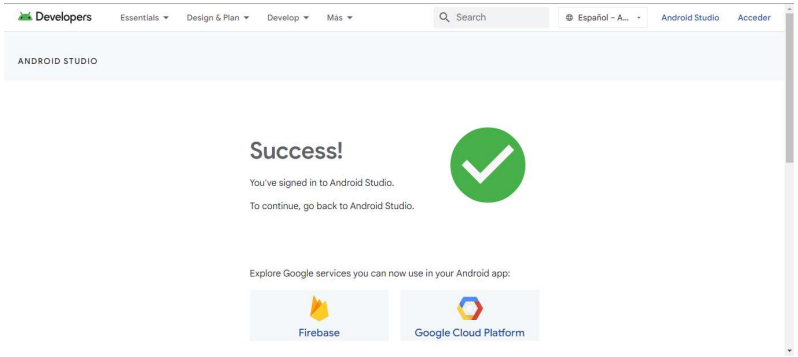



# Conectar Android Studio a Firebase

Ahora logea tu Android Studio a tu cuenta de google



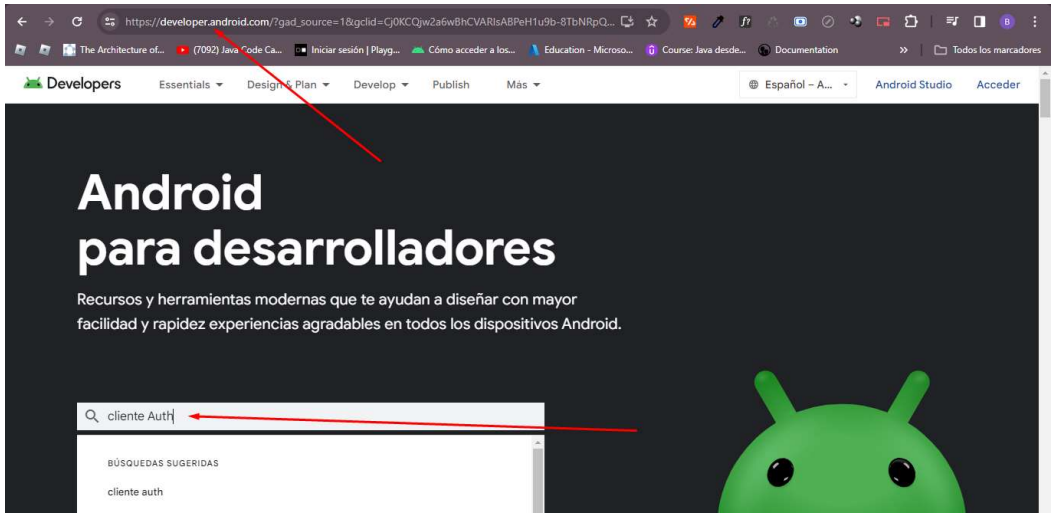



# Conectar Android Studio a Firebase

Una vez que tienes tu Android studio conectado a Firebase, vamos a conectar nuestro proyecto a la base de datos en especifico:

- Para esto vamos a usa la clave SHA-1 que nos pide la configuración de la base de datos:

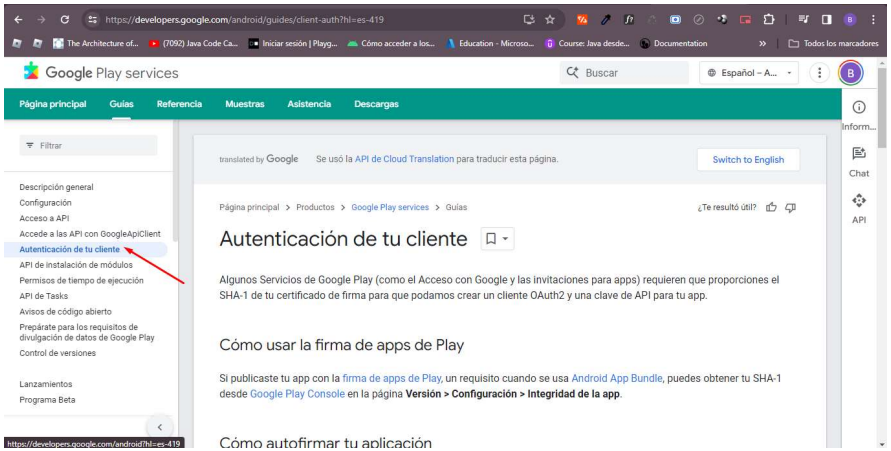
- Esta clave la vas a obtener de la siguiente manera:

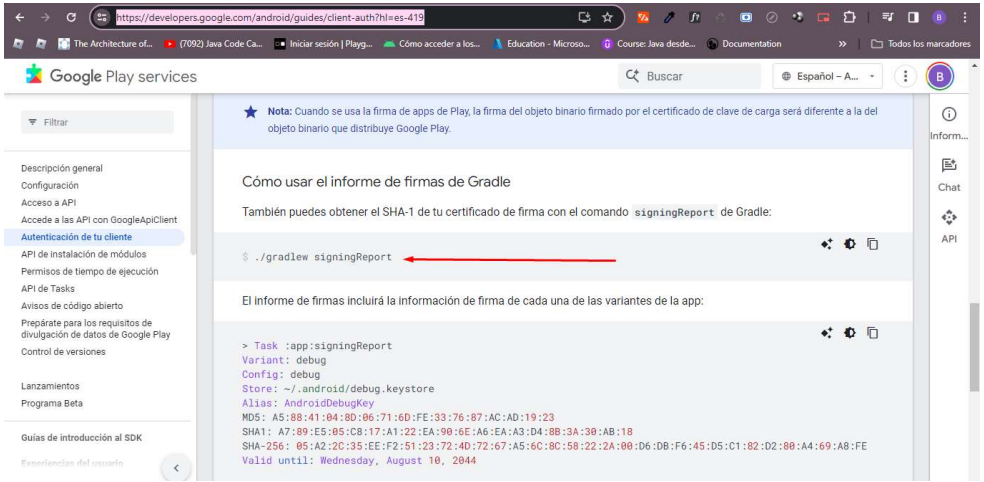



https://developer.android.com/?gad_source=1&gclid=Cj0KCQjw2a6wBhCVARIsABPeH1u9b-8TbNRpQn90eAJoERUyFuFx-Wm9vIwtboEz5I0PezihPqYIxcMaAmIgEALw_wcB&gclsrc=aw.ds&hl=es-419

# Conectar Android Studio a Firebase

Busca  client authentication



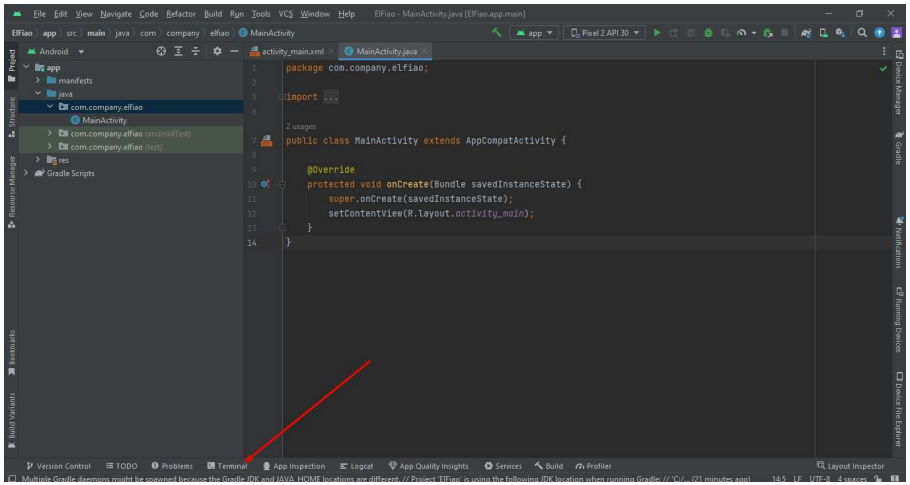



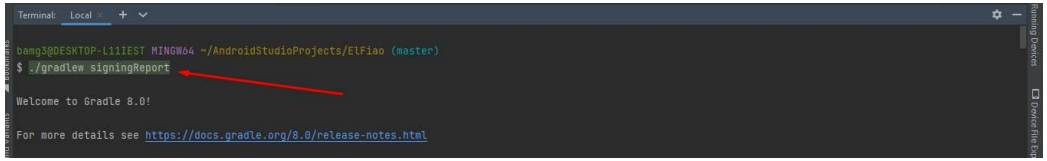

Copia este comando

https://developers.google.com/android/guides/client-auth?hl=es-419

# Conectar Android Studio a Firebase

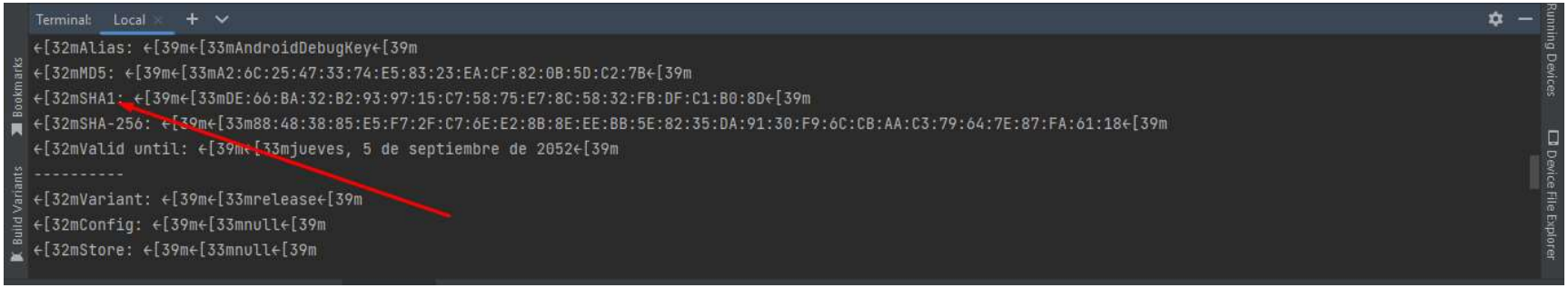
Abre la terminal de tu Android studio



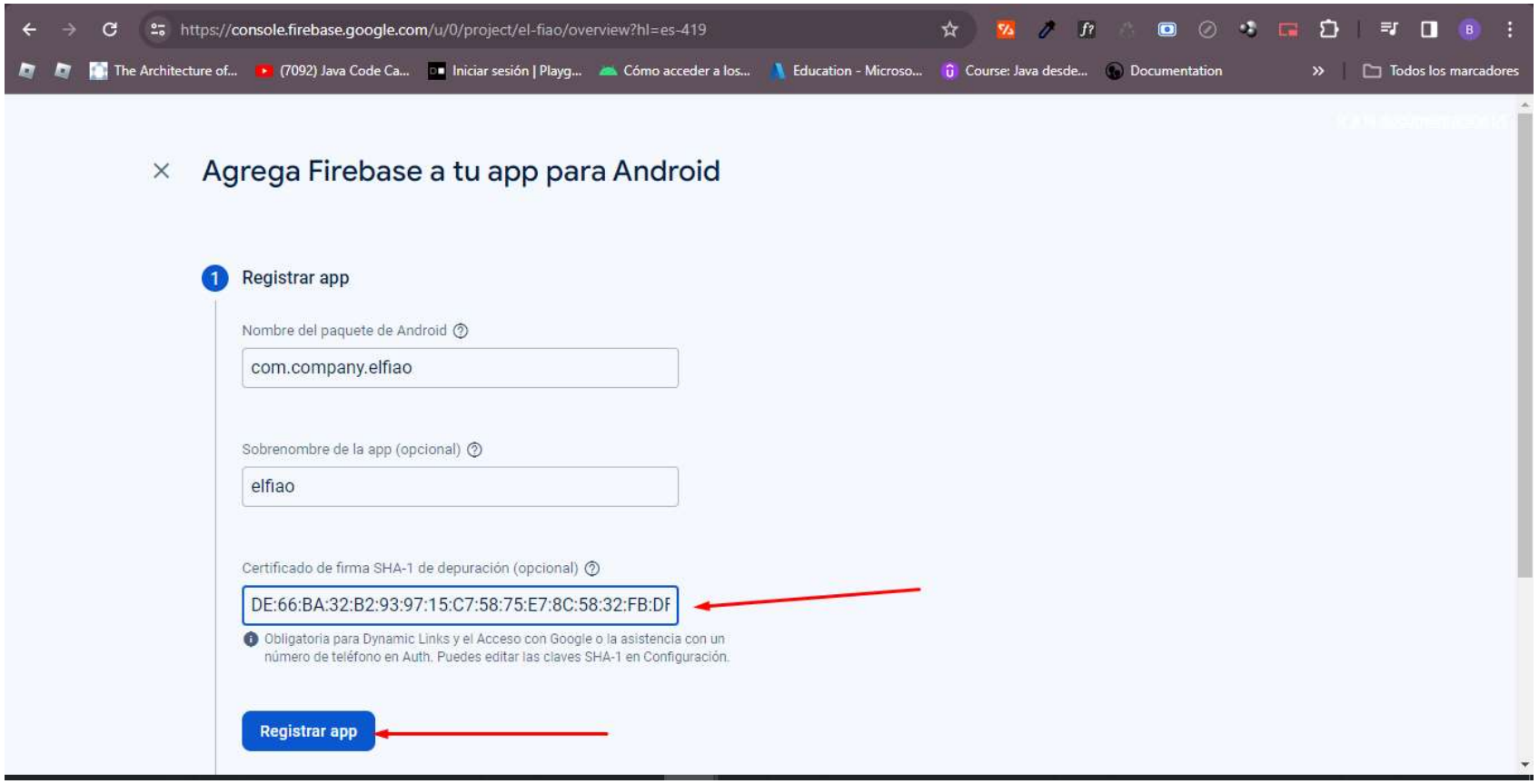

Pega el  comando

Ahora ejecuta y obtén la clave SHA-1



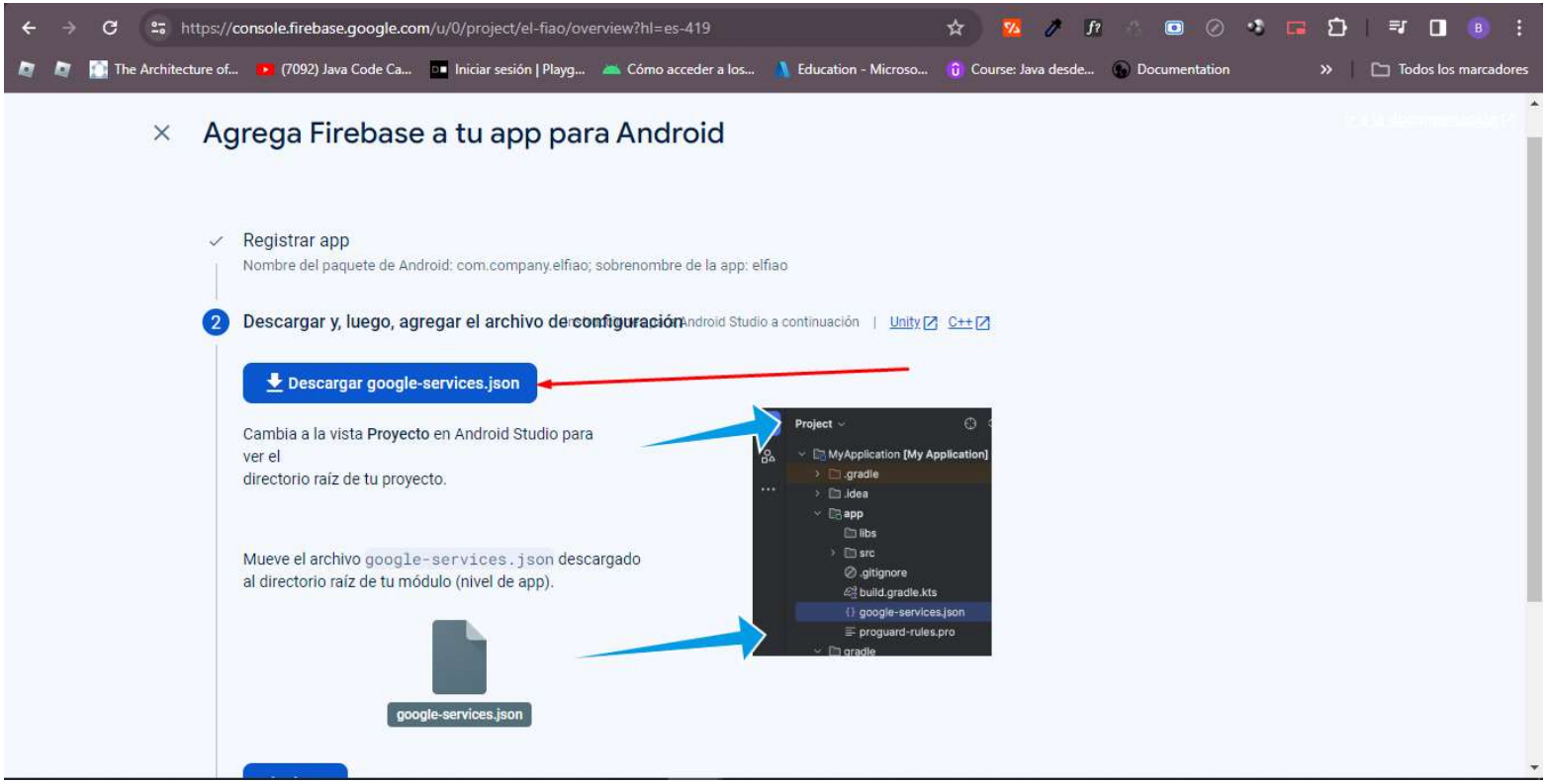
# Conectar Android Studio a Firebase

Copia la clave y pegala en la configuración de firebase y registra y app



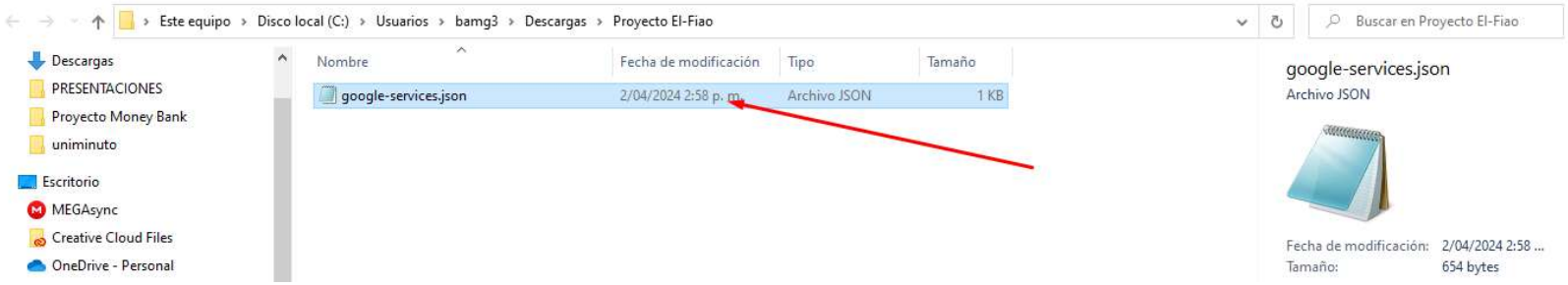
# Conectar Android Studio a Firebase

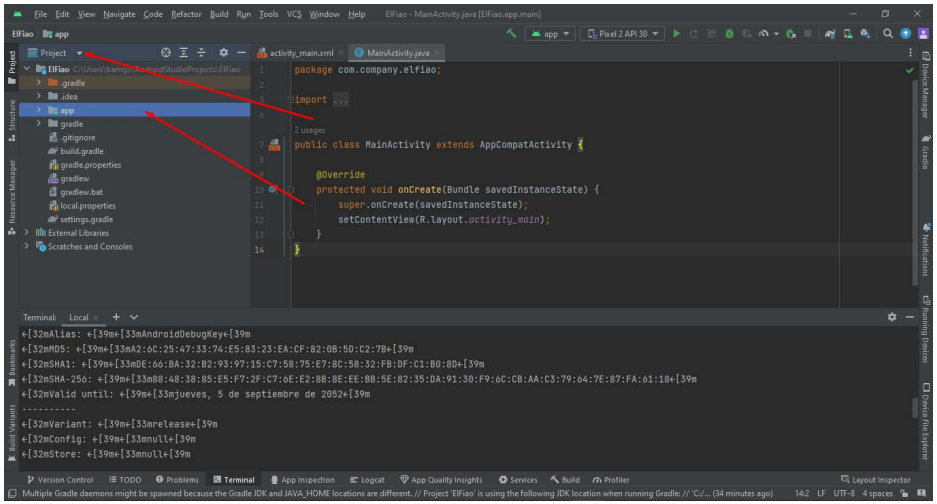
Ahora vamos a descargar el archivo jSON con la configuración del servicio y lo vamos a pegar en nuestro proyecto:

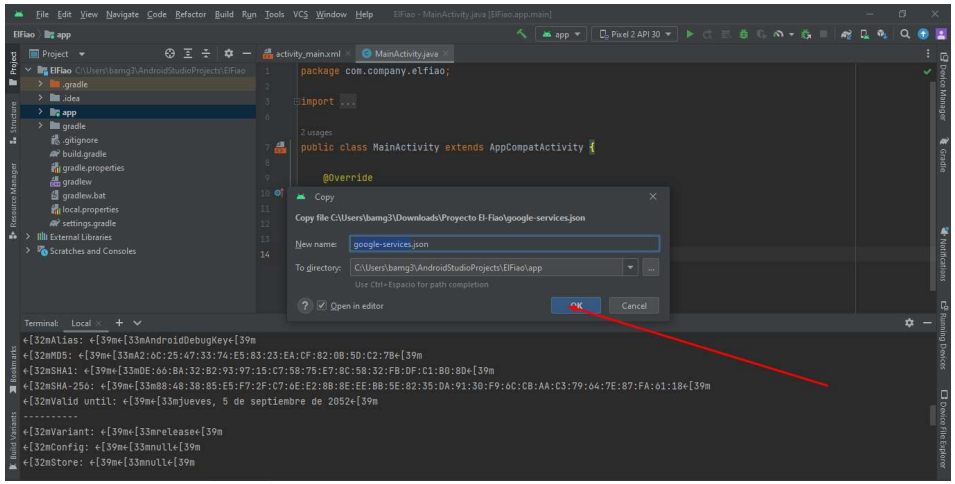


# Conectar Android Studio a Firebase

Copia el json y pégalo en la Android Studio:

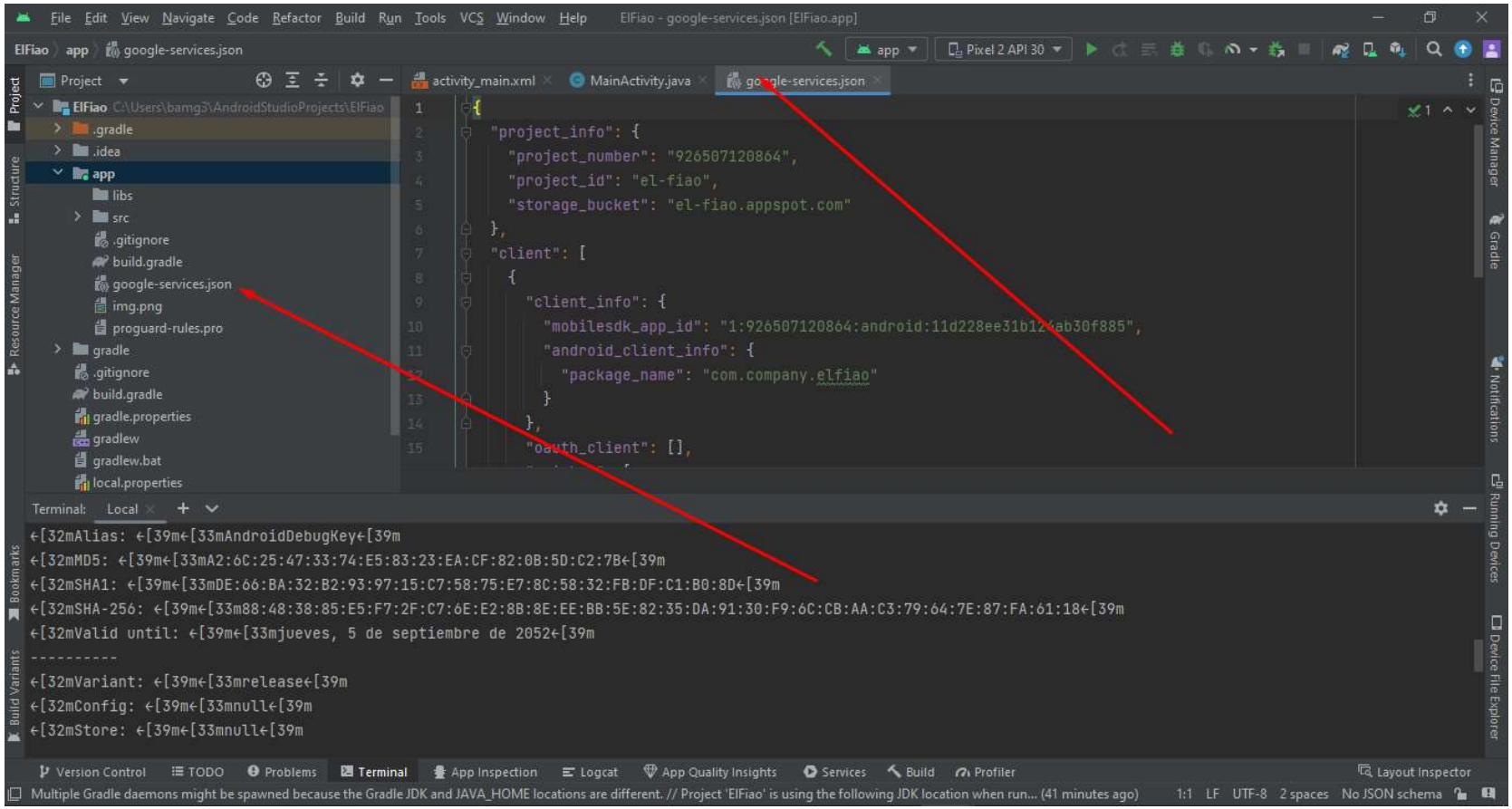


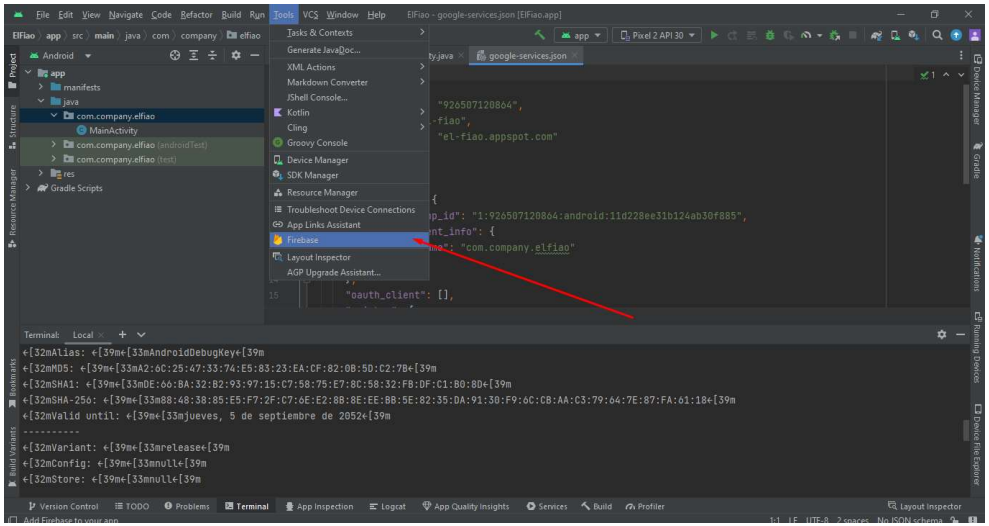
# Conectar Android Studio a Firebase

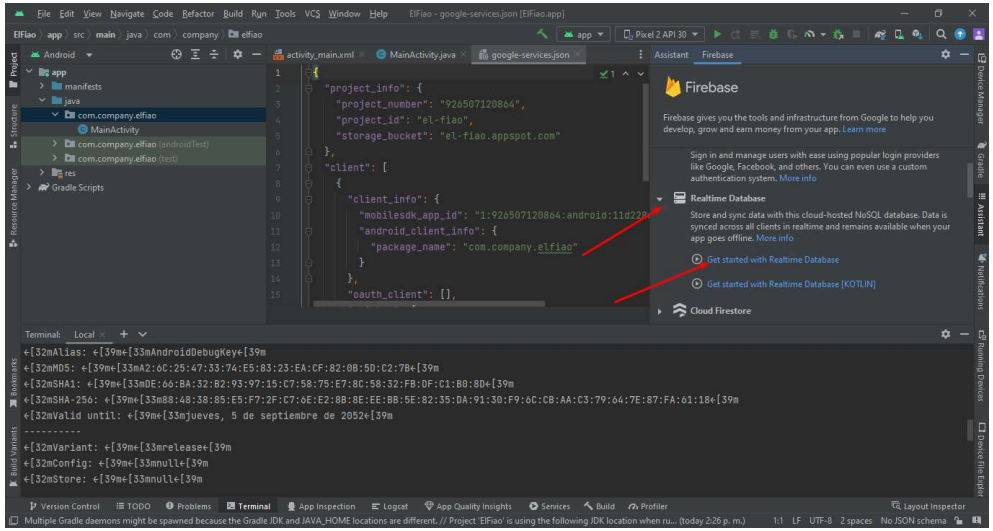
Deberías poder visualizarlo en la carpeta app del proyecto:



# Conectar Android Studio a Firebase

Ahora vamos a tolos y buscamos firebase:

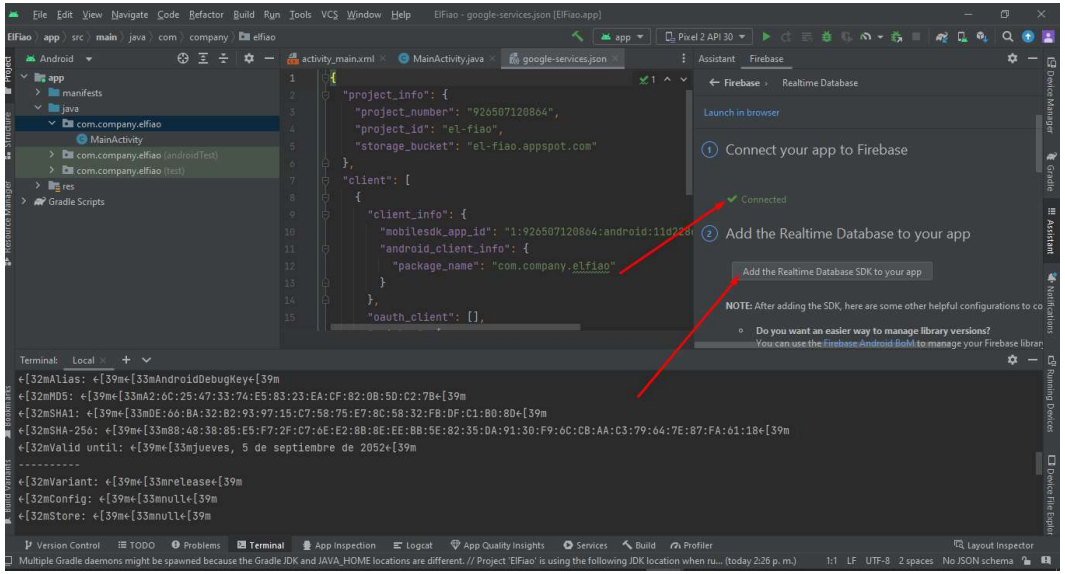


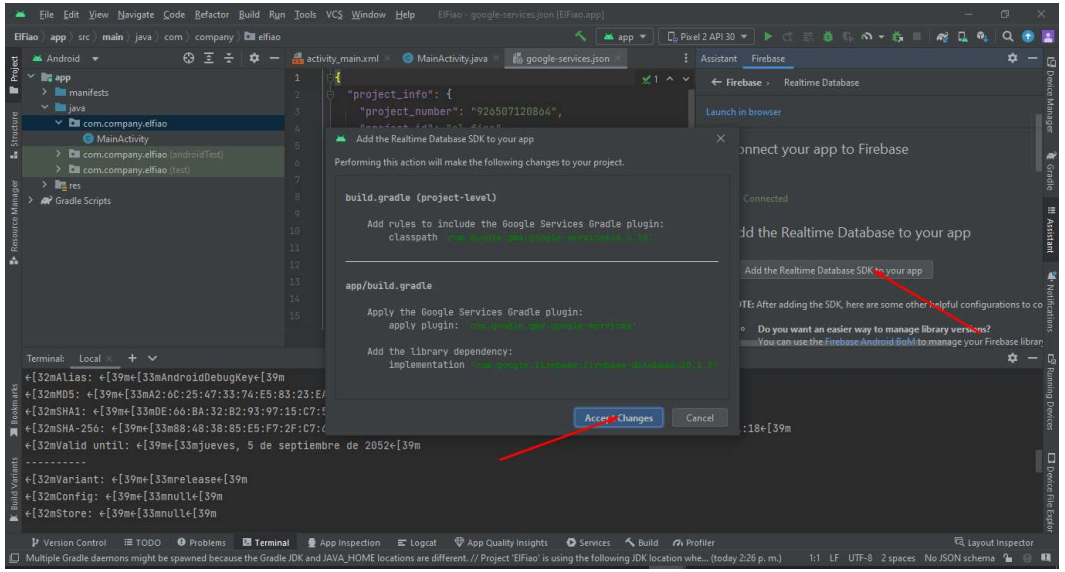




Buscamos realtime database e inicializamos el servicio

# Conectar Android Studio a Firebase

Debe aparecer conectada la base de datos , ahora vamos a añadir la base de datos a la app:

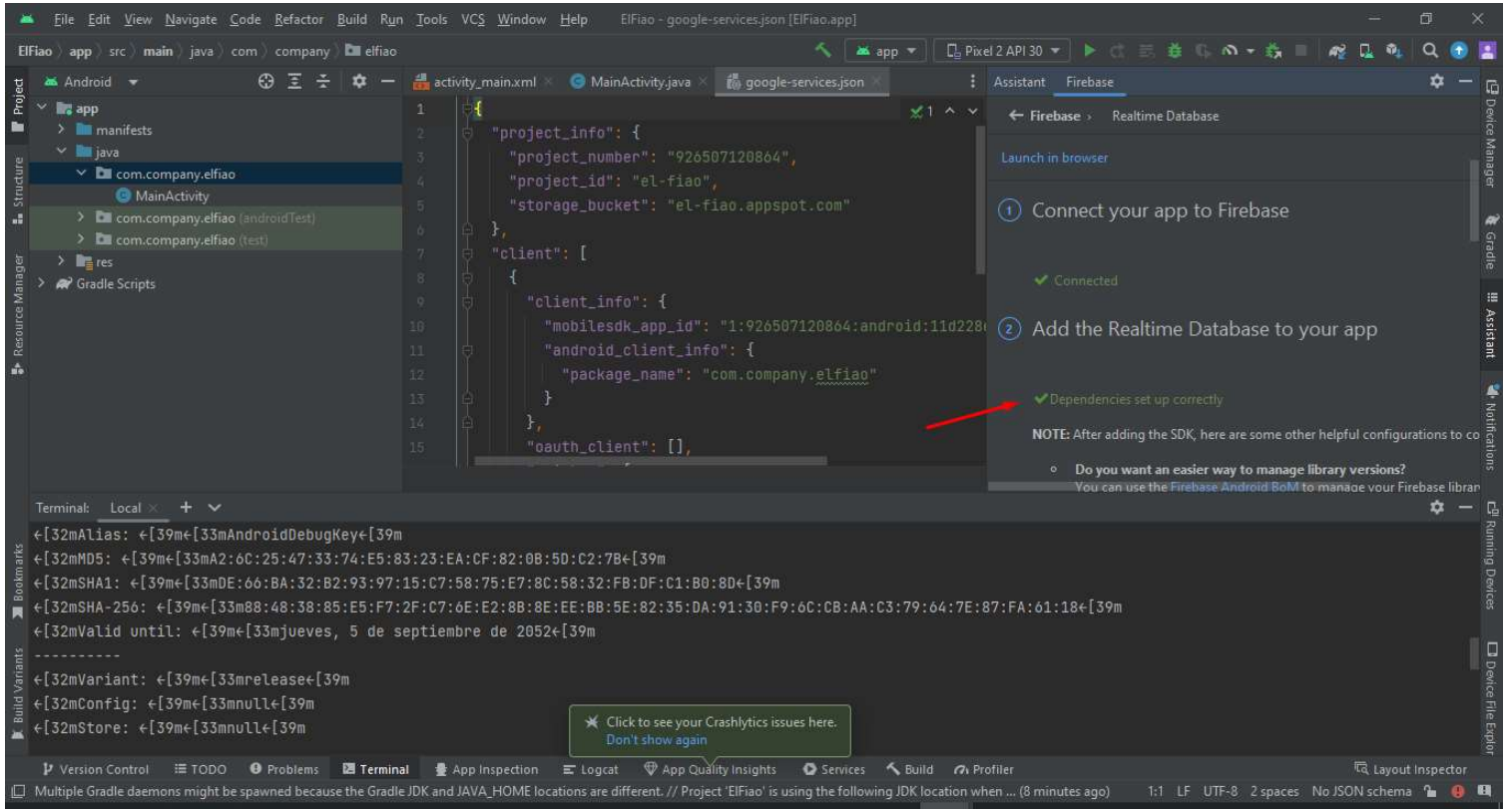



Acepta los cambios

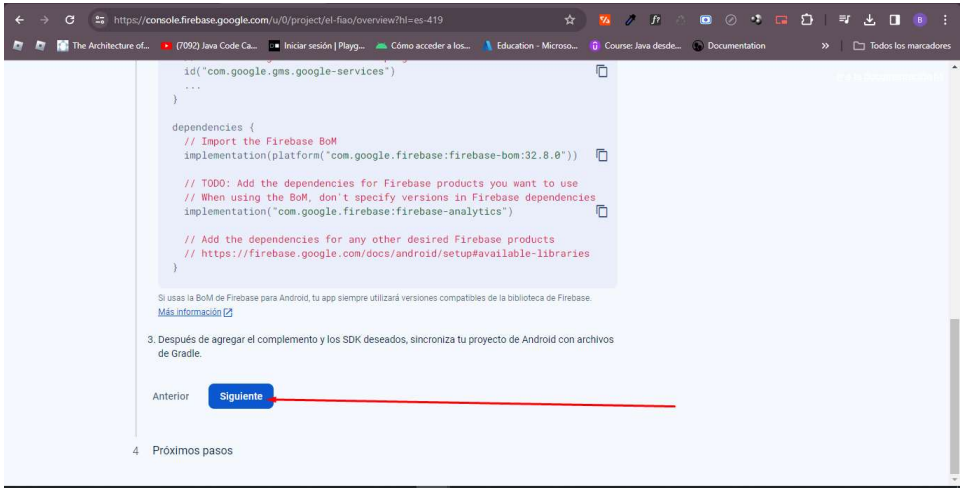
# Conectar Android Studio a Firebase

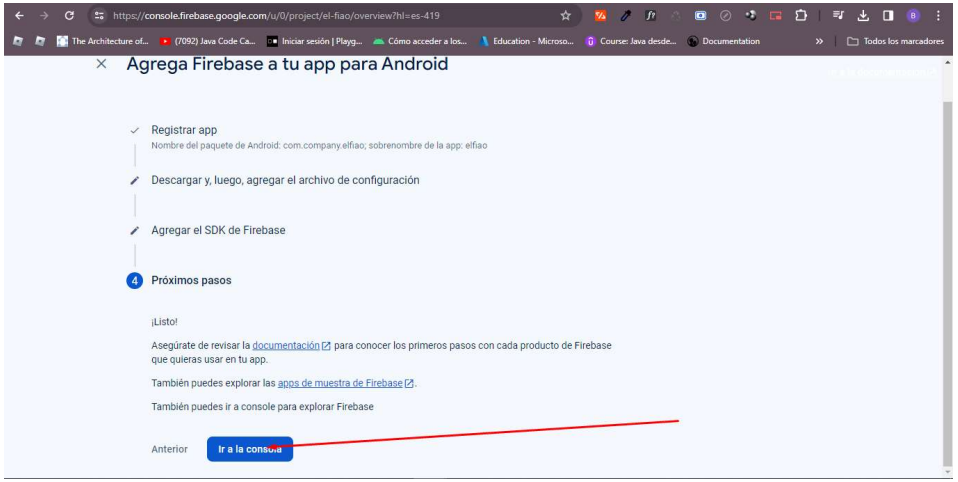
Debes tener ambos pasos ok.



# Conectar Android Studio a Firebase

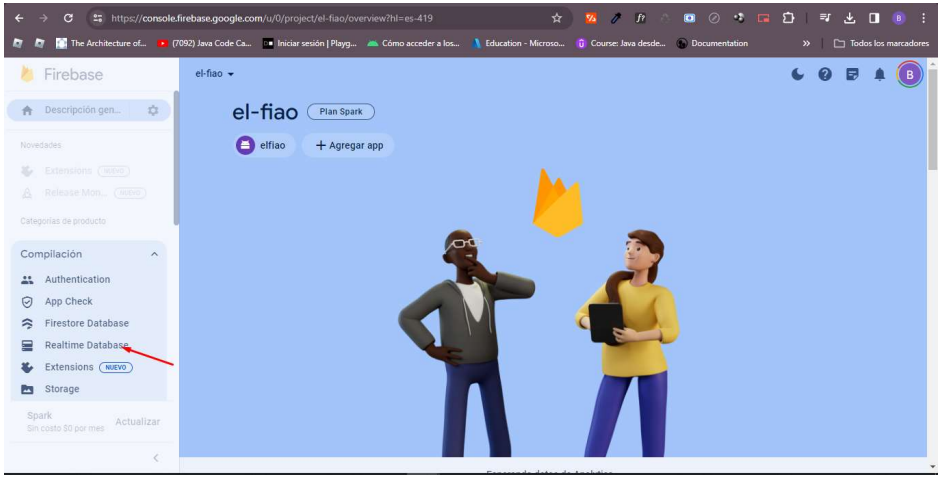
Con esto esta listo el tercer paso en la configuración de la base de datos.

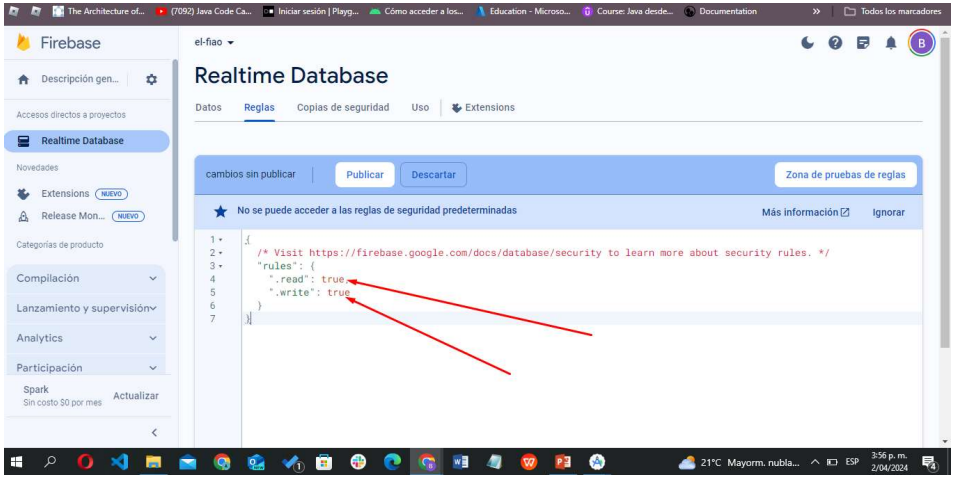


# Conectar Android Studio a Firebase

Ahora vamos a cambiar las reglas de la firebase para poder leer y escribir datos.



# Conectar Android Studio a Firebase

Ahora vamos a cambiar las reglas de la firebase para
poder leer y escribir datos.